Matplotlib

Knowledge-Base Reference

Version 3.11.2

The Figure and Axes model, pyplot versus the object-oriented interface, layouts, ticks and scales, legends, colors and colormaps, text and annotations, styling with rcParams, and performance, plus an API quick reference.

Source: official documentation, https://matplotlib.org/stable/
Content © 2012-2026 The Matplotlib development team (Matplotlib License, PSF-based)
Compiled October 29, 2026

Contents

About this reference **4**

Quick start guide **5**
- A simple example . . . 5
- Parts of a Figure . . . 5
- Types of inputs to plotting functions . . . 7
- Coding styles . . . 8
- Styling Artists . . . 9
- Labelling plots . . . 10
- Axis scales and ticks . . . 11
- Color mapped data . . . 13
- Working with multiple Figures and Axes . . . 13
- More reading . . . 14

Matplotlib application interfaces (pyplot vs. object-oriented) **15**
- Native Matplotlib interfaces . . . 15
- Third-party library ”Data-object” interfaces . . . 18
- Summary . . . 18
- Appendix: ”Axes” interface with data structures . . . 19
- Appendix: ”pylab” interface . . . 19

Introduction to Figures **20**
- Viewing Figures . . . 20
- Creating Figures . . . 25
- Saving Figures . . . 26

Introduction to Axes **27**
- Creating Axes . . . 28
- Axes plotting methods . . . 28
- Axes labelling and annotation . . . 29
- Axes limits, scales, and ticking . . . 29
- Axes layout . . . 30

Arranging multiple Axes in a Figure **31**
- Overview . . . 31
- High-level methods for making grids . . . 32
- Low-level and advanced grid methods . . . 34
- More reading . . . 36

Constrained layout guide **37**
- Simple example . . . 37
- Colorbars . . . 38
- Suptitle . . . 39
- Legends . . . 39
- Padding and spacing . . . 40
- rcParams . . . 41
- Use with GridSpec . . . 41
- Manually setting Axes positions . . . 43
- Grids of fixed aspect-ratio Axes: ”compressed” layout . . . 43
- Manually turning off *constrained layout* . . . 44

Limitations . . . 44
Debugging . . . 45
Notes on the algorithm . . . 46

Axis ticks **48**
Manual location and formats . . . 48
Locators and Formatters . . . 48
Styling ticks (tick parameters) . . . 51
Tick objects . . . 52

Axis scales **53**
loglog and semilogx/y . . . 53
Other built-in scales . . . 54
Optional arguments for scales . . . 54
Arbitrary function scales . . . 55
What is a ”scale”? . . . 55

Legend guide **57**
Controlling the legend entries . . . 57
Creating artists specifically for adding to the legend (aka. Proxy artists) . . . 58
Legend location . . . 58
Multiple legends on the same Axes . . . 59
Legend handlers . . . 60

Specifying colors **63**
Color formats . . . 63
Transparency . . . 64
”CN” color selection . . . 65
Comparison between X11/CSS4 and xkcd colors . . . 66

Choosing colormaps **67**
Overview . . . 67
Classes of colormaps . . . 67
Lightness of Matplotlib colormaps . . . 70
Grayscale conversion . . . 72
Color vision deficiencies . . . 73
References . . . 73

Text in Matplotlib **74**
Basic text commands . . . 74
Labels for x- and y-axis . . . 75
Titles . . . 76
Ticks and ticklabels . . . 77
Legends and annotations . . . 79

Annotations **80**
Annotating data . . . 81
Annotating an Artist . . . 81
Annotating with arrows . . . 81
Placing text annotations relative to data . . . 82
Annotating with boxed text . . . 82
Defining custom box styles . . . 83
Customizing annotation arrows . . . 85
Placing Artist at anchored Axes locations . . . 88
Using an Artist as an annotation . . . 89
Blended coordinate specification . . . 92

Using ConnectionPatch . . . 92
Zoom effect between Axes . . . 93

Customizing with style sheets and rcParams **94**
Runtime rc settings . . . 94
Using style sheets . . . 95
The `matplotlibrc` file . . . 96

Performance **98**
Line segment simplification . . . 98
Marker subsampling . . . 99
Splitting lines into smaller chunks . . . 99
Legends . . . 99
Using the *fast* style . . . 100

API quick reference **101**
pyplot entry points . . . 101
Plotting on an Axes . . . 107
Labels, legends and annotation . . . 121
Figures, layout and colors . . . 127

About this reference

This knowledge-base reference collects core chapters of the official **Matplotlib 3.11.2** documentation (https://matplotlib.org/stable/).

- **Chapters 2 onward** are converted from the project's own documentation source files (the reStructuredText files the website is built from), taken from the `v3.11.2` release tag of github.com/matplotlib/matplotlib. Text and code are the project's; Sphinx-only elements such as cross-reference links, rendered plot output and interactive widgets were simplified for print.
- **The API quick reference** at the end is generated from the docstrings of the installed 3.11.2 package, the same text the official API pages are built from. Each entry is trimmed to signature, parameters, returns, notes and a short excerpt of the notes and examples.

Note on figures. Most Matplotlib guide pages are example scripts; the website runs them when it is built to draw the plots. This PDF includes the full text and code of those scripts and the static diagrams from the documentation source, but not the generated plots. Running a code block yourself reproduces its figure.

Built October 29, 2026 for a personal knowledge base. Selection and assembly by Claude; no content was rewritten.

License. Matplotlib and its documentation are distributed under the Matplotlib License, which is based on the PSF license. Copyright (c) 2012-2026 Matplotlib Development Team; All Rights Reserved. Redistribution with this notice is permitted. Full text: https://matplotlib.org/stable/project/license.html

Chapter sources

- Quick start guide: https://matplotlib.org/stable/users/explain/quick_start.html
- Matplotlib application interfaces (pyplot vs. object-oriented): https://matplotlib.org/stable/users/explain/figure/api_interfaces.html
- Introduction to Figures: https://matplotlib.org/stable/users/explain/figure/figure_intro.html
- Introduction to Axes: https://matplotlib.org/stable/users/explain/axes/axes_intro.html
- Arranging multiple Axes in a Figure: https://matplotlib.org/stable/users/explain/axes/arranging_axes.html
- Constrained layout guide: https://matplotlib.org/stable/users/explain/axes/constrainedlayout_guide.html
- Axis ticks: https://matplotlib.org/stable/users/explain/axes/axes_ticks.html
- Axis scales: https://matplotlib.org/stable/users/explain/axes/axes_scales.html
- Legend guide: https://matplotlib.org/stable/users/explain/axes/legend_guide.html
- Specifying colors: https://matplotlib.org/stable/users/explain/colors/colors.html
- Choosing colormaps: https://matplotlib.org/stable/users/explain/colors/colormaps.html
- Text in Matplotlib: https://matplotlib.org/stable/users/explain/text/text_intro.html
- Annotations: https://matplotlib.org/stable/users/explain/text/annotations.html
- Customizing with style sheets and rcParams: https://matplotlib.org/stable/users/explain/customizing.html
- Performance: https://matplotlib.org/stable/users/explain/artists/performance.html

Quick start guide

Source: https://matplotlib.org/stable/users/explain/quick_start.html

This tutorial covers some basic usage patterns and best practices to help you get started with Matplotlib.

```
import matplotlib.pyplot as plt
import numpy as np
```

A simple example

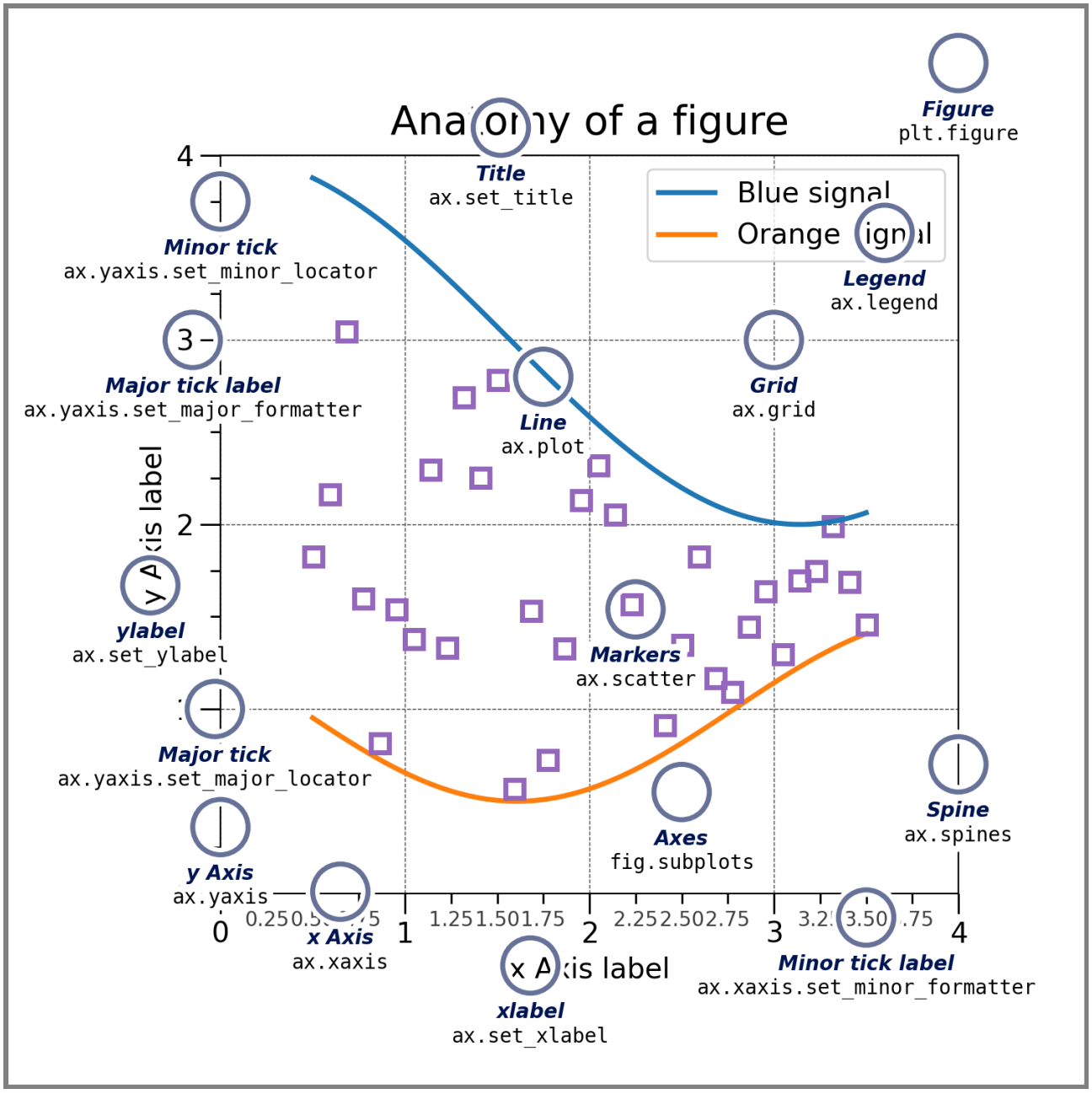
Matplotlib graphs your data on `Figures` (e.g., windows, Jupyter widgets, etc.), each of which can contain one or more `Axes`, an area where points can be specified in terms of x-y coordinates (or theta-r in a polar plot, x-y-z in a 3D plot, etc.). The simplest way of creating a Figure with an Axes is using `pyplot.subplots`. We can then use `Axes.plot` to draw some data on the Axes, and `show` to display the figure:

```
fig, ax = plt.subplots()             # Create a figure containing a single Axes.
ax.plot([1, 2, 3, 4], [1, 4, 2, 3])  # Plot some data on the Axes.
plt.show()                           # Show the figure.
```

Depending on the environment you are working in, `plt.show()` can be left out. This is for example the case with Jupyter notebooks, which automatically show all figures created in a code cell.

Parts of a Figure

Here are the components of a Matplotlib Figure.



`Figure`

The **whole** figure. The Figure keeps track of all the child `Axes`, a group of 'special' Artists (titles, figure legends, colorbars, etc.), and even nested subfigures.

Typically, you'll create a new Figure through one of the following functions:

```
fig = plt.figure()              # an empty figure with no Axes
fig, ax = plt.subplots()        # a figure with a single Axes
fig, axs = plt.subplots(2, 2)  # a figure with a 2x2 grid of Axes
# a figure with one Axes on the left, and two on the right:
fig, axs = plt.subplot_mosaic([['left', 'right_top'],
                               ['left', 'right_bottom']])
```

`subplots()` and `subplot_mosaic` are convenience functions that additionally create Axes objects

inside the Figure, but you can also manually add Axes later on.

For more on Figures, including panning and zooming, see figure-intro.

`Axes`

An Axes is an Artist attached to a Figure that contains a region for plotting data, and usually includes two (or three in the case of 3D) `Axis` objects (be aware of the difference between **Axes** and **Axis**) that provide ticks and tick labels to provide scales for the data in the Axes. Each `Axes` also has a title (set via `set_title`), an x-label (set via `set_xlabel`), and a y-label set via `set_ylabel`).

The `Axes` methods are the primary interface for configuring most parts of your plot (adding data, controlling axis scales and limits, adding labels etc.).

`Axis`

These objects set the scale and limits and generate ticks (the marks on the Axis) and ticklabels (strings labeling the ticks). The location of the ticks is determined by a `Locator` object and the ticklabel strings are formatted by a `Formatter`. The combination of the correct `Locator` and `Formatter` gives very fine control over the tick locations and labels.

`Artist`

Basically, everything visible on the Figure is an Artist (even `Figure`, Axes <.axes.Axes>, and `Axis` objects). This includes `Text` objects, `Line2D` objects, `.collections` objects, `Patch` objects, etc. When the Figure is rendered, all of the Artists are drawn to the **canvas**. Most Artists are tied to an Axes; such an Artist cannot be shared by multiple Axes, or moved from one to another.

Types of inputs to plotting functions

Plotting functions expect `numpy.array` or `numpy.ma.masked_array` as input, or objects that can be passed to `numpy.asarray`. Classes that are similar to arrays ('array-like') such as `pandas` data objects and `numpy.matrix` may not work as intended. Common convention is to convert these to `numpy.array` objects prior to plotting. For example, to convert a `numpy.matrix` :

```
b = np.matrix([[1, 2], [3, 4]])
b_asarray = np.asarray(b)
```

Most methods will also parse a string-indexable object like a *dict*, a structured numpy array, or a `pandas.DataFrame`. Matplotlib allows you to provide the `data` keyword argument and generate plots passing the strings corresponding to the *x* and *y* variables.

```
np.random.seed(19680801)  # seed the random number generator.
data = {'a': np.arange(50),
        'c': np.random.randint(0, 50, 50),
        'd': np.random.randn(50)}
data['b'] = data['a'] + 10 * np.random.randn(50)
data['d'] = np.abs(data['d']) * 100

fig, ax = plt.subplots(figsize=(5, 2.7), layout='constrained')
ax.scatter('a', 'b', c='c', s='d', data=data)
ax.set_xlabel('entry a')
ax.set_ylabel('entry b')
```

Coding styles

The explicit and the implicit interfaces

As noted above, there are essentially two ways to use Matplotlib:

- Explicitly create Figures and Axes, and call methods on them (the ”object-oriented (OO) style”).
- Rely on pyplot to implicitly create and manage the Figures and Axes, and use pyplot functions for plotting.

See api_interfaces for an explanation of the tradeoffs between the implicit and explicit interfaces.

So one can use the OO-style

```
x = np.linspace(0, 2, 100)  # Sample data.

# Note that even in the OO-style, we use ``pyplot.figure`` to create the Figure.
fig, ax = plt.subplots(figsize=(5, 2.7), layout='constrained')
ax.plot(x, x, label='linear')  # Plot some data on the Axes.
ax.plot(x, x**2, label='quadratic')  # Plot more data on the Axes...
ax.plot(x, x**3, label='cubic')  # ... and some more.
ax.set_xlabel('x label')  # Add an x-label to the Axes.
ax.set_ylabel('y label')  # Add a y-label to the Axes.
ax.set_title("Simple Plot")  # Add a title to the Axes.
ax.legend()  # Add a legend.
```

or the pyplot-style:

```
x = np.linspace(0, 2, 100)  # Sample data.

plt.figure(figsize=(5, 2.7), layout='constrained')
plt.plot(x, x, label='linear')  # Plot some data on the (implicit) Axes.
plt.plot(x, x**2, label='quadratic')  # etc.
plt.plot(x, x**3, label='cubic')
plt.xlabel('x label')
plt.ylabel('y label')
plt.title("Simple Plot")
plt.legend()
```

(In addition, there is a third approach, for the case when embedding Matplotlib in a GUI application, which completely drops pyplot, even for figure creation. See the corresponding section in the gallery for more info: user_interfaces.)

Matplotlib's documentation and examples use both the OO and the pyplot styles. In general, we suggest using the OO style, particularly for complicated plots, and functions and scripts that are intended to be reused as part of a larger project. However, the pyplot style can be very convenient for quick interactive work.

> Note
> You may find older examples that use the `pylab` interface, via `from pylab import *`. This approach is strongly deprecated.

Making a helper functions

If you need to make the same plots over and over again with different data sets, or want to easily wrap Matplotlib methods, use the recommended signature function below.

```
def my_plotter(ax, data1, data2, param_dict):
    """
    A helper function to make a graph.
    """
    out = ax.plot(data1, data2, **param_dict)
    return out
```

which you would then use twice to populate two subplots:

```
data1, data2, data3, data4 = np.random.randn(4, 100)  # make 4 random data sets
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(5, 2.7))
my_plotter(ax1, data1, data2, {'marker': 'x'})
my_plotter(ax2, data3, data4, {'marker': 'o'})
```

Note that if you want to install these as a python package, or any other customizations you could use one of the many templates on the web; Matplotlib has one at mpl-cookiecutter

Styling Artists

Most plotting methods have styling options for the Artists, accessible either when a plotting method is called, or from a "setter" on the Artist. In the plot below we manually set the *color*, *linewidth*, and *linestyle* of the Artists created by `plot`, and we set the linestyle of the second line after the fact with `set_linestyle`.

```
fig, ax = plt.subplots(figsize=(5, 2.7))
x = np.arange(len(data1))
ax.plot(x, np.cumsum(data1), color='blue', linewidth=3, linestyle='--')
l, = ax.plot(x, np.cumsum(data2), color='orange', linewidth=2)
l.set_linestyle(':')
```

Colors

Matplotlib has a very flexible array of colors that are accepted for most Artists; see allowable color definitions for a list of specifications. Some Artists will take multiple colors. i.e. for a `scatter` plot, the edge of the markers can be different colors from the interior:

```
fig, ax = plt.subplots(figsize=(5, 2.7))
ax.scatter(data1, data2, s=50, facecolor='C0', edgecolor='k')
```

Linewidths, linestyles, and markersizes

Line widths are typically in typographic points (1 pt = 1/72 inch) and available for Artists that have stroked lines. Similarly, stroked lines can have a linestyle. See the `linestyles example </gallery/lines_bars_and_markers/linestyles>`.

Marker size depends on the method being used. `plot` specifies markersize in points, and is generally the "diameter" or width of the marker. `scatter` specifies markersize as approximately proportional to the visual area of the marker. There is an array of markerstyles available as string codes (see `markers`), or users can define their own `MarkerStyle` (see /gallery/lines_bars_and_markers/marker_reference):

```
fig, ax = plt.subplots(figsize=(5, 2.7))
ax.plot(data1, 'o', label='data1')
ax.plot(data2, 'd', label='data2')
ax.plot(data3, 'v', label='data3')
```

```
ax.plot(data4, 's', label='data4')
ax.legend()
```

Labelling plots

Axes labels and text

`set_xlabel`, `set_ylabel`, and `set_title` are used to add text in the indicated locations (see text_intro for more discussion). Text can also be directly added to plots using `text`:

```
mu, sigma = 115, 15
x = mu + sigma * np.random.randn(10000)
fig, ax = plt.subplots(figsize=(5, 2.7), layout='constrained')
# the histogram of the data
n, bins, patches = ax.hist(x, 50, density=True, facecolor='C0', alpha=0.75)

ax.set_xlabel('Length [cm]')
ax.set_ylabel('Probability')
ax.set_title('Aardvark lengths\n (not really)')
ax.text(75, .025, r'$\mu=115,\ \sigma=15$')
ax.axis([55, 175, 0, 0.03])
ax.grid(True)
```

All of the `text` functions return a `matplotlib.text.Text` instance. Just as with lines above, you can customize the properties by passing keyword arguments into the text functions:

```
t = ax.set_xlabel('my data', fontsize=14, color='red')
```

These properties are covered in more detail in text_props.

Using mathematical expressions in text

Matplotlib accepts TeX equation expressions in any text expression. For example to write the expression $\sigma_i = 15$ in the title, you can write a TeX expression surrounded by dollar signs:

```
ax.set_title(r'$\sigma_i=15$')
```

where the `r` preceding the title string signifies that the string is a *raw* string and not to treat backslashes as python escapes. Matplotlib has a built-in TeX expression parser and layout engine, and ships its own math fonts – for details see mathtext. You can also use LaTeX directly to format your text and incorporate the output directly into your display figures or saved postscript – see usetex.

Annotations

We can also annotate points on a plot, often by connecting an arrow pointing to *xy*, to a piece of text at *xytext*:

```
fig, ax = plt.subplots(figsize=(5, 2.7))

t = np.arange(0.0, 5.0, 0.01)
s = np.cos(2 * np.pi * t)
line, = ax.plot(t, s, lw=2)

ax.annotate('local max', xy=(2, 1), xytext=(3, 1.5),
            arrowprops=dict(facecolor='black', shrink=0.05))

ax.set_ylim(-2, 2)
```

In this basic example, both *xy* and *xytext* are in data coordinates. There are a variety of other coordinate systems one can choose -- see annotations-tutorial and plotting-guide-annotation for details. More examples also can be found in /gallery/-text_labels_and_annotations/annotation_demo.

Legends

Often we want to identify lines or markers with a `Axes.legend`:

```
fig, ax = plt.subplots(figsize=(5, 2.7))
ax.plot(np.arange(len(data1)), data1, label='data1')
ax.plot(np.arange(len(data2)), data2, label='data2')
ax.plot(np.arange(len(data3)), data3, 'd', label='data3')
ax.legend()
```

Legends in Matplotlib are quite flexible in layout, placement, and what Artists they can represent. They are discussed in detail in legend_guide.

Axis scales and ticks

Each Axes has two (or three) `Axis` objects representing the x- and y-axis. These control the *scale* of the Axis, the tick *locators* and the tick *formatters*. Additional Axes can be attached to display further Axis objects.

Scales

In addition to the linear scale, Matplotlib supplies non-linear scales, such as a log-scale. Since log-scales are used so much there are also direct methods like `loglog`, `semilogx`, and `semilogy`. There are a number of scales (see /gallery/scales/scales for other examples). Here we set the scale manually:

```
fig, axs = plt.subplots(1, 2, figsize=(5, 2.7), layout='constrained')
xdata = np.arange(len(data1))  # make an ordinal for this
data = 10**data1
axs[0].plot(xdata, data)

axs[1].set_yscale('log')
axs[1].plot(xdata, data)
```

The scale sets the mapping from data values to spacing along the Axis. This happens in both directions, and gets combined into a *transform*, which is the way that Matplotlib maps from data coordinates to Axes, Figure, or screen coordinates. See transforms_tutorial.

Tick locators and formatters

Each Axis has a tick *locator* and *formatter* that choose where along the Axis objects to put tick marks. A simple interface to this is `set_xticks`:

```
fig, axs = plt.subplots(2, 1, layout='constrained')
axs[0].plot(xdata, data1)
axs[0].set_title('Automatic ticks')

axs[1].plot(xdata, data1)
axs[1].set_xticks(np.arange(0, 100, 30), ['zero', '30', 'sixty', '90'])
```

```
axs[1].set_yticks([-1.5, 0, 1.5])  # note that we don't need to specify labels
axs[1].set_title('Manual ticks')
```

Different scales can have different locators and formatters; for instance the log-scale above uses `LogLocator` and `LogFormatter`. See /gallery/ticks/tick-locators and /gallery/ticks/tick-formatters for other formatters and locators and information for writing your own.

Plotting dates and strings

Matplotlib can handle plotting arrays of dates and arrays of strings, as well as floating point numbers. These get special locators and formatters as appropriate. For dates:

```
from matplotlib.dates import ConciseDateFormatter

fig, ax = plt.subplots(figsize=(5, 2.7), layout='constrained')
dates = np.arange(np.datetime64('2021-11-15'), np.datetime64('2021-12-25'),
                  np.timedelta64(1, 'h'))
data = np.cumsum(np.random.randn(len(dates)))
ax.plot(dates, data)
ax.xaxis.set_major_formatter(ConciseDateFormatter(ax.xaxis.get_major_locator()))
```

For more information see the date examples (e.g. /gallery/ticks/date)

For strings, we get categorical plotting (see: /gallery/lines_bars_and_markers/categorical_variables).

```
fig, ax = plt.subplots(figsize=(5, 2.7), layout='constrained')
categories = ['turnips', 'rutabaga', 'cucumber', 'pumpkins']

ax.bar(categories, np.random.rand(len(categories)))
```

One caveat about categorical plotting is that some methods of parsing text files return a list of strings, even if the strings all represent numbers or dates. If you pass 1000 strings, Matplotlib will think you meant 1000 categories and will add 1000 ticks to your plot!

Additional Axis objects

Plotting data of different magnitude in one chart may require an additional y-axis. Such an Axis can be created by using `twinx` to add a new Axes with an invisible x-axis and a y-axis positioned at the right (analogously for `twiny`). See /gallery/subplots_axes_and_figures/two_scales for another example.

Similarly, you can add a `secondary_xaxis` or `secondary_yaxis` having a different scale than the main Axis to represent the data in different scales or units. See /gallery/subplots_axes_and_figures/secondary_axis for further examples.

```
fig, (ax1, ax3) = plt.subplots(1, 2, figsize=(7, 2.7), layout='constrained')
l1, = ax1.plot(t, s)
ax2 = ax1.twinx()
l2, = ax2.plot(t, range(len(t)), 'C1')
ax2.legend([l1, l2], ['Sine (left)', 'Straight (right)'])

ax3.plot(t, s)
ax3.set_xlabel('Angle [rad]')
ax4 = ax3.secondary_xaxis('top', (np.rad2deg, np.deg2rad))
ax4.set_xlabel('Angle [°]')
```

Color mapped data

Often we want to have a third dimension in a plot represented by colors in a colormap. Matplotlib has a number of plot types that do this:

```
from matplotlib.colors import LogNorm

X, Y = np.meshgrid(np.linspace(-3, 3, 128), np.linspace(-3, 3, 128))
Z = (1 - X/2 + X**5 + Y**3) * np.exp(-X**2 - Y**2)

fig, axs = plt.subplots(2, 2, layout='constrained')
pc = axs[0, 0].pcolormesh(X, Y, Z, vmin=-1, vmax=1, cmap='RdBu_r')
fig.colorbar(pc, ax=axs[0, 0])
axs[0, 0].set_title('pcolormesh()')

co = axs[0, 1].contourf(X, Y, Z, levels=np.linspace(-1.25, 1.25, 11))
fig.colorbar(co, ax=axs[0, 1])
axs[0, 1].set_title('contourf()')

pc = axs[1, 0].imshow(Z**2 * 100, cmap='plasma', norm=LogNorm(vmin=0.01, vmax=100))
fig.colorbar(pc, ax=axs[1, 0], extend='both')
axs[1, 0].set_title('imshow() with LogNorm()')

pc = axs[1, 1].scatter(data1, data2, c=data3, cmap='RdBu_r')
fig.colorbar(pc, ax=axs[1, 1], extend='both')
axs[1, 1].set_title('scatter()')
```

Colormaps

These are all examples of Artists that derive from `ScalarMappable` objects. They all can set a linear mapping between *vmin* and *vmax* into the colormap specified by *cmap*. Matplotlib has many colormaps to choose from (colormaps) you can make your own (colormap-manipulation) or download as third-party packages.

Normalizations

Sometimes we want a non-linear mapping of the data to the colormap, as in the `LogNorm` example above. We do this by supplying the ScalarMappable with the *norm* argument instead of *vmin* and *vmax*. More normalizations are shown at colormapnorms.

Colorbars

Adding a `colorbar` gives a key to relate the color back to the underlying data. Colorbars are figure-level Artists, and are attached to a ScalarMappable (where they get their information about the norm and colormap) and usually steal space from a parent Axes. Placement of colorbars can be complex: see colorbar_placement for details. You can also change the appearance of colorbars with the *extend* keyword to add arrows to the ends, and *shrink* and *aspect* to control the size. Finally, the colorbar will have default locators and formatters appropriate to the norm. These can be changed as for other Axis objects.

Working with multiple Figures and Axes

You can open multiple Figures with multiple calls to `fig = plt.figure()` or `fig2, ax = plt.subplots()`. By keeping the object references you can add Artists to either Figure.

Multiple Axes can be added a number of ways, but the most basic is `plt.subplots()` as used above. One can achieve more complex layouts, with Axes objects spanning columns or rows, using `subplot_mosaic`.

```
fig, axd = plt.subplot_mosaic([['upleft', 'right'],
                               ['lowleft', 'right']], layout='constrained')
axd['upleft'].set_title('upleft')
axd['lowleft'].set_title('lowleft')
axd['right'].set_title('right')
```

Matplotlib has quite sophisticated tools for arranging Axes: See arranging_axes and mosaic.

More reading

For more plot types see Plot types and the API reference, in particular the Axes API.

Matplotlib application interfaces (pyplot vs. object-oriented)

Source: https://matplotlib.org/stable/users/explain/figure/api_interfaces.html

Matplotlib has two major application interfaces, or styles of using the library:

- An explicit "Axes" interface that uses methods on a Figure or Axes object to create other Artists, and build a visualization step by step. This has also been called an "object-oriented" interface.
- An implicit "pyplot" interface that keeps track of the last Figure and Axes created, and adds Artists to the object it thinks the user wants.

In addition, a number of downstream libraries (like `pandas` and xarray) offer a `plot` method implemented directly on their data classes so that users can call `data.plot()`.

The difference between these interfaces can be a bit confusing, particularly given snippets on the web that use one or the other, or sometimes multiple interfaces in the same example. Here we attempt to point out how the "pyplot" and downstream interfaces relate to the explicit "Axes" interface to help users better navigate the library.

Native Matplotlib interfaces

The explicit "Axes" interface

The "Axes" interface is how Matplotlib is implemented, and many customizations and fine-tuning end up being done at this level.

This interface works by instantiating an instance of a `Figure` class (`fig` below), using a `subplots` method (or similar) on that object to create one or more `Axes` objects (`ax` below), and then calling drawing methods on the Axes (`plot` in this example):

```
import matplotlib.pyplot as plt

fig = plt.figure()
ax = fig.subplots()
ax.plot([1, 2, 3, 4], [0, 0.5, 1, 0.2])
```

We call this an "explicit" interface because each object is explicitly referenced, and used to make the next object. Keeping references to the objects is very flexible, and allows us to customize the objects after they are created, but before they are displayed.

The implicit "pyplot" interface

The `pyplot` module shadows most of the `Axes` plotting methods to give the equivalent of the above, where the creation of the Figure and Axes is done for the user:

```
import matplotlib.pyplot as plt

plt.plot([1, 2, 3, 4], [0, 0.5, 1, 0.2])
```

This can be convenient, particularly when doing interactive work or simple scripts. A reference to the current Figure can be retrieved using `gcf` and to the current Axes by `gca`. The `pyplot`

module retains a list of Figures, and each Figure retains a list of Axes on the figure for the user so that the following:

```
import matplotlib.pyplot as plt

plt.subplot(1, 2, 1)
plt.plot([1, 2, 3], [0, 0.5, 0.2])

plt.subplot(1, 2, 2)
plt.plot([3, 2, 1], [0, 0.5, 0.2])
```

is equivalent to:

```
import matplotlib.pyplot as plt

plt.subplot(1, 2, 1)
ax = plt.gca()
ax.plot([1, 2, 3], [0, 0.5, 0.2])

plt.subplot(1, 2, 2)
ax = plt.gca()
ax.plot([3, 2, 1], [0, 0.5, 0.2])
```

In the explicit interface, this would be:

```
import matplotlib.pyplot as plt

fig, axs = plt.subplots(1, 2)
axs[0].plot([1, 2, 3], [0, 0.5, 0.2])
axs[1].plot([3, 2, 1], [0, 0.5, 0.2])
```

Translating between the Axes interface and the pyplot interface

You may find either interface in existing code, and unfortunately sometimes even mixtures. This section describes the patterns for specific operations in both interfaces and how to translate from one to the other.

- Creating figures is the same for both interfaces: Use the respective `pyplot` functions `plt.figure()`, `plt.subplots()`, `plt.subplot_mosaic()`. For the Axes interface, you typically store the created Figure (and possibly Axes) in variables for later use. When using the pyplot interface, these values are typically not stored. Example:
  - Axes: `fig, ax = plt.subplots()`
  - pyplot: `plt.subplots()`
- ”Plotting” functions, i.e. functions that add data, are named the same and have identical parameters on the Axes and in pyplot. Example:
  - Axes: `ax.plot(x, y)`
  - pyplot: `plt.plot(x, y)`
- Functions that retrieve properties are named like the property in pyplot and are prefixed with `get_` on the Axes. Example:
  - Axes: `label = ax.get_xlabel()`
  - pyplot: `label = plt.xlabel()`
- Functions that set properties are named like the property in pyplot and are prefixed with `set_` on the Axes. Example:
  - Axes: `ax.set_xlabel("time")`
  - pyplot: `plt.xlabel("time")`

Here is a short summary of the examples again as a side-by-side comparison:

Operation	Axes interface	pyplot interface
Creating figures	`fig, ax = plt.subplots()`	`plt.subplots()`
Plotting data	`ax.plot(x, y)`	`plt.plot(x, y)`
Getting properties	`label = ax.get_xlabel()`	`label = plt.xlabel()`
Setting properties	`ax.set_xlabel("time")`	`plt.xlabel("time")`

Why be explicit?

What happens if you have to backtrack, and operate on an old axes that is not referenced by `plt.gca()`? One simple way is to call `subplot` again with the same arguments. However, that quickly becomes inelegant. You can also inspect the Figure object and get its list of Axes objects, however, that can be misleading (colorbars are Axes too!). The best solution is probably to save a handle to every Axes you create, but if you do that, why not simply create all the Axes objects at the start?

The first approach is to call `plt.subplot` again:

```
import matplotlib.pyplot as plt

plt.subplot(1, 2, 1)
plt.plot([1, 2, 3], [0, 0.5, 0.2])

plt.subplot(1, 2, 2)
plt.plot([3, 2, 1], [0, 0.5, 0.2])

plt.suptitle('Implicit Interface: re-call subplot')

for i in range(1, 3):
    plt.subplot(1, 2, i)
    plt.xlabel('Boo')
```

The second is to save a handle:

```
import matplotlib.pyplot as plt

axs = []
ax = plt.subplot(1, 2, 1)
axs += [ax]
plt.plot([1, 2, 3], [0, 0.5, 0.2])

ax = plt.subplot(1, 2, 2)
axs += [ax]
plt.plot([3, 2, 1], [0, 0.5, 0.2])

plt.suptitle('Implicit Interface: save handles')

for i in range(2):
    plt.sca(axs[i])
    plt.xlabel('Boo')
```

However, the recommended way would be to be explicit from the outset:

```
import matplotlib.pyplot as plt

fig, axs = plt.subplots(1, 2)
axs[0].plot([1, 2, 3], [0, 0.5, 0.2])
axs[1].plot([3, 2, 1], [0, 0.5, 0.2])
fig.suptitle('Explicit Interface')
```

```
for i in range(2):
    axs[i].set_xlabel('Boo')
```

Third-party library "Data-object" interfaces

Some third party libraries have chosen to implement plotting for their data objects, e.g. `data.plot()`, is seen in `pandas`, xarray, and other third-party libraries. For illustrative purposes, a downstream library may implement a simple data container that has `x` and `y` data stored together, and then implements a `plot` method:

```
import matplotlib.pyplot as plt

# supplied by downstream library:
class DataContainer:

    def __init__(self, x, y):
        """
        Proper docstring here!
        """
        self._x = x
        self._y = y

    def plot(self, ax=None, **kwargs):
        if ax is None:
            ax = plt.gca()
        ax.plot(self._x, self._y, **kwargs)
        ax.set_title('Plotted from DataClass!')
        return ax


# what the user usually calls:
data = DataContainer([0, 1, 2, 3], [0, 0.2, 0.5, 0.3])
data.plot()
```

So the library can hide all the nitty-gritty from the user, and can make a visualization appropriate to the data type, often with good labels, choices of colormaps, and other convenient features.

In the above, however, we may not have liked the title the library provided. Thankfully, they pass us back the Axes from the `plot()` method, and understanding the explicit Axes interface, we could call: `ax.set_title('My preferred title')` to customize the title.

Many libraries also allow their `plot` methods to accept an optional *ax* argument. This allows us to place the visualization in an Axes that we have placed and perhaps customized.

Summary

Overall, it is useful to understand the explicit "Axes" interface since it is the most flexible and underlies the other interfaces. A user can usually figure out how to drop down to the explicit interface and operate on the underlying objects. While the explicit interface can be a bit more verbose to setup, complicated plots will often end up simpler than trying to use the implicit "pyplot" interface.

> Note
> It is sometimes confusing to people that we import `pyplot` for both interfaces. Currently, the `pyplot` module implements the "pyplot" interface, but it also provides top-level Figure and

Axes creation methods, and ultimately spins up the graphical user interface, if one is being used. So `pyplot` is still needed regardless of the interface chosen.

Similarly, the declarative interfaces provided by partner libraries use the objects accessible by the "Axes" interface, and often accept these as arguments or pass them back from methods. It is usually essential to use the explicit "Axes" interface to perform any customization of the default visualization, or to unpack the data into NumPy arrays and pass directly to Matplotlib.

Appendix: "Axes" interface with data structures

Most `Axes` methods allow yet another API addressing by passing a *data* object to the method and specifying the arguments as strings:

```
import matplotlib.pyplot as plt

data = {'xdat': [0, 1, 2, 3], 'ydat': [0, 0.2, 0.4, 0.1]}
fig, ax = plt.subplots(figsize=(2, 2))
ax.plot('xdat', 'ydat', data=data)
```

Appendix: "pylab" interface

There is one further interface that is highly discouraged, and that is to basically do `from matplotlib.pylab import *`. This imports all the functions from `matplotlib.pyplot`, `numpy`, `numpy.fft`, `numpy.linalg`, and `numpy.random`, and some additional functions into the global namespace.

Such a pattern is considered bad practice in modern python, as it clutters the global namespace. Even more severely, in the case of `pylab`, this will overwrite some builtin functions (e.g. the builtin `sum` will be replaced by `numpy.sum`), which can lead to unexpected behavior.

Introduction to Figures

Source: https://matplotlib.org/stable/users/explain/figure/figure_intro.html

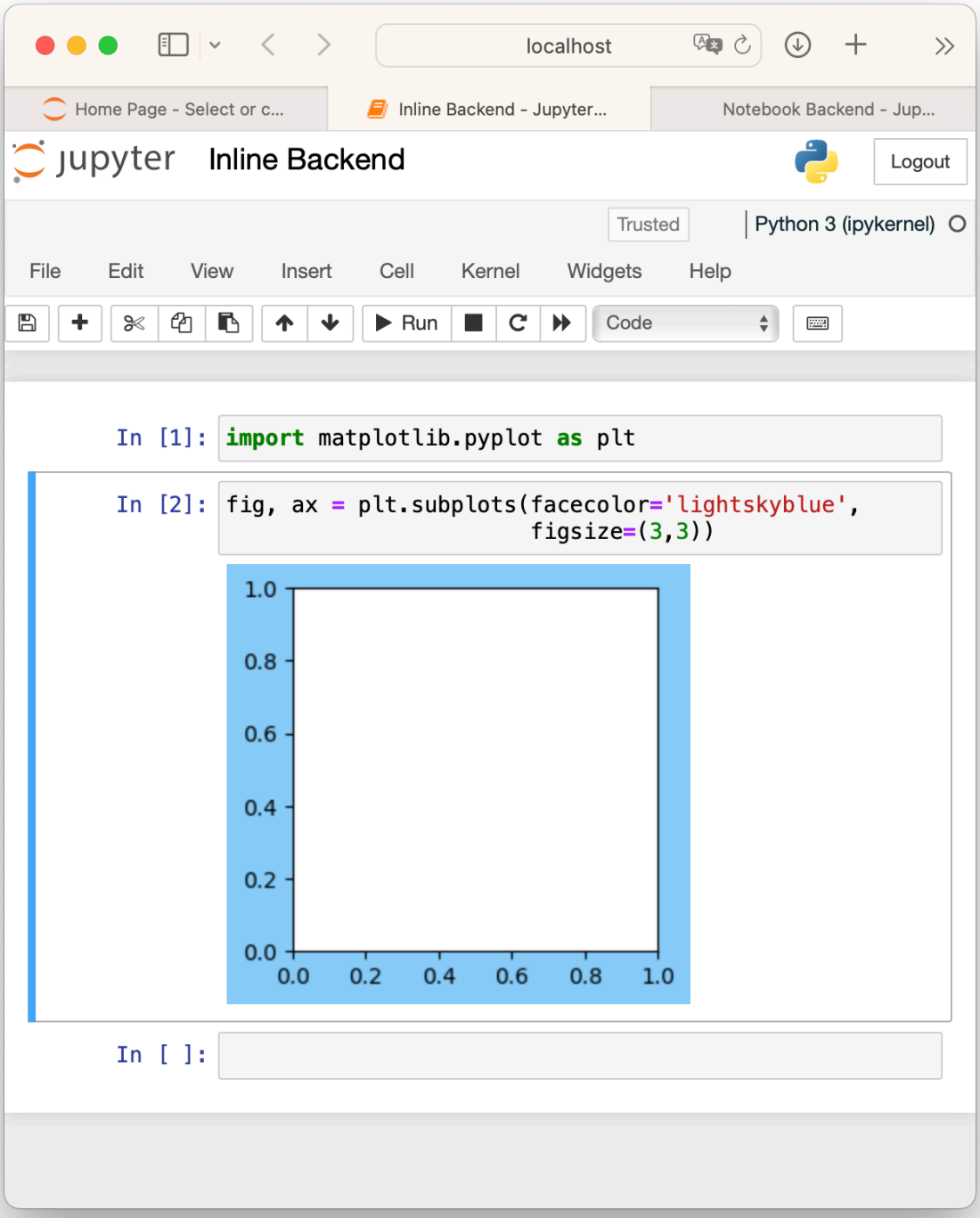
```
fig = plt.figure(figsize=(2, 2), facecolor='lightskyblue',
                 layout='constrained')
fig.suptitle('Figure')
ax = fig.add_subplot()
ax.set_title('Axes', loc='left', fontstyle='oblique', fontsize='medium')
```

When looking at Matplotlib visualization, you are almost always looking at Artists placed on a `Figure`. In the example above, the figure is the blue region and `add_subplot` has added an `Axes` artist to the `Figure` (see figure_parts). A more complicated visualization can add multiple Axes to the Figure, colorbars, legends, annotations, and the Axes themselves can have multiple Artists added to them (e.g. `ax.plot` or `ax.imshow`).

Viewing Figures

We will discuss how to create Figures in more detail below, but first it is helpful to understand how to view a Figure. This varies based on how you are using Matplotlib, and what Backend you are using.

Notebooks and IDEs

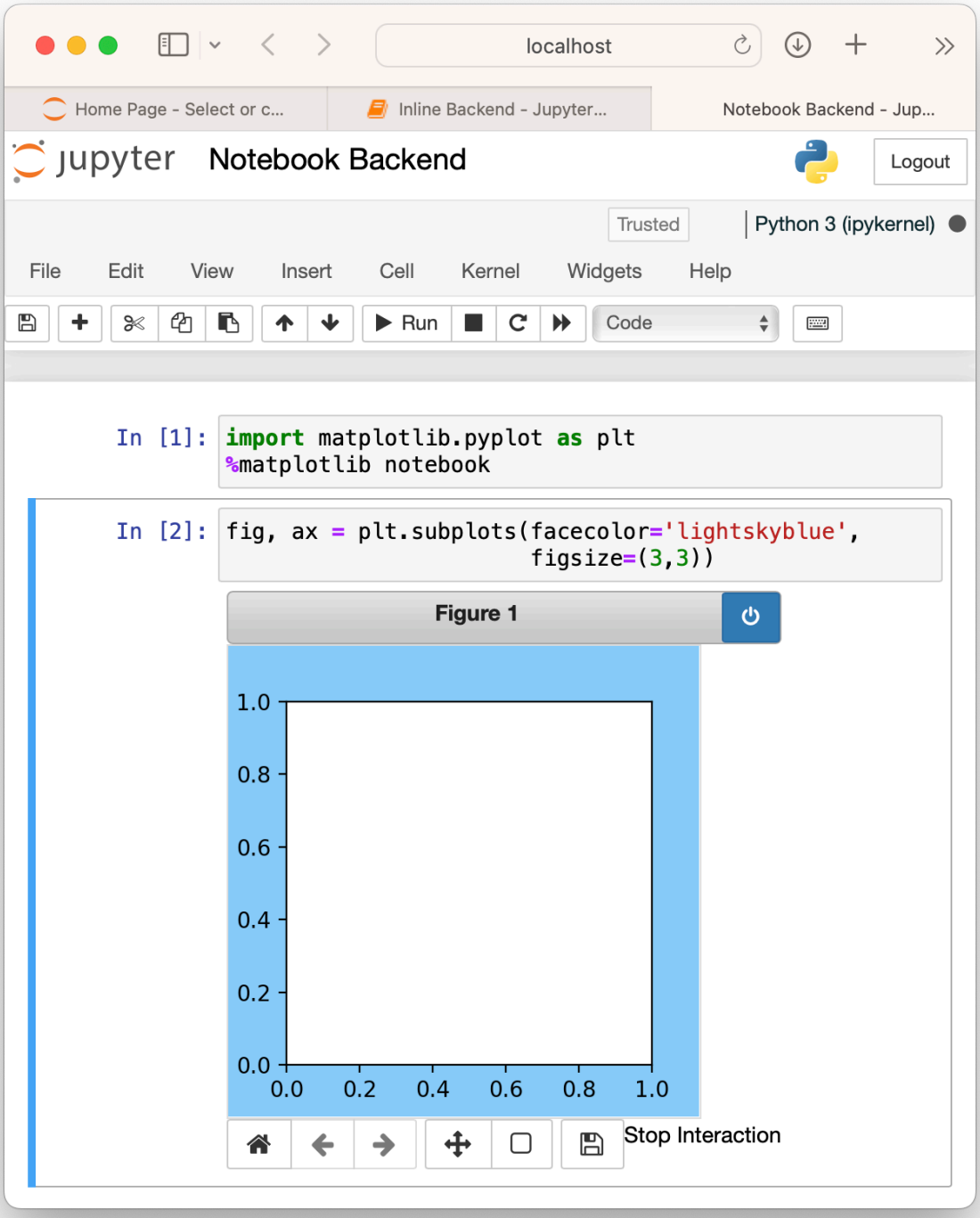


Screenshot of a Jupyter Notebook, with a figure generated via the default inline backend.

If you are using a Notebook (e.g. Jupyter) or an IDE that renders Notebooks (PyCharm, VSCode, etc), then they have a backend that will render the Matplotlib Figure when a code cell is ex-

ecuted. The default Jupyter backend (`%matplotlib inline`) creates static plots that by default trim or expand the figure size to have a tight box around Artists added to the Figure (see saving_figures, below). For interactive plots in Jupyter you will need to use an ipython "magic" like `%matplotlib widget` for the ipympl backend in `jupyter lab` or `notebook>=7`, or `%matplotlib notebook` for the Matplotlib notebook in `notebook<7` or `nbclassic`.

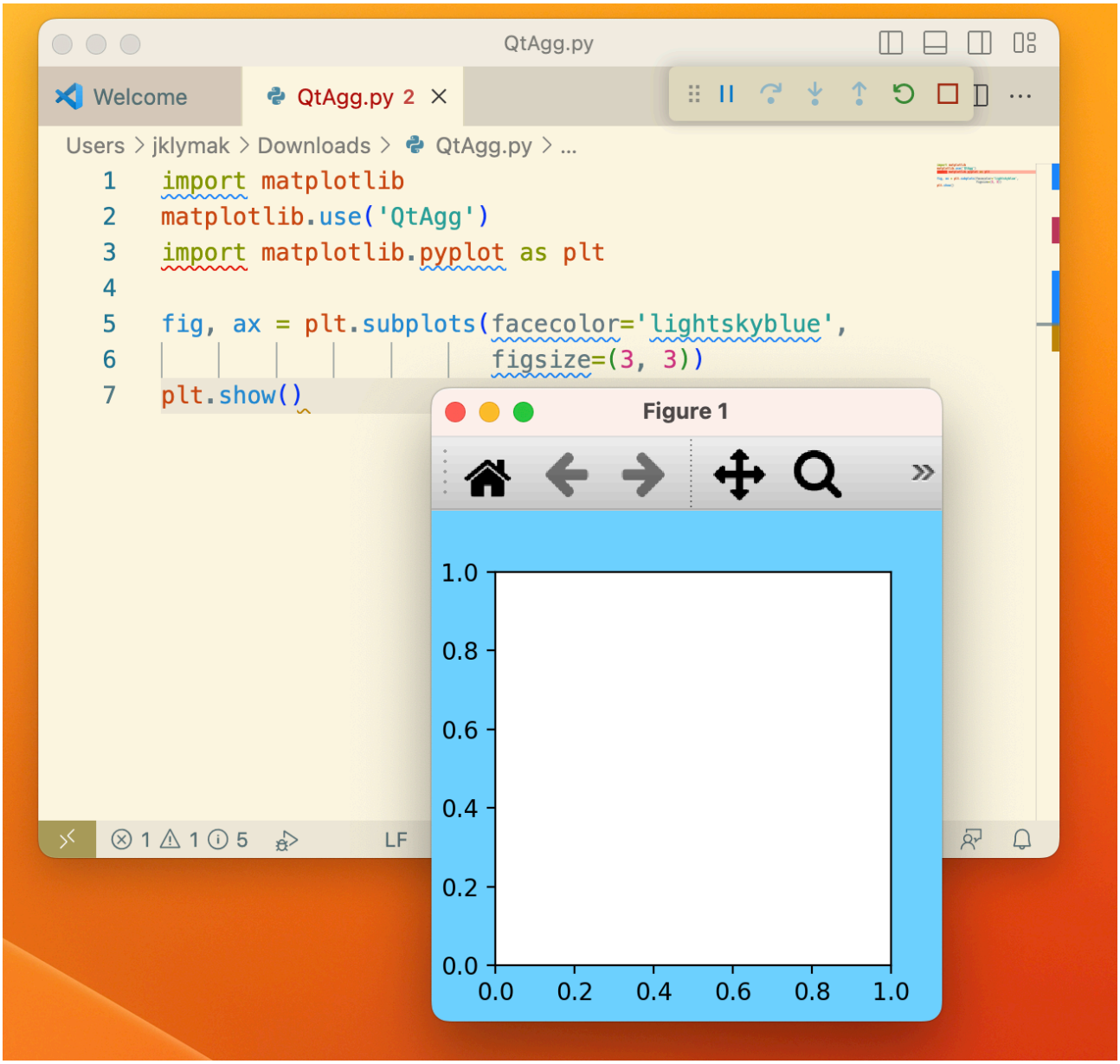
> Note
> The ipympl backend is in a separate package, see Installing ipympl.



Screenshot of a Jupyter Notebook with an interactive figure generated via the `%matplotlib notebook` magic. Users should also try the similar widget backend if using JupyterLab.

Standalone scripts and interactive use

If the user is on a client with a windowing system, there are a number of Backends that can be used to render the Figure to the screen, usually using a Python Qt, Tk, or Wx toolkit, or the native MacOS backend. These are typically chosen either in the user's `matplotlibrc <customizing-with-matplotlibrc-files>`, or by calling, for example, `matplotlib.use('QtAgg')` at the beginning of a session or script.



Screenshot of a Figure generated via a python script and shown using the QtAgg backend.

When run from a script, or interactively (e.g. from an IPython shell) the Figure will not be shown until we call `plt.show()`. The Figure will appear in a new GUI window, and usually will have a toolbar with Zoom, Pan, and other tools for interacting with the Figure. By default, `plt.show()` blocks further interaction from the script or shell until the Figure window is closed, though that can be toggled off for some purposes. For more details, please see controlling-interactive.

Note that if you are on a client that does not have access to a windowing system, the Figure will fallback to being drawn using the ”Agg” backend, and cannot be viewed, though it can be saved.

Creating Figures

By far the most common way to create a figure is using the pyplot interface. As noted in api_interfaces, the pyplot interface serves two purposes. One is to spin up the Backend and keep track of GUI windows. The other is a global state for Axes and Artists that allow a short-form API to plotting methods. In the example above, we use pyplot for the first purpose, and create the Figure object, `fig`. As a side effect `fig` is also added to pyplot's global state, and can be accessed via `gcf`.

Users typically want an Axes or a grid of Axes when they create a Figure, so in addition to `figure`, there are convenience methods that return both a Figure and some Axes. A simple grid of Axes can be achieved with `pyplot.subplots` (which simply wraps `Figure.subplots`):

```
fig, axs = plt.subplots(2, 2, figsize=(4, 3), layout='constrained')
```

More complex grids can be achieved with `pyplot.subplot_mosaic` (which wraps `Figure.subplot_mosaic`):

```
fig, axs = plt.subplot_mosaic([['A', 'right'], ['B', 'right']],
                              figsize=(4, 3), layout='constrained')
for ax_name, ax in axs.items():
    ax.text(0.5, 0.5, ax_name, ha='center', va='center')
```

Sometimes we want to have a nested layout in a Figure, with two or more sets of Axes that do not share the same subplot grid. We can use `add_subfigure` or `subfigures` to create virtual figures inside a parent Figure; see /gallery/subplots_axes_and_figures/subfigures for more details.

```
fig = plt.figure(layout='constrained', facecolor='lightskyblue')
fig.suptitle('Figure')
figL, figR = fig.subfigures(1, 2)
figL.set_facecolor('thistle')
axL = figL.subplots(2, 1, sharex=True)
axL[1].set_xlabel('x [m]')
figL.suptitle('Left subfigure')
figR.set_facecolor('paleturquoise')
axR = figR.subplots(1, 2, sharey=True)
axR[0].set_title('Axes 1')
figR.suptitle('Right subfigure')
```

It is possible to directly instantiate a `Figure` instance without using the pyplot interface. This is usually only necessary if you want to create your own GUI application or service that you do not want carrying the pyplot global state. See the embedding examples in user_interfaces for examples of how to do this.

Figure options

There are a few options available when creating figures. The Figure size on the screen is set by *figsize* and *dpi*. *figsize* is the `(width, height)` of the Figure in inches (or, if preferred, units of 72 typographic points). *dpi* are how many pixels per inch the figure will be rendered at. To make your Figures appear on the screen at the physical size you requested, you should set *dpi* to the same *dpi* as your graphics system. Note that many graphics systems now use a ”dpi ratio” to specify how many screen pixels are used to represent a graphics pixel. Matplotlib applies the dpi ratio to the *dpi* passed to the figure to make it have higher resolution, so you should pass the lower number to the figure.

The *facecolor*, *edgecolor*, *linewidth*, and *frameon* options all change the appearance of the figure in expected ways, with *frameon* making the figure transparent if set to *False*.

Finally, the user can specify a layout engine for the figure with the *layout* parameter. Currently

Matplotlib supplies "constrained", "compressed" and "tight" layout engines. These rescale axes inside the Figure to prevent overlap of ticklabels, and try and align axes, and can save significant manual adjustment of artists on a Figure for many common cases.

Adding Artists

The `Figure` class has a number of methods for adding artists to a `Figure` or a `SubFigure`. By far the most common are to add Axes of various configurations (`add_axes`, `add_subplot`, `subplots`, `subplot_mosaic`) and subfigures (`subfigures`). Colorbars are added to Axes or group of Axes at the Figure level (`colorbar`). It is also possible to have a Figure-level legend (`legend`). Other Artists include figure-wide labels (`suptitle`, `supxlabel`, `supylabel`) and text (`text`). Finally, low-level Artists can be added directly using `add_artist` usually with care being taken to use the appropriate transform. Usually these include `Figure.transFigure` which ranges from 0 to 1 in each direction, and represents the fraction of the current Figure size, or `Figure.dpi_scale_trans` which will be in physical units of inches from the bottom left corner of the Figure (see transforms_tutorial for more details).

Saving Figures

Finally, Figures can be saved to disk using the `savefig` method. `fig.savefig('MyFigure.png', dpi=200)` will save a PNG formatted figure to the file `MyFigure.png` in the current directory on disk with 200 dots-per-inch resolution. Note that the filename can include a relative or absolute path to any place on the file system.

Many types of output are supported, including raster formats like PNG, GIF, JPEG, TIFF and vector formats like PDF, EPS, and SVG.

By default, the size of the saved Figure is set by the Figure size (in inches) and, for the raster formats, the *dpi*. If *dpi* is not set, then the *dpi* of the Figure is used. Note that *dpi* still has meaning for vector formats like PDF if the Figure includes Artists that have been rasterized; the *dpi* specified will be the resolution of the rasterized objects.

It is possible to change the size of the Figure using the *bbox_inches* argument to savefig. This can be specified manually, again in inches. However, by far the most common use is `bbox_inches='tight'`. This option "shrink-wraps", trimming or expanding as needed, the size of the figure so that it is tight around all the artists in a figure, with a small pad that can be specified by *pad_inches*, which defaults to 0.1 inches. The dashed box in the plot below shows the portion of the figure that would be saved if `bbox_inches='tight'` were used in savefig.

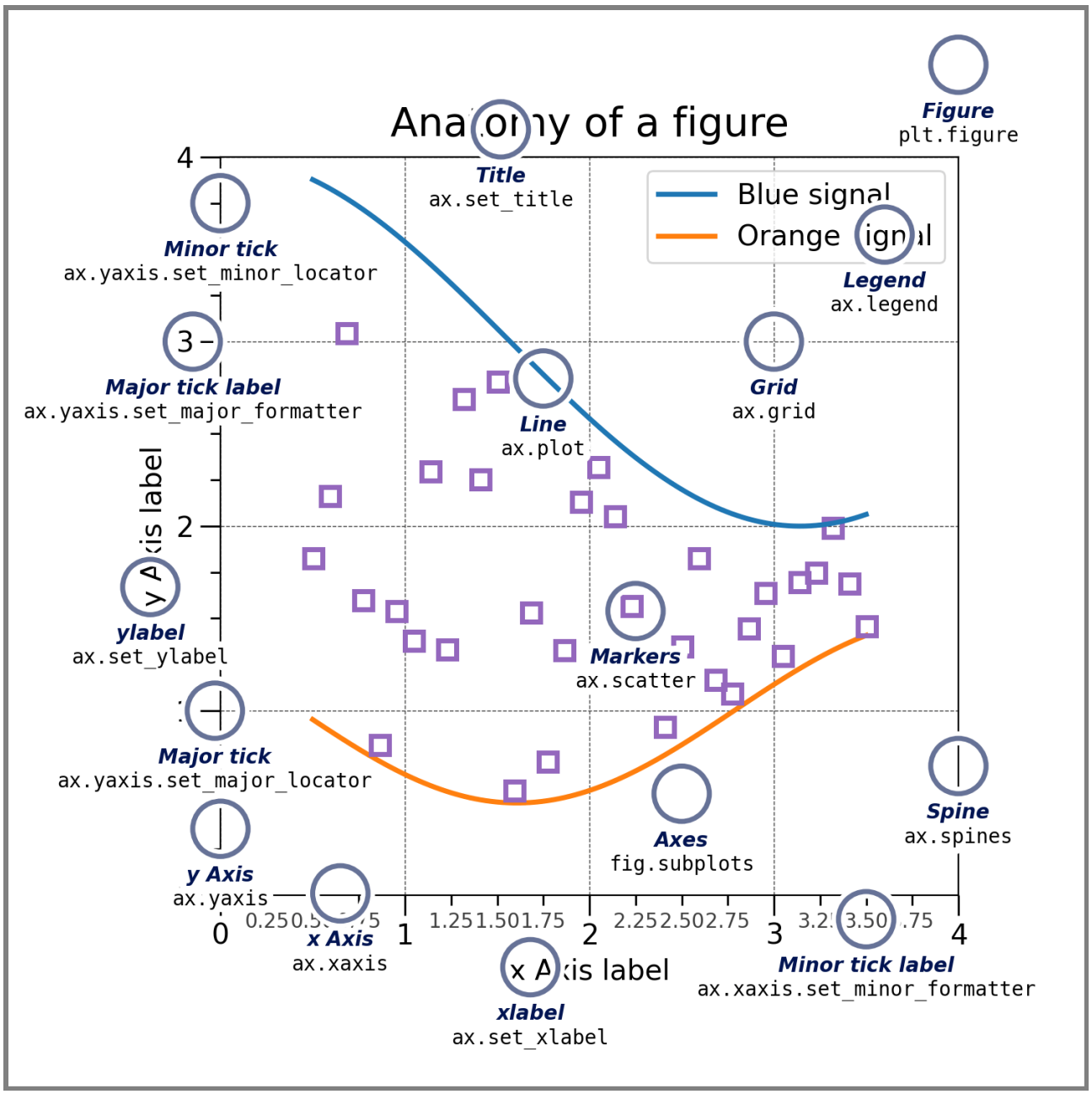
```
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch

fig, ax = plt.subplots(figsize=(4, 2), facecolor='lightskyblue')
ax.set_position([0.1, 0.2, 0.8, 0.7])
ax.set_aspect(1)
bb = ax.get_tightbbox()
bb = bb.padded(10)
bb = bb.transformed(fig.dpi_scale_trans.inverted())
fancy = FancyBboxPatch(bb.p0, bb.width, bb.height, fc='none',
                       ec=(0, 0.0, 0, 0.5), lw=2, linestyle='--',
                       transform=fig.dpi_scale_trans,
                       clip_on=False, boxstyle='Square, pad=0')
ax.add_patch(fancy)
```

Introduction to Axes

Source: https://matplotlib.org/stable/users/explain/axes/axes_intro.html

Matplotlib `Axes` are the gateway to creating your data visualizations. Once an Axes is placed on a figure there are many methods that can be used to add data to the Axes. An Axes typically has a pair of `Axis` Artists that define the data coordinate system, and include methods to add annotations like x- and y-labels, titles, and legends.



Anatomy of a Figure

In the picture above, the Axes object was created with `ax = fig.subplots()`. Everything else on

the figure was created with methods on this `ax` object, or can be accessed from it. If we want to change the label on the x-axis, we call `ax.set_xlabel('New Label')`, if we want to plot some data we call `ax.plot(x, y)`. Indeed, in the figure above, the only Artist that is not part of the Axes is the Figure itself, so the `axes.Axes` class is really the gateway to much of Matplotlib's functionality.

Note that Axes are so fundamental to the operation of Matplotlib that a lot of material here is duplicate of that in quick_start.

Creating Axes

```
import matplotlib.pyplot as plt
import numpy as np

fig, axs = plt.subplots(ncols=2, nrows=2, figsize=(3.5, 2.5),
                        layout="constrained")
# for each Axes, add an artist, in this case a nice label in the middle...
for row in range(2):
    for col in range(2):
        axs[row, col].annotate(f'axs[{row}, {col}]', (0.5, 0.5),
                               transform=axs[row, col].transAxes,
                               ha='center', va='center', fontsize=18,
                               color='darkgrey')
fig.suptitle('plt.subplots()')
```

Axes are added using methods on `Figure` objects, or via the `pyplot` interface. These methods are discussed in more detail in creating_figures and arranging_axes. However, for instance `add_axes` will manually position an Axes on the page. In the example above `subplots` put a grid of subplots on the figure, and `axs` is a (2, 2) array of Axes, each of which can have data added to them.

There are a number of other methods for adding Axes to a Figure:

- `Figure.add_axes`: manually position an Axes. `fig.add_axes((0, 0, 1, 1))` makes an Axes that fills the whole figure.
- `pyplot.subplots` and `Figure.subplots`: add a grid of Axes as in the example above. The pyplot version returns both the Figure object and an array of Axes. Note that `fig, ax = plt.subplots()` adds a single Axes to a Figure.
- `pyplot.subplot_mosaic` and `Figure.subplot_mosaic`: add a grid of named Axes and return a dictionary of axes. For `fig, axs = plt.subplot_mosaic([['left', 'right'], ['bottom', 'bottom']])`, `axs['left']` is an Axes in the top row on the left, and `axs['bottom']` is an Axes that spans both columns on the bottom.

See arranging_axes for more detail on how to arrange grids of Axes on a Figure.

Axes plotting methods

Most of the high-level plotting methods are accessed from the `axes.Axes` class. See the API documentation for a full curated list, and plot_types for examples. A basic example is `axes.Axes.plot`:

```
fig, ax = plt.subplots(figsize=(4, 3))
np.random.seed(19680801)
t = np.arange(100)
x = np.cumsum(np.random.randn(100))
lines = ax.plot(t, x)
```

Note that `plot` returns a list of *lines* Artists which can subsequently be manipulated, as discussed in users_artists.

A very incomplete list of plotting methods is below. Again, see plot_types for more examples, and `axes.Axes` for the full list of methods.

basic_plots ‘plot	, scatter, bar, step‘,
arrays “pcolo	rmesh,contour,quiver,streamplot,imshow“
stats_plots ‘hist	, errorbar, hist2d, pie, boxplot, violinplot‘
unstructured_plots “trico	ntour,tripcolor“

Axes labelling and annotation

Usually we want to label the Axes with an xlabel, ylabel, and title, and often we want to have a legend to differentiate plot elements. The `Axes` class has a number of methods to create these annotations.

```
fig, ax = plt.subplots(figsize=(5, 3), layout='constrained')
np.random.seed(19680801)
t = np.arange(200)
x = np.cumsum(np.random.randn(200))
y = np.cumsum(np.random.randn(200))
linesx = ax.plot(t, x, label='Random walk x')
linesy = ax.plot(t, y, label='Random walk y')

ax.set_xlabel('Time [s]')
ax.set_ylabel('Distance [km]')
ax.set_title('Random walk example')
ax.legend()
```

These methods are relatively straight-forward, though there are a number of text_props that can be set on the text objects, like *fontsize*, *fontname*, *horizontalalignment*. Legends can be much more complicated; see legend_guide for more details.

Note that text can also be added to axes using `text`, and `annotate`. This can be quite sophisticated: see text_props and annotations for more information.

Axes limits, scales, and ticking

Each Axes has two (or more) `Axis` objects, that can be accessed via `matplotlib.axes.Axes.xaxis` and `matplotlib.axes.Axes.yaxis` properties. These have substantial number of methods on them, and for highly customizable Axis-es it is useful to read the API at `Axis`. However, the Axes class offers a number of helpers for the most common of these methods. Indeed, the `set_xlabel`, discussed above, is a helper for the `set_label_text`.

Other important methods set the extent on the axes (`set_xlim`, `set_ylim`), or more fundamentally the scale of the axes. So for instance, we can make an Axis have a logarithmic scale, and zoom in on a sub-portion of the data:

```
fig, ax = plt.subplots(figsize=(4, 2.5), layout='constrained')
np.random.seed(19680801)
t = np.arange(200)
x = 2**np.cumsum(np.random.randn(200))
linesx = ax.plot(t, x)
ax.set_yscale('log')
ax.set_xlim(20, 180)
```

The Axes class also has helpers to deal with Axis ticks and their labels. Most straight-forward is `set_xticks` and `set_yticks` which manually set the tick locations and optionally their labels. Minor

ticks can be toggled with `minorticks_on` or `minorticks_off`.

Many aspects of Axes ticks and tick labeling can be adjusted using `tick_params`. For instance, to label the top of the axes instead of the bottom,color the ticks red, and color the ticklabels green:

```
fig, ax = plt.subplots(figsize=(4, 2.5))
ax.plot(np.arange(10))
ax.tick_params(top=True, labeltop=True, color='red', axis='x',
               labelcolor='green')
```

More fine-grained control on ticks, setting scales, and controlling the Axis can be highly customized beyond these Axes-level helpers.

Axes layout

Sometimes it is important to set the aspect ratio of a plot in data space, which we can do with `set_aspect`:

```
fig, axs = plt.subplots(ncols=2, figsize=(7, 2.5), layout='constrained')
np.random.seed(19680801)
t = np.arange(200)
x = np.cumsum(np.random.randn(200))
axs[0].plot(t, x)
axs[0].set_title('aspect="auto"')

axs[1].plot(t, x)
axs[1].set_aspect(3)
axs[1].set_title('aspect=3')
```

Arranging multiple Axes in a Figure

Source: https://matplotlib.org/stable/users/explain/axes/arranging_axes.html

Often more than one Axes is wanted on a figure at a time, usually organized into a regular grid. Matplotlib has a variety of tools for working with grids of Axes that have evolved over the history of the library. Here we will discuss the tools we think users should use most often, the tools that underpin how Axes are organized, and mention some of the older tools.

> Note
> Matplotlib uses *Axes* to refer to the drawing area that contains data, x- and y-axis, ticks, labels, title, etc. See figure_parts for more details. Another term that is often used is "subplot", which refers to an Axes that is in a grid with other Axes objects.

Overview

Create grid-shaped combinations of Axes

`subplots` The primary function used to create figures and a grid of Axes. It creates and places all Axes on the figure at once, and returns an object array with handles for the Axes in the grid. See `Figure.subplots`.

or

`subplot_mosaic` A simple way to create figures and a grid of Axes, with the added flexibility that Axes can also span rows or columns. The Axes are returned in a labelled dictionary instead of an array. See also `Figure.subplot_mosaic` and mosaic.

Sometimes it is natural to have more than one distinct group of Axes grids, in which case Matplotlib has the concept of `SubFigure`:

`SubFigure` A virtual figure within a figure.

Underlying tools

Underlying these are the concept of a `GridSpec` and a `SubplotSpec`:

`GridSpec` Specifies the geometry of the grid that a subplot will be placed. The number of rows and number of columns of the grid need to be set. Optionally, the subplot layout parameters (e.g., left, right, etc.) can be tuned.

`SubplotSpec` Specifies the location of the subplot in the given `GridSpec`.

Adding single Axes at a time

The above functions create all Axes in a single function call. It is also possible to add Axes one at a time, and this was originally how Matplotlib used to work. Doing so is generally less elegant and flexible, though sometimes useful for interactive work or to place an Axes in a custom location:

`add_axes` Adds a single Axes at a location specified by `[left, bottom, width, height]` in fractions of figure width or height.

`subplot` or `Figure.add_subplot` Adds a single subplot on a figure, with 1-based indexing (inherited from Matlab). Columns and rows can be spanned by specifying a range of grid cells.

subplot2grid Similar to `pyplot.subplot`, but uses 0-based indexing and two-d python slicing to choose cells.

```
import matplotlib.pyplot as plt
import numpy as np

w, h = 4, 3
margin = 0.5
fig = plt.figure(figsize=(w, h), facecolor='lightblue')
ax = fig.add_axes((margin / w, margin / h,
                   (w - 2 * margin) / w, (h - 2 * margin) / h))
```

High-level methods for making grids

Basic 2×2 grid

We can create a basic 2-by-2 grid of Axes using `subplots`. It returns a `Figure` instance and an array of `Axes` objects. The Axes objects can be used to access methods to place artists on the Axes; here we use `annotate`, but other examples could be `plot`, `pcolormesh`, etc.

```
fig, axs = plt.subplots(ncols=2, nrows=2, figsize=(5.5, 3.5),
                        layout="constrained")
# add an artist, in this case a nice label in the middle...
for row in range(2):
    for col in range(2):
        axs[row, col].annotate(f'axs[{row}, {col}]', (0.5, 0.5),
                               transform=axs[row, col].transAxes,
                               ha='center', va='center', fontsize=18,
                               color='darkgrey')
fig.suptitle('plt.subplots()')
```

We will annotate a lot of Axes, so let's encapsulate the annotation, rather than having that large piece of annotation code every time we need it:

```
def annotate_axes(ax, text, fontsize=18):
    ax.text(0.5, 0.5, text, transform=ax.transAxes,
            ha="center", va="center", fontsize=fontsize, color="darkgrey")
```

The same effect can be achieved with `subplot_mosaic`, but the return type is a dictionary instead of an array, where the user can give the keys useful meanings. Here we provide two lists, each list representing a row, and each element in the list a key representing the column.

```
fig, axd = plt.subplot_mosaic([['upper left', 'upper right'],
                               ['lower left', 'lower right']],
                              figsize=(5.5, 3.5), layout="constrained")
for k, ax in axd.items():
    annotate_axes(ax, f'axd[{k!r}]', fontsize=14)
fig.suptitle('plt.subplot_mosaic()')
```

Grids of fixed-aspect ratio Axes

Fixed-aspect ratio Axes are common for images or maps. However, they present a challenge to layout because two sets of constraints are being imposed on the size of the Axes - that they fit in the figure and that they have a set aspect ratio. This leads to large gaps between Axes by default:

```
fig, axs = plt.subplots(2, 2, layout="constrained",
                        figsize=(5.5, 3.5), facecolor='lightblue')
```

```
for ax in axs.flat:
    ax.set_aspect(1)
fig.suptitle('Fixed aspect Axes')
```

One way to address this is to change the aspect of the figure to be close to the aspect ratio of the Axes, however that requires trial and error. Matplotlib also supplies `layout="compressed"`, which will work with simple grids to reduce the gaps between Axes. (The `mpl_toolkits` also provides `ImageGrid` to accomplish a similar effect, but with a non-standard Axes class).

```
fig, axs = plt.subplots(2, 2, layout="compressed", figsize=(5.5, 3.5),
                        facecolor='lightblue')
for ax in axs.flat:
    ax.set_aspect(1)
fig.suptitle('Fixed aspect Axes: compressed')
```

Axes spanning rows or columns in a grid

Sometimes we want Axes to span rows or columns of the grid. There are actually multiple ways to accomplish this, but the most convenient is probably to use `subplot_mosaic` by repeating one of the keys:

```
fig, axd = plt.subplot_mosaic([['upper left', 'right'],
                               ['lower left', 'right']],
                              figsize=(5.5, 3.5), layout="constrained")
for k, ax in axd.items():
    annotate_axes(ax, f'axd[{k!r}]', fontsize=14)
fig.suptitle('plt.subplot_mosaic()')
```

See below for the description of how to do the same thing using `GridSpec` or `subplot2grid`.

Variable widths or heights in a grid

Both `subplots` and `subplot_mosaic` allow the rows in the grid to be different heights, and the columns to be different widths using the *gridspec_kw* keyword argument. Spacing parameters accepted by `GridSpec` can be passed to `subplots` and `subplot_mosaic`:

```
gs_kw = dict(width_ratios=[1.4, 1], height_ratios=[1, 2])
fig, axd = plt.subplot_mosaic([['upper left', 'right'],
                               ['lower left', 'right']],
                              gridspec_kw=gs_kw, figsize=(5.5, 3.5),
                              layout="constrained")
for k, ax in axd.items():
    annotate_axes(ax, f'axd[{k!r}]', fontsize=14)
fig.suptitle('plt.subplot_mosaic()')
```

Nested Axes layouts

Sometimes it is helpful to have two or more grids of Axes that may not need to be related to one another. The most simple way to accomplish this is to use `Figure.subfigures`. Note that the subfigure layouts are independent, so the Axes spines in each subfigure are not necessarily aligned. See below for a more verbose way to achieve the same effect with `GridSpecFromSubplotSpec`.

```
fig = plt.figure(layout="constrained")
subfigs = fig.subfigures(1, 2, wspace=0.07, width_ratios=[1.5, 1.])
axs0 = subfigs[0].subplots(2, 2)
subfigs[0].set_facecolor('lightblue')
subfigs[0].suptitle('subfigs[0]\nLeft side')
```

```
subfigs[0].supxlabel('xlabel for subfigs[0]')

axs1 = subfigs[1].subplots(3, 1)
subfigs[1].suptitle('subfigs[1]')
subfigs[1].supylabel('ylabel for subfigs[1]')
```

It is also possible to nest Axes using `subplot_mosaic` using nested lists. This method does not use subfigures, like above, so lacks the ability to add per-subfigure `suptitle` and `supxlabel`, etc. Rather it is a convenience wrapper around the `subgridspec` method described below.

```
inner = [['innerA'],
         ['innerB']]
outer = [['upper left',  inner],
          ['lower left', 'lower right']]

fig, axd = plt.subplot_mosaic(outer, layout="constrained")
for k, ax in axd.items():
    annotate_axes(ax, f'axd[{k!r}]')
```

Low-level and advanced grid methods

Internally, the arrangement of a grid of Axes is controlled by creating instances of `GridSpec` and `SubplotSpec`. *GridSpec* defines a (possibly non-uniform) grid of cells. Indexing into the *GridSpec* returns a SubplotSpec that covers one or more grid cells, and can be used to specify the location of an Axes.

The following examples show how to use low-level methods to arrange Axes using *GridSpec* objects.

Basic 2×2 grid

We can accomplish a 2x2 grid in the same manner as `plt.subplots(2, 2)`:

```
fig = plt.figure(figsize=(5.5, 3.5), layout="constrained")
spec = fig.add_gridspec(ncols=2, nrows=2)

ax0 = fig.add_subplot(spec[0, 0])
annotate_axes(ax0, 'ax0')

ax1 = fig.add_subplot(spec[0, 1])
annotate_axes(ax1, 'ax1')

ax2 = fig.add_subplot(spec[1, 0])
annotate_axes(ax2, 'ax2')

ax3 = fig.add_subplot(spec[1, 1])
annotate_axes(ax3, 'ax3')

fig.suptitle('Manually added subplots using add_gridspec')
```

Axes spanning rows or grids in a grid

We can index the *spec* array using NumPy slice syntax and the new Axes will span the slice. This would be the same as `fig, axd = plt.subplot_mosaic([['ax0', 'ax0'], ['ax1', 'ax2']], ...)`:

```
fig = plt.figure(figsize=(5.5, 3.5), layout="constrained")
spec = fig.add_gridspec(2, 2)

ax0 = fig.add_subplot(spec[0, :])
annotate_axes(ax0, 'ax0')

ax10 = fig.add_subplot(spec[1, 0])
annotate_axes(ax10, 'ax10')

ax11 = fig.add_subplot(spec[1, 1])
annotate_axes(ax11, 'ax11')

fig.suptitle('Manually added subplots, spanning a column')
```

Manual adjustments to a *GridSpec* layout

When a *GridSpec* is explicitly used, you can adjust the layout parameters of subplots that are created from the *GridSpec*. Note this option is not compatible with *constrained layout* or `Figure.tight_layout` which both ignore *left* and *right* and adjust subplot sizes to fill the figure. Usually such manual placement requires iterations to make the Axes tick labels not overlap the Axes.

These spacing parameters can also be passed to `subplots` and `subplot_mosaic` as the *gridspec_kw* argument.

```
fig = plt.figure(layout=None, facecolor='lightblue')
gs = fig.add_gridspec(nrows=3, ncols=3, left=0.05, right=0.75,
                      hspace=0.1, wspace=0.05)
ax0 = fig.add_subplot(gs[:-1, :])
annotate_axes(ax0, 'ax0')
ax1 = fig.add_subplot(gs[-1, :-1])
annotate_axes(ax1, 'ax1')
ax2 = fig.add_subplot(gs[-1, -1])
annotate_axes(ax2, 'ax2')
fig.suptitle('Manual gridspec with right=0.75')
```

Nested layouts with SubplotSpec

You can create nested layout similar to `subfigures` using `subgridspec`. Here the Axes spines *are* aligned.

Note this is also available from the more verbose `gridspec.GridSpecFromSubplotSpec`.

```
fig = plt.figure(layout="constrained")
gs0 = fig.add_gridspec(1, 2)

gs00 = gs0[0].subgridspec(2, 2)
gs01 = gs0[1].subgridspec(3, 1)

for a in range(2):
    for b in range(2):
        ax = fig.add_subplot(gs00[a, b])
        annotate_axes(ax, f'axLeft[{a}, {b}]', fontsize=10)
        if a == 1 and b == 1:
            ax.set_xlabel('xlabel')
for a in range(3):
    ax = fig.add_subplot(gs01[a])
    annotate_axes(ax, f'axRight[{a}, {b}]')
```

```
    if a == 2:
        ax.set_ylabel('ylabel')

fig.suptitle('nested gridspecs')
```

Here's a more sophisticated example of nested *GridSpec*: We create an outer 4x4 grid with each cell containing an inner 3x3 grid of Axes. We outline the outer 4x4 grid by hiding appropriate spines in each of the inner 3x3 grids.

```
def squiggle_xy(a, b, c, d, i=np.arange(0.0, 2*np.pi, 0.05)):
    return np.sin(i*a)*np.cos(i*b), np.sin(i*c)*np.cos(i*d)

fig = plt.figure(figsize=(8, 8), layout='constrained')
outer_grid = fig.add_gridspec(4, 4, wspace=0, hspace=0)

for a in range(4):
    for b in range(4):
        # gridspec inside gridspec
        inner_grid = outer_grid[a, b].subgridspec(3, 3, wspace=0, hspace=0)
        axs = inner_grid.subplots()  # Create all subplots for the inner grid.
        for (c, d), ax in np.ndenumerate(axs):
            ax.plot(*squiggle_xy(a + 1, b + 1, c + 1, d + 1))
            ax.set(xticks=[], yticks=[])

# show only the outside spines
for ax in fig.get_axes():
    ss = ax.get_subplotspec()
    ax.spines.top.set_visible(ss.is_first_row())
    ax.spines.bottom.set_visible(ss.is_last_row())
    ax.spines.left.set_visible(ss.is_first_col())
    ax.spines.right.set_visible(ss.is_last_col())

plt.show()
```

More reading

- More details about subplot mosaic.
- More details about `constrained layout` `<constrainedlayout_guide>`, used to align spacing in most of these examples.

References
The use of the following functions, methods, classes and modules is shown in this example:

- `matplotlib.pyplot.subplots`
- `matplotlib.pyplot.subplot_mosaic`
- `matplotlib.figure.Figure.add_gridspec`
- `matplotlib.figure.Figure.add_subplot`
- `matplotlib.gridspec.GridSpec`
- `matplotlib.gridspec.SubplotSpec.subgridspec`
- `matplotlib.gridspec.GridSpecFromSubplotSpec`

Constrained layout guide

Source: https://matplotlib.org/stable/users/explain/axes/constrainedlayout_guide.html

Use *constrained layout* to fit plots within your figure cleanly.

Constrained layout automatically adjusts subplots so that decorations like tick labels, legends, and colorbars do not overlap, while still preserving the logical layout requested by the user.

Constrained layout is similar to `Tight layout<tight_layout_guide>`, but is substantially more flexible. It handles colorbars placed on multiple Axes (colorbar_placement) nested layouts (`subfigures`) and Axes that span rows or columns (`subplot_mosaic`), striving to align spines from Axes in the same row or column. In addition, `Compressed layout <compressed_layout>` will try and move fixed aspect-ratio Axes closer together. These features are described in this document, as well as some implementation details discussed at the end.

Constrained layout typically needs to be activated before any Axes are added to a figure. Two ways of doing so are

- using the respective argument to `subplots`, `figure`, `subplot_mosaic` e.g.:

  ```
  plt.subplots(layout="constrained")
  ```

- activate it via rcParams, like:

  ```
  plt.rcParams['figure.constrained_layout.use'] = True
  ```

Those are described in detail throughout the following sections.

> Warning
> Calling `tight_layout` will turn off *constrained layout*!

Simple example

With the default Axes positioning, the axes title, axis labels, or tick labels can sometimes go outside the figure area, and thus get clipped.

```
import matplotlib.pyplot as plt
import numpy as np

import matplotlib.colors as mcolors
import matplotlib.gridspec as gridspec

plt.rcParams['savefig.facecolor'] = "0.8"
plt.rcParams['figure.figsize'] = 4.5, 4.
plt.rcParams['figure.max_open_warning'] = 50


def example_plot(ax, fontsize=12, hide_labels=False):
    ax.plot([1, 2])

    ax.locator_params(nbins=3)
    if hide_labels:
        ax.set_xticklabels([])
        ax.set_yticklabels([])
    else:
```

```
        ax.set_xlabel('x-label', fontsize=fontsize)
        ax.set_ylabel('y-label', fontsize=fontsize)
        ax.set_title('Title', fontsize=fontsize)

fig, ax = plt.subplots(layout=None)
example_plot(ax, fontsize=24)
```

To prevent this, the location of Axes needs to be adjusted. For subplots, this can be done manually by adjusting the subplot parameters using `Figure.subplots_adjust`. However, specifying your figure with the `layout="constrained"` keyword argument will do the adjusting automatically.

```
fig, ax = plt.subplots(layout="constrained")
example_plot(ax, fontsize=24)
```

When you have multiple subplots, often you see labels of different Axes overlapping each other.

```
fig, axs = plt.subplots(2, 2, layout=None)
for ax in axs.flat:
    example_plot(ax)
```

Specifying `layout="constrained"` in the call to `plt.subplots` causes the layout to be properly constrained.

```
fig, axs = plt.subplots(2, 2, layout="constrained")
for ax in axs.flat:
    example_plot(ax)
```

Colorbars

If you create a colorbar with `Figure.colorbar`, you need to make room for it. *Constrained layout* does this automatically. Note that if you specify `use_gridspec=True` it will be ignored because this option is made for improving the layout via `tight_layout`.

> Note
> For the `pcolormesh` keyword arguments (`pc_kwargs`) we use a dictionary to keep the calls consistent across this document.

```
arr = np.arange(100).reshape((10, 10))
norm = mcolors.Normalize(vmin=0., vmax=100.)
# see note above: this makes all pcolormesh calls consistent:
pc_kwargs = {'rasterized': True, 'cmap': 'viridis', 'norm': norm}
fig, ax = plt.subplots(figsize=(4, 4), layout="constrained")
im = ax.pcolormesh(arr, **pc_kwargs)
fig.colorbar(im, ax=ax, shrink=0.6)
```

If you specify a list of Axes (or other iterable container) to the `ax` argument of `colorbar`, *constrained layout* will take space from the specified Axes.

```
fig, axs = plt.subplots(2, 2, figsize=(4, 4), layout="constrained")
for ax in axs.flat:
    im = ax.pcolormesh(arr, **pc_kwargs)
fig.colorbar(im, ax=axs, shrink=0.6)
```

If you specify a list of Axes from inside a grid of Axes, the colorbar will steal space appropriately, and leave a gap, but all subplots will still be the same size.

```
fig, axs = plt.subplots(3, 3, figsize=(4, 4), layout="constrained")
for ax in axs.flat:
```

```
    im = ax.pcolormesh(arr, **pc_kwargs)
fig.colorbar(im, ax=axs[1:, 1], shrink=0.8)
fig.colorbar(im, ax=axs[:, -1], shrink=0.6)
```

Suptitle

Constrained layout can also make room for `suptitle`.

```
fig, axs = plt.subplots(2, 2, figsize=(4, 4), layout="constrained")
for ax in axs.flat:
    im = ax.pcolormesh(arr, **pc_kwargs)
fig.colorbar(im, ax=axs, shrink=0.6)
fig.suptitle('Big Suptitle')
```

Legends

Legends can be placed outside of their parent axes. *Constrained layout* is designed to handle this for `.Axes.legend`. However, *constrained layout* does *not* handle legends being created via `.Figure.legend` (yet).

```
fig, ax = plt.subplots(layout="constrained")
ax.plot(np.arange(10), label='This is a plot')
ax.legend(loc='center left', bbox_to_anchor=(0.8, 0.5))
```

However, this will steal space from a subplot layout:

```
fig, axs = plt.subplots(1, 2, figsize=(4, 2), layout="constrained")
axs[0].plot(np.arange(10))
axs[1].plot(np.arange(10), label='This is a plot')
axs[1].legend(loc='center left', bbox_to_anchor=(0.8, 0.5))
```

In order for a legend or other artist to *not* steal space from the subplot layout, we can `leg.set_in_layout(False)`. Of course this can mean the legend ends up cropped, but can be useful if the plot is subsequently called with `fig.savefig('outname.png', bbox_inches='tight')`. Note, however, that the legend's `get_in_layout` status will have to be toggled again to make the saved file work, and we must manually trigger a draw if we want *constrained layout* to adjust the size of the Axes before printing.

```
fig, axs = plt.subplots(1, 2, figsize=(4, 2), layout="constrained")

axs[0].plot(np.arange(10))
axs[1].plot(np.arange(10), label='This is a plot')
leg = axs[1].legend(loc='center left', bbox_to_anchor=(0.8, 0.5))
leg.set_in_layout(False)
# trigger a draw so that constrained layout is executed once
# before we turn it off when printing....
fig.canvas.draw()
# we want the legend included in the bbox_inches='tight' calcs.
leg.set_in_layout(True)
# we don't want the layout to change at this point.
fig.set_layout_engine('none')
try:
    fig.savefig('../../../doc/_static/constrained_layout_1b.png',
                bbox_inches='tight', dpi=100)
except FileNotFoundError:
    # this allows the script to keep going if run interactively and
```

```
    # the directory above doesn't exist
    pass
```

The saved file looks like:

A better way to get around this awkwardness is to simply use the legend method provided by `Figure.legend`:

```
fig, axs = plt.subplots(1, 2, figsize=(4, 2), layout="constrained")
axs[0].plot(np.arange(10))
lines = axs[1].plot(np.arange(10), label='This is a plot')
labels = [l.get_label() for l in lines]
leg = fig.legend(lines, labels, loc='center left',
                 bbox_to_anchor=(0.8, 0.5), bbox_transform=axs[1].transAxes)
try:
    fig.savefig('../../../doc/_static/constrained_layout_2b.png',
                bbox_inches='tight', dpi=100)
except FileNotFoundError:
    # this allows the script to keep going if run interactively and
    # the directory above doesn't exist
    pass
```

The saved file looks like:

Padding and spacing

Padding between Axes is controlled in the horizontal by *w_pad* and *wspace*, and vertical by *h_pad* and *hspace*. These can be edited via `set`. *w/h_pad* are the minimum space around the Axes in units of inches:

```
fig, axs = plt.subplots(2, 2, layout="constrained")
for ax in axs.flat:
    example_plot(ax, hide_labels=True)
fig.get_layout_engine().set(w_pad=4 / 72, h_pad=4 / 72, hspace=0,
                            wspace=0)
```

Spacing between subplots is further set by *wspace* and *hspace*. These are specified as a fraction of the size of the subplot group as a whole. If these values are smaller than *w_pad* or *h_pad*, then the fixed pads are used instead. Note in the below how the space at the edges doesn't change from the above, but the space between subplots does.

```
fig, axs = plt.subplots(2, 2, layout="constrained")
for ax in axs.flat:
    example_plot(ax, hide_labels=True)
fig.get_layout_engine().set(w_pad=4 / 72, h_pad=4 / 72, hspace=0.2,
                            wspace=0.2)
```

If there are more than two columns, the *wspace* is shared between them, so here the wspace is divided in two, with a *wspace* of 0.1 between each column:

```
fig, axs = plt.subplots(2, 3, layout="constrained")
for ax in axs.flat:
    example_plot(ax, hide_labels=True)
fig.get_layout_engine().set(w_pad=4 / 72, h_pad=4 / 72, hspace=0.2,
                            wspace=0.2)
```

GridSpecs also have optional *hspace* and *wspace* keyword arguments, that will be used instead of the pads set by *constrained layout*:

```
fig, axs = plt.subplots(2, 2, layout="constrained",
                        gridspec_kw={'wspace': 0.3, 'hspace': 0.2})
for ax in axs.flat:
    example_plot(ax, hide_labels=True)
# this has no effect because the space set in the gridspec trumps the
# space set in *constrained layout*.
fig.get_layout_engine().set(w_pad=4 / 72, h_pad=4 / 72, hspace=0.0,
                            wspace=0.0)
```

Spacing with colorbars

Colorbars are placed a distance *pad* from their parent, where *pad* is a fraction of the width of the parent(s). The spacing to the next subplot is then given by *w/hspace*.

```
fig, axs = plt.subplots(2, 2, layout="constrained")
pads = [0, 0.05, 0.1, 0.2]
for pad, ax in zip(pads, axs.flat):
    pc = ax.pcolormesh(arr, **pc_kwargs)
    fig.colorbar(pc, ax=ax, shrink=0.6, pad=pad)
    ax.set_xticklabels([])
    ax.set_yticklabels([])
    ax.set_title(f'pad: {pad}')
fig.get_layout_engine().set(w_pad=2 / 72, h_pad=2 / 72, hspace=0.2,
                            wspace=0.2)
```

rcParams

There are five rcParams that can be set, either in a script or in the `matplotlibrc` file. They all have the prefix `figure.constrained_layout`:

- *use*: Whether to use *constrained layout*. Default is False
- *w_pad, h_pad*: Padding around Axes objects. Float representing inches. Default is 3./72. inches (3 pts)
- *wspace, hspace*: Space between subplot groups. Float representing a fraction of the subplot widths being separated. Default is 0.02.

```
plt.rcParams['figure.constrained_layout.use'] = True
fig, axs = plt.subplots(2, 2, figsize=(3, 3))
for ax in axs.flat:
    example_plot(ax)
```

Use with GridSpec

Constrained layout is meant to be used with `subplots`, `subplot_mosaic`, or `GridSpec` with `add_subplot`.

Note that in what follows `layout="constrained"`

```
plt.rcParams['figure.constrained_layout.use'] = False
fig = plt.figure(layout="constrained")

gs1 = gridspec.GridSpec(2, 1, figure=fig)
ax1 = fig.add_subplot(gs1[0])
ax2 = fig.add_subplot(gs1[1])

example_plot(ax1)
example_plot(ax2)
```

More complicated gridspec layouts are possible. Note here we use the convenience functions `add_gridspec` and `subgridspec`.

```
fig = plt.figure(layout="constrained")

gs0 = fig.add_gridspec(1, 2)

gs1 = gs0[0].subgridspec(2, 1)
ax1 = fig.add_subplot(gs1[0])
ax2 = fig.add_subplot(gs1[1])

example_plot(ax1)
example_plot(ax2)

gs2 = gs0[1].subgridspec(3, 1)

for ss in gs2:
    ax = fig.add_subplot(ss)
    example_plot(ax)
    ax.set_title("")
    ax.set_xlabel("")

ax.set_xlabel("x-label", fontsize=12)
```

Note that in the above the left and right columns don't have the same vertical extent. If we want the top and bottom of the two grids to line up then they need to be in the same gridspec. We need to make this figure larger as well in order for the Axes not to collapse to zero height:

```
fig = plt.figure(figsize=(4, 6), layout="constrained")

gs0 = fig.add_gridspec(6, 2)

ax1 = fig.add_subplot(gs0[:3, 0])
ax2 = fig.add_subplot(gs0[3:, 0])

example_plot(ax1)
example_plot(ax2)

ax = fig.add_subplot(gs0[0:2, 1])
example_plot(ax, hide_labels=True)
ax = fig.add_subplot(gs0[2:4, 1])
example_plot(ax, hide_labels=True)
ax = fig.add_subplot(gs0[4:, 1])
example_plot(ax, hide_labels=True)
fig.suptitle('Overlapping Gridspecs')
```

This example uses two gridspecs to have the colorbar only pertain to one set of pcolors. Note how the left column is wider than the two right-hand columns because of this. Of course, if you wanted the subplots to be the same size you only needed one gridspec. Note that the same effect can be achieved using `subfigures`.

```
fig = plt.figure(layout="constrained")
gs0 = fig.add_gridspec(1, 2, figure=fig, width_ratios=[1, 2])
gs_left = gs0[0].subgridspec(2, 1)
gs_right = gs0[1].subgridspec(2, 2)

for gs in gs_left:
    ax = fig.add_subplot(gs)
    example_plot(ax)
```

```
axs = []
for gs in gs_right:
    ax = fig.add_subplot(gs)
    pcm = ax.pcolormesh(arr, **pc_kwargs)
    ax.set_xlabel('x-label')
    ax.set_ylabel('y-label')
    ax.set_title('title')
    axs += [ax]
fig.suptitle('Nested plots using subgridspec')
fig.colorbar(pcm, ax=axs)
```

Rather than using subgridspecs, Matplotlib now provides `subfigures` which also work with *constrained layout*:

```
fig = plt.figure(layout="constrained")
sfigs = fig.subfigures(1, 2, width_ratios=[1, 2])

axs_left = sfigs[0].subplots(2, 1)
for ax in axs_left.flat:
    example_plot(ax)

axs_right = sfigs[1].subplots(2, 2)
for ax in axs_right.flat:
    pcm = ax.pcolormesh(arr, **pc_kwargs)
    ax.set_xlabel('x-label')
    ax.set_ylabel('y-label')
    ax.set_title('title')
fig.colorbar(pcm, ax=axs_right)
fig.suptitle('Nested plots using subfigures')
```

Manually setting Axes positions

There can be good reasons to manually set an Axes position. A manual call to `set_position` will set the Axes so *constrained layout* has no effect on it anymore. (Note that *constrained layout* still leaves the space for the Axes that is moved).

```
fig, axs = plt.subplots(1, 2, layout="constrained")
example_plot(axs[0], fontsize=12)
axs[1].set_position([0.2, 0.2, 0.4, 0.4])
```

Grids of fixed aspect-ratio Axes: ”compressed” layout

Constrained layout operates on the grid of ”original” positions for Axes. However, when Axes have fixed aspect ratios, one side is usually made shorter, and leaves large gaps in the shortened direction. In the following, the Axes are square, but the figure quite wide so there is a horizontal gap:

```
fig, axs = plt.subplots(2, 2, figsize=(5, 3),
                        sharex=True, sharey=True, layout="constrained")
for ax in axs.flat:
    ax.imshow(arr)
fig.suptitle("fixed-aspect plots, layout='constrained'")
```

One obvious way of fixing this is to make the figure size more square, however, closing the gaps exactly requires trial and error. For simple grids of Axes we can use `layout="compressed"` to do the job for us:

```
fig, axs = plt.subplots(2, 2, figsize=(5, 3),
                        sharex=True, sharey=True, layout='compressed')
for ax in axs.flat:
    ax.imshow(arr)
fig.suptitle("fixed-aspect plots, layout='compressed'")
```

Compressed layout will also attempt to size colorbars to match the size of the fixed-aspect-ratio parent Axes as the figure is resized or the aspect ratio changes. In the following figure, the colorbar is taller than its parent Axes:

```
fig, ax = plt.subplots(layout='constrained', figsize=(3, 3))
pcm = ax.imshow(np.random.randn(10, 10), cmap='viridis')
ax.set_title("Colorbar with layout='constrained'", fontsize='medium')
fig.colorbar(pcm, ax=ax)
```

Compressed layout ensures that the height of the colorbar matches the height of its parent Axes, maintaining a consistent appearance:

```
fig, ax = plt.subplots(layout='compressed', figsize=(3, 3))
pcm = ax.imshow(np.random.randn(10, 10), cmap='viridis')
ax.set_title("Colorbar with layout='compressed'", fontsize='medium')
fig.colorbar(pcm, ax=ax)
```

If the Axes is zoomed in or out, or the figure is resized, the colorbar will dynamically resize to match the parent Axes. Whether this behavior is desired depends on the specific application:

```
fig, ax = plt.subplots(layout='compressed', figsize=(3, 3))
pcm = ax.imshow(np.random.randn(10, 10), cmap='viridis')
ax.set_ylim([4, 8])
ax.set_title("Layout='compressed' with zoom", fontsize='medium')
fig.colorbar(pcm, ax=ax)
```

Manually turning off *constrained layout*

Constrained layout usually adjusts the Axes positions on each draw of the figure. If you want to get the spacing provided by *constrained layout* but not have it update, then do the initial draw and then call `fig.set_layout_engine('none')`. This is potentially useful for animations where the tick labels may change length.

Note that *constrained layout* is turned off for `ZOOM` and `PAN` GUI events for the backends that use the toolbar. This prevents the Axes from changing position during zooming and panning.

Limitations

Incompatible functions

Constrained layout will work with `pyplot.subplot`, but only if the number of rows and columns is the same for each call. The reason is that each call to `pyplot.subplot` will create a new `GridSpec` instance if the geometry is not the same, and *constrained layout*. So the following works fine:

```
fig = plt.figure(layout="constrained")

ax1 = plt.subplot(2, 2, 1)
ax2 = plt.subplot(2, 2, 3)
# third Axes that spans both rows in second column:
ax3 = plt.subplot(2, 2, (2, 4))
```

```
example_plot(ax1)
example_plot(ax2)
example_plot(ax3)
plt.suptitle('Homogeneous nrows, ncols')
```

but the following leads to a poor layout:

```
fig = plt.figure(layout="constrained")

ax1 = plt.subplot(2, 2, 1)
ax2 = plt.subplot(2, 2, 3)
ax3 = plt.subplot(1, 2, 2)

example_plot(ax1)
example_plot(ax2)
example_plot(ax3)
plt.suptitle('Mixed nrows, ncols')
```

Similarly, `subplot2grid` works with the same limitation that nrows and ncols cannot change for the layout to look good.

```
fig = plt.figure(layout="constrained")

ax1 = plt.subplot2grid((3, 3), (0, 0))
ax2 = plt.subplot2grid((3, 3), (0, 1), colspan=2)
ax3 = plt.subplot2grid((3, 3), (1, 0), colspan=2, rowspan=2)
ax4 = plt.subplot2grid((3, 3), (1, 2), rowspan=2)

example_plot(ax1)
example_plot(ax2)
example_plot(ax3)
example_plot(ax4)
fig.suptitle('subplot2grid')
```

Other caveats

- *Constrained layout* only considers ticklabels, axis labels, titles, and legends. Thus, other artists may be clipped and also may overlap.
- It assumes that the extra space needed for ticklabels, axis labels, and titles is independent of original location of Axes. This is often true, but there are rare cases where it is not.
- There are small differences in how the backends handle rendering fonts, so the results will not be pixel-identical.
- An artist using Axes coordinates that extend beyond the Axes boundary will result in unusual layouts when added to an Axes. This can be avoided by adding the artist directly to the `Figure` using `add_artist`. See `ConnectionPatch` for an example.

Debugging

Constrained layout can fail in somewhat unexpected ways. Because it uses a constraint solver the solver can find solutions that are mathematically correct, but that aren't at all what the user wants. The usual failure mode is for all sizes to collapse to their smallest allowable value. If this happens, it is for one of two reasons:

1. There was not enough room for the elements you were requesting to draw.
2. There is a bug - in which case open an issue at https://github.com/matplotlib/matplotlib/issues.

If there is a bug, please report with a self-contained example that does not require outside data or dependencies (other than numpy).

Notes on the algorithm

The algorithm for the constraint is relatively straightforward, but has some complexity due to the complex ways we can lay out a figure.

Layout in Matplotlib is carried out with gridspecs via the `GridSpec` class. A gridspec is a logical division of the figure into rows and columns, with the relative width of the Axes in those rows and columns set by *width_ratios* and *height_ratios*.

In *constrained layout*, each gridspec gets a *layoutgrid* associated with it. The *layoutgrid* has a series of `left` and `right` variables for each column, and `bottom` and `top` variables for each row, and further it has a margin for each of left, right, bottom and top. In each row, the bottom/top margins are widened until all the decorators in that row are accommodated. Similarly, for columns and the left/right margins.

Simple case: one Axes

For a single Axes the layout is straight forward. There is one parent layoutgrid for the figure consisting of one column and row, and a child layoutgrid for the gridspec that contains the Axes, again consisting of one row and column. Space is made for the "decorations" on each side of the Axes. In the code, this is accomplished by the entries in `do_constrained_layout()` like:

```
gridspec._layoutgrid[0, 0].edit_margin_min('left',
      -bbox.x0 + pos.x0 + w_pad)
```

where `bbox` is the tight bounding box of the Axes, and `pos` its position. Note how the four margins encompass the Axes decorations.

```
from matplotlib._layoutgrid import plot_children

fig, ax = plt.subplots(layout="constrained")
example_plot(ax, fontsize=24)
plot_children(fig)
```

Simple case: two Axes

When there are multiple Axes they have their layouts bound in simple ways. In this example the left Axes has much larger decorations than the right, but they share a bottom margin, which is made large enough to accommodate the larger xlabel. Same with the shared top margin. The left and right margins are not shared, and hence are allowed to be different.

```
fig, ax = plt.subplots(1, 2, layout="constrained")
example_plot(ax[0], fontsize=32)
example_plot(ax[1], fontsize=8)
plot_children(fig)
```

Two Axes and colorbar

A colorbar is simply another item that expands the margin of the parent layoutgrid cell:

```
fig, ax = plt.subplots(1, 2, layout="constrained")
im = ax[0].pcolormesh(arr, **pc_kwargs)
fig.colorbar(im, ax=ax[0], shrink=0.6)
```

```
im = ax[1].pcolormesh(arr, **pc_kwargs)
plot_children(fig)
```

Colorbar associated with a Gridspec

If a colorbar belongs to more than one cell of the grid, then it makes a larger margin for each:

```
fig, axs = plt.subplots(2, 2, layout="constrained")
for ax in axs.flat:
    im = ax.pcolormesh(arr, **pc_kwargs)
fig.colorbar(im, ax=axs, shrink=0.6)
plot_children(fig)
```

Uneven sized Axes

There are two ways to make Axes have an uneven size in a Gridspec layout, either by specifying them to cross Gridspecs rows or columns, or by specifying width and height ratios.

The first method is used here. Note that the middle `top` and `bottom` margins are not affected by the left-hand column. This is a conscious decision of the algorithm, and leads to the case where the two right-hand Axes have the same height, but it is not 1/2 the height of the left-hand Axes. This is consistent with how `gridspec` works without *constrained layout*.

```
fig = plt.figure(layout="constrained")
gs = gridspec.GridSpec(2, 2, figure=fig)
ax = fig.add_subplot(gs[:, 0])
im = ax.pcolormesh(arr, **pc_kwargs)
ax = fig.add_subplot(gs[0, 1])
im = ax.pcolormesh(arr, **pc_kwargs)
ax = fig.add_subplot(gs[1, 1])
im = ax.pcolormesh(arr, **pc_kwargs)
plot_children(fig)
```

One case that requires finessing is if margins do not have any artists constraining their width. In the case below, the right margin for column 0 and the left margin for column 3 have no margin artists to set their width, so we take the maximum width of the margin widths that do have artists. This makes all the Axes have the same size:

```
fig = plt.figure(layout="constrained")
gs = fig.add_gridspec(2, 4)
ax00 = fig.add_subplot(gs[0, 0:2])
ax01 = fig.add_subplot(gs[0, 2:])
ax10 = fig.add_subplot(gs[1, 1:3])
example_plot(ax10, fontsize=14)
plot_children(fig)
plt.show()
```

Axis ticks

Source: *https://matplotlib.org/stable/users/explain/axes/axes_ticks.html*

The x and y Axis on each Axes have default tick ”locators” and ”formatters” that depend on the scale being used (see user_axes_scales). It is possible to customize the ticks and tick labels with either high-level methods like `set_xticks` or set the locators and formatters directly on the axis.

Manual location and formats

The simplest method to customize the tick locations and formats is to use `set_xticks` and `set_yticks`. These can be used on either the major or the minor ticks.

```
import numpy as np
import matplotlib.pyplot as plt

import matplotlib.ticker as ticker


fig, axs = plt.subplots(2, 1, figsize=(5.4, 5.4), layout='constrained')
x = np.arange(100)
for nn, ax in enumerate(axs):
    ax.plot(x, x)
    if nn == 1:
        ax.set_title('Manual ticks')
        ax.set_yticks(np.arange(0, 100.1, 100/3))
        xticks = np.arange(0.50, 101, 20)
        xlabels = [f'\\${x:1.2f}' for x in xticks]
        ax.set_xticks(xticks, labels=xlabels)
    else:
        ax.set_title('Automatic ticks')
```

Note that the length of the `labels` argument must have the same length as the array used to specify the ticks.

By default `set_xticks` and `set_yticks` act on the major ticks of an Axis, however it is possible to add minor ticks:

```
fig, axs = plt.subplots(2, 1, figsize=(5.4, 5.4), layout='constrained')
x = np.arange(100)
for nn, ax in enumerate(axs):
    ax.plot(x, x)
    if nn == 1:
        ax.set_title('Manual ticks')
        ax.set_yticks(np.arange(0, 100.1, 100/3))
        ax.set_yticks(np.arange(0, 100.1, 100/30), minor=True)
    else:
        ax.set_title('Automatic ticks')
```

Locators and Formatters

Manually setting the ticks as above works well for specific final plots, but does not adapt as the user interacts with the Axes. At a lower level, Matplotlib has `Locators` that are meant to

automatically choose ticks depending on the current view limits of the axis, and `Formatters` that are meant to format the tick labels automatically.

The full list of locators provided by Matplotlib are listed at locators, and the formatters at formatters.

```
def setup(ax, title):
    """Set up common parameters for the Axes in the example."""
    # only show the bottom spine
    ax.yaxis.set_major_locator(ticker.NullLocator())
    ax.spines[['left', 'right', 'top']].set_visible(False)

    ax.xaxis.set_ticks_position('bottom')
    ax.tick_params(which='major', width=1.00, length=5)
    ax.tick_params(which='minor', width=0.75, length=2.5)
    ax.set_xlim(0, 5)
    ax.set_ylim(0, 1)
    ax.text(0.0, 0.2, title, transform=ax.transAxes,
            fontsize=14, fontname='Monospace', color='tab:blue')


fig, axs = plt.subplots(8, 1, layout='constrained')

# Null Locator
setup(axs[0], title="NullLocator()")
axs[0].xaxis.set_major_locator(ticker.NullLocator())
axs[0].xaxis.set_minor_locator(ticker.NullLocator())

# Multiple Locator
setup(axs[1], title="MultipleLocator(0.5)")
axs[1].xaxis.set_major_locator(ticker.MultipleLocator(0.5))
axs[1].xaxis.set_minor_locator(ticker.MultipleLocator(0.1))

# Fixed Locator
setup(axs[2], title="FixedLocator([0, 1, 5])")
axs[2].xaxis.set_major_locator(ticker.FixedLocator([0, 1, 5]))
axs[2].xaxis.set_minor_locator(ticker.FixedLocator(np.linspace(0.2, 0.8, 4)))

# Linear Locator
setup(axs[3], title="LinearLocator(numticks=3)")
axs[3].xaxis.set_major_locator(ticker.LinearLocator(3))
axs[3].xaxis.set_minor_locator(ticker.LinearLocator(31))

# Index Locator
setup(axs[4], title="IndexLocator(base=0.5, offset=0.25)")
axs[4].plot(range(0, 5), [0]*5, color='white')
axs[4].xaxis.set_major_locator(ticker.IndexLocator(base=0.5, offset=0.25))

# Auto Locator
setup(axs[5], title="AutoLocator()")
axs[5].xaxis.set_major_locator(ticker.AutoLocator())
axs[5].xaxis.set_minor_locator(ticker.AutoMinorLocator())

# MaxN Locator
setup(axs[6], title="MaxNLocator(n=4)")
axs[6].xaxis.set_major_locator(ticker.MaxNLocator(4))
axs[6].xaxis.set_minor_locator(ticker.MaxNLocator(40))

# Log Locator
setup(axs[7], title="LogLocator(base=10, numticks=15)")
```

```
axs[7].set_xlim(10**3, 10**10)
axs[7].set_xscale('log')
axs[7].xaxis.set_major_locator(ticker.LogLocator(base=10, numticks=15))
plt.show()
```

Similarly, we can specify ”Formatters” for the major and minor ticks on each axis.

The tick format is configured via the function `set_major_formatter` or `set_minor_formatter`. It accepts:

- a format string, which implicitly creates a `StrMethodFormatter`.
- a function, implicitly creates a `FuncFormatter`.
- an instance of a `Formatter` subclass. The most common are
  - `NullFormatter`: No labels on the ticks.
  - `StrMethodFormatter`: Use string `str.format` method.
  - `FormatStrFormatter`: Use %-style formatting.
  - `FuncFormatter`: Define labels through a function.
  - `FixedFormatter`: Set the label strings explicitly.
  - `ScalarFormatter`: Default formatter for scalars: auto-pick the format string.
  - `PercentFormatter`: Format labels as a percentage.

See formatters for the complete list.

```
def setup(ax, title):
    """Set up common parameters for the Axes in the example."""
    # only show the bottom spine
    ax.yaxis.set_major_locator(ticker.NullLocator())
    ax.spines[['left', 'right', 'top']].set_visible(False)

    # define tick positions
    ax.xaxis.set_major_locator(ticker.MultipleLocator(1.00))
    ax.xaxis.set_minor_locator(ticker.MultipleLocator(0.25))

    ax.xaxis.set_ticks_position('bottom')
    ax.tick_params(which='major', width=1.00, length=5)
    ax.tick_params(which='minor', width=0.75, length=2.5, labelsize=10)
    ax.set_xlim(0, 5)
    ax.set_ylim(0, 1)
    ax.text(0.0, 0.2, title, transform=ax.transAxes,
            fontsize=14, fontname='Monospace', color='tab:blue')


fig = plt.figure(figsize=(8, 8), layout='constrained')
fig0, fig1, fig2 = fig.subfigures(3, height_ratios=[1.5, 1.5, 7.5])

fig0.suptitle('String Formatting', fontsize=16, x=0, ha='left')
ax0 = fig0.subplots()

setup(ax0, title="'{x} km'")
ax0.xaxis.set_major_formatter('{x} km')

fig1.suptitle('Function Formatting', fontsize=16, x=0, ha='left')
ax1 = fig1.subplots()

setup(ax1, title="def(x, pos): return str(x-5)")
ax1.xaxis.set_major_formatter(lambda x, pos: str(x-5))

fig2.suptitle('Formatter Object Formatting', fontsize=16, x=0, ha='left')
axs2 = fig2.subplots(7, 1)
```

```
setup(axs2[0], title="NullFormatter()")
axs2[0].xaxis.set_major_formatter(ticker.NullFormatter())

setup(axs2[1], title="StrMethodFormatter('{x:.3f}')")
axs2[1].xaxis.set_major_formatter(ticker.StrMethodFormatter("{x:.3f}"))

setup(axs2[2], title="FormatStrFormatter('#%d')")
axs2[2].xaxis.set_major_formatter(ticker.FormatStrFormatter("#%d"))


def fmt_two_digits(x, pos):
    return f'[{x:.2f}]'


setup(axs2[3], title='FuncFormatter("[{:.2f}]".format)')
axs2[3].xaxis.set_major_formatter(ticker.FuncFormatter(fmt_two_digits))

setup(axs2[4], title="FixedFormatter(['A', 'B', 'C', 'D', 'E', 'F'])")
# FixedFormatter should only be used together with FixedLocator.
# Otherwise, one cannot be sure where the labels will end up.
positions = [0, 1, 2, 3, 4, 5]
labels = ['A', 'B', 'C', 'D', 'E', 'F']
axs2[4].xaxis.set_major_locator(ticker.FixedLocator(positions))
axs2[4].xaxis.set_major_formatter(ticker.FixedFormatter(labels))

setup(axs2[5], title="ScalarFormatter()")
axs2[5].xaxis.set_major_formatter(ticker.ScalarFormatter(useMathText=True))

setup(axs2[6], title="PercentFormatter(xmax=5)")
axs2[6].xaxis.set_major_formatter(ticker.PercentFormatter(xmax=5))
```

Styling ticks (tick parameters)

The appearance of ticks can be controlled at a low level by finding the individual `Tick` on the axis. However, usually it is simplest to use `tick_params` to change all the objects at once.

The `tick_params` method can change the properties of ticks:

- length
- direction (in or out of the frame)
- colors
- width and length
- and whether the ticks are drawn at the bottom, top, left, or right of the Axes.

It also can control the tick labels:

- labelsize (fontsize)
- labelcolor (color of the label)
- labelrotation
- labelbottom, labeltop, labelleft, labelright

In addition there is a *pad* keyword argument that specifies how far the tick label is from the tick.

Finally, the grid linestyles can be set:

- grid_color
- grid_alpha
- grid_linewidth
- grid_linestyle

All these properties can be restricted to one axis, and can be applied to just the major or minor ticks

```
fig, axs = plt.subplots(1, 2, figsize=(6.4, 3.2), layout='constrained')

for nn, ax in enumerate(axs):
    ax.plot(np.arange(100))
    if nn == 1:
        ax.grid('on')
        ax.tick_params(right=True, left=False, axis='y', color='r', length=16,
                       grid_color='none')
        ax.tick_params(axis='x', color='m', length=4, direction='in', width=4,
                       labelcolor='g', grid_color='b')
```

Tick objects

> Warning
> Ticks are managed through the autoscaling / view limit mechanism, which may create, move and delete ticks as necessary.
> Working with tick instances should only be an option of last resort and requires careful handling to not accidentally overwrite any manual changes through this mechanism.
> If a tick configuration can be achieved through `Axes.tick_params` (see

On the technical level, ticks are realized through `Tick` objects. They consist of

- `tick1line` / `tick2line` for the tick lines on either side of the axis.
- `label1` / `label2` for the tick labels on either side of the axis.
- `gridline` for the grid line.

These objects are publicly accessible through Axes methods and Axis methods and allow extreme customization such as coloring a specific tick line or tick label separately.

```
x = 1.1 * np.exp(-3 * np.random.random(500))
y = np.random.random(500)

fig, ax = plt.subplots()

ax.plot(x, y, 'o')
ax.grid()
ticks = ax.set_xticks([0, 0.2, 0.4, 0.6, 0.8, 1, 1.2])
ticks[5].label1.set_color("red")
ticks[5].tick1line.set_markeredgecolor("red")
ticks[5].gridline.set(visible=True, linestyle=':', linewidth=1.5, color='red')
```

Because of the managed nature of ticks, such operations only make sense for static output (e.g., when using the jupyter inline backend or saving the figure to a file) or when you are sure that the view limits of the axis will not change (e.g., when using a fixed locator).

In contrast, `Axes.tick_params` configures the default properties so that they are applied to all current and future ticks.

Axis scales

Source: https://matplotlib.org/stable/users/explain/axes/axes_scales.html

By default Matplotlib displays data on the axis using a linear scale. Matplotlib also supports logarithmic scales, and other less common scales as well. Usually this can be done directly by using the `set_xscale` or `set_yscale` methods.

```
import matplotlib.pyplot as plt
import numpy as np

import matplotlib.scale as mscale
from matplotlib.ticker import FixedLocator, NullFormatter

fig, axs = plt.subplot_mosaic([['linear', 'linear-log'],
                               ['log-linear', 'log-log']], layout='constrained')

x = np.arange(0, 3*np.pi, 0.1)
y = 2 * np.sin(x) + 3

ax = axs['linear']
ax.plot(x, y)
ax.set_xlabel('linear')
ax.set_ylabel('linear')

ax = axs['linear-log']
ax.plot(x, y)
ax.set_yscale('log')
ax.set_xlabel('linear')
ax.set_ylabel('log')

ax = axs['log-linear']
ax.plot(x, y)
ax.set_xscale('log')
ax.set_xlabel('log')
ax.set_ylabel('linear')

ax = axs['log-log']
ax.plot(x, y)
ax.set_xscale('log')
ax.set_yscale('log')
ax.set_xlabel('log')
ax.set_ylabel('log')
```

loglog and semilogx/y

The logarithmic axis is used so often that there are a set helper functions, that do the same thing: `semilogy`, `semilogx`, and `loglog`.

```
fig, axs = plt.subplot_mosaic([['linear', 'linear-log'],
                               ['log-linear', 'log-log']], layout='constrained')

x = np.arange(0, 3*np.pi, 0.1)
y = 2 * np.sin(x) + 3
```

```
ax = axs['linear']
ax.plot(x, y)
ax.set_xlabel('linear')
ax.set_ylabel('linear')
ax.set_title('plot(x, y)')

ax = axs['linear-log']
ax.semilogy(x, y)
ax.set_xlabel('linear')
ax.set_ylabel('log')
ax.set_title('semilogy(x, y)')

ax = axs['log-linear']
ax.semilogx(x, y)
ax.set_xlabel('log')
ax.set_ylabel('linear')
ax.set_title('semilogx(x, y)')

ax = axs['log-log']
ax.loglog(x, y)
ax.set_xlabel('log')
ax.set_ylabel('log')
ax.set_title('loglog(x, y)')
```

Other built-in scales

There are other scales that can be used. The list of registered scales can be returned from `scale.get_scale_names`:

```
print(mscale.get_scale_names())
```

```
fig, axs = plt.subplot_mosaic([['asinh', 'symlog'],
                               ['log', 'logit']], layout='constrained')

x = np.arange(0, 1000)

for name, ax in axs.items():
    if name in ['asinh', 'symlog']:
        yy = x - np.mean(x)
    elif name in ['logit']:
        yy = (x-np.min(x))
        yy = yy / np.max(np.abs(yy))
    else:
        yy = x

    ax.plot(yy, yy)
    ax.set_yscale(name)
    ax.set_title(name)
```

Optional arguments for scales

Some of the default scales have optional arguments. These are documented in the API reference for the respective scales at `scale`. One can change the base of the logarithm being plotted (eg 2 below) or the linear threshold range for `'symlog'`.

```
fig, axs = plt.subplot_mosaic([['log', 'symlog']], layout='constrained',
                              figsize=(6.4, 3))

for name, ax in axs.items():
    if name in ['log']:
        ax.plot(x, x)
        ax.set_yscale('log', base=2)
        ax.set_title('log base=2')
    else:
        ax.plot(x - np.mean(x), x - np.mean(x))
        ax.set_yscale('symlog', linthresh=100)
        ax.set_title('symlog linthresh=100')
```

Arbitrary function scales

Users can define a full scale class and pass that to `set_xscale` and `set_yscale` (see custom_scale). A short cut for this is to use the 'function' scale, and pass as extra arguments a `forward` and an `inverse` function. The following performs a Mercator transform to the y-axis.

Function Mercator transform

```
def forward(a):
    a = np.deg2rad(a)
    return np.rad2deg(np.log(np.abs(np.tan(a) + 1.0 / np.cos(a))))


def inverse(a):
    a = np.deg2rad(a)
    return np.rad2deg(np.arctan(np.sinh(a)))


t = np.arange(0, 170.0, 0.1)
s = t / 2.

fig, ax = plt.subplots(layout='constrained')
ax.plot(t, s, '-', lw=2)

ax.set_yscale('function', functions=(forward, inverse))
ax.set_title('function: Mercator')
ax.grid(True)
ax.set_xlim(0, 180)
ax.yaxis.set_minor_formatter(NullFormatter())
ax.yaxis.set_major_locator(FixedLocator(np.arange(0, 90, 10)))
```

What is a "scale"?

A scale is an object that gets attached to an axis. The class documentation is at `scale`. `set_xscale` and `set_yscale` set the scale on the respective Axis objects. You can determine the scale on an axis with `get_scale`:

```
fig, ax = plt.subplots(layout='constrained',
                              figsize=(3.2, 3))
ax.semilogy(x, x)

print(ax.xaxis.get_scale())
print(ax.yaxis.get_scale())
```

Setting a scale does three things. First it defines a transform on the axis that maps between data values to position along the axis. This transform can be accessed via `get_transform`:

```
print(ax.yaxis.get_transform())
```

Transforms on the axis are a relatively low-level concept, but is one of the important roles played by `set_scale`.

Setting the scale also sets default tick locators (`ticker`) and tick formatters appropriate for the scale. An axis with a 'log' scale has a `LogLocator` to pick ticks at decade intervals, and a `LogFormatter` to use scientific notation on the decades.

```
print('X axis')
print(ax.xaxis.get_major_locator())
print(ax.xaxis.get_major_formatter())

print('Y axis')
print(ax.yaxis.get_major_locator())
print(ax.yaxis.get_major_formatter())
```

Legend guide

Source: https://matplotlib.org/stable/users/explain/axes/legend_guide.html

This legend guide extends the `legend` docstring -please read it before proceeding with this guide.

This guide makes use of some common terms, which are documented here for clarity:

legend entry A legend is made up of one or more legend entries. An entry is made up of exactly one key and one label.

legend key The colored/patterned marker to the left of each legend label.

legend label The text which describes the handle represented by the key.

legend handle The original object which is used to generate an appropriate entry in the legend.

Controlling the legend entries

Calling `legend` with no arguments automatically fetches the legend handles and their associated labels. This functionality is equivalent to:

```
handles, labels = ax.get_legend_handles_labels()
ax.legend(handles, labels)
```

The `get_legend_handles_labels` function returns a list of handles/artists which exist on the Axes which can be used to generate entries for the resulting legend - it is worth noting however that not all artists can be added to a legend, at which point a "proxy" will have to be created (see proxy_legend_handles for further details).

> Note
> Artists with an empty string as label or with a label starting with an underscore, "_", will be ignored.

For full control of what is being added to the legend, it is common to pass the appropriate handles directly to `legend`:

```
fig, ax = plt.subplots()
line_up, = ax.plot([1, 2, 3], label='Line 2')
line_down, = ax.plot([3, 2, 1], label='Line 1')
ax.legend(handles=[line_up, line_down])
```

Renaming legend entries

When the labels cannot directly be set on the handles, they can be directly passed to `Axes.legend`:

```
fig, ax = plt.subplots()
line_up, = ax.plot([1, 2, 3], label='Line 2')
line_down, = ax.plot([3, 2, 1], label='Line 1')
ax.legend([line_up, line_down], ['Line Up', 'Line Down'])
```

If the handles are not directly accessible, for example when using some Third-party packages, they can be accessed via `Axes.get_legend_handles_labels`. Here we use a dictionary to rename existing labels:

```
my_map = {'Line Up':'Up', 'Line Down':'Down'}

handles, labels = ax.get_legend_handles_labels()
ax.legend(handles, [my_map[l] for l in labels])
```

Creating artists specifically for adding to the legend (aka. Proxy artists)

Not all handles can be turned into legend entries automatically, so it is often necessary to create an artist which *can*. Legend handles don't have to exist on the Figure or Axes in order to be used.

Suppose we wanted to create a legend which has an entry for some data which is represented by a red color:

```
import matplotlib.pyplot as plt

import matplotlib.patches as mpatches

fig, ax = plt.subplots()
red_patch = mpatches.Patch(color='red', label='The red data')
ax.legend(handles=[red_patch])

plt.show()
```

There are many supported legend handles. Instead of creating a patch of color we could have created a line with a marker:

```
import matplotlib.lines as mlines

fig, ax = plt.subplots()
blue_line = mlines.Line2D([], [], color='blue', marker='*',
                          markersize=15, label='Blue stars')
ax.legend(handles=[blue_line])

plt.show()
```

Legend location

The location of the legend can be specified by the keyword argument *loc*. Please see the documentation at `legend` for more details.

The `bbox_to_anchor` keyword gives a great degree of control for manual legend placement. For example, if you want your Axes legend located at the figure's top right-hand corner instead of the Axes' corner, simply specify the corner's location and the coordinate system of that location:

```
ax.legend(bbox_to_anchor=(1, 1),
          bbox_transform=fig.transFigure)
```

More examples of custom legend placement:

```
fig, ax_dict = plt.subplot_mosaic([['top', 'top'], ['bottom', 'BLANK']],
                                  empty_sentinel="BLANK")
ax_dict['top'].plot([1, 2, 3], label="test1")
ax_dict['top'].plot([3, 2, 1], label="test2")
# Place a legend above this subplot, expanding itself to
# fully use the given bounding box.
ax_dict['top'].legend(bbox_to_anchor=(0., 1.02, 1., .102), loc='lower left',
                      ncols=2, mode="expand", borderaxespad=0.)
```

```
ax_dict['bottom'].plot([1, 2, 3], label="test1")
ax_dict['bottom'].plot([3, 2, 1], label="test2")
# Place a legend to the right of this smaller subplot.
ax_dict['bottom'].legend(bbox_to_anchor=(1.05, 1),
                         loc='upper left', borderaxespad=0.)
```

Figure legends

Sometimes it makes more sense to place a legend relative to the (sub)figure rather than individual Axes. By using *constrained layout* and specifying "outside" at the beginning of the *loc* keyword argument, the legend is drawn outside the Axes on the (sub)figure.

```
fig, axs = plt.subplot_mosaic([['left', 'right']], layout='constrained')

axs['left'].plot([1, 2, 3], label="test1")
axs['left'].plot([3, 2, 1], label="test2")

axs['right'].plot([1, 2, 3], 'C2', label="test3")
axs['right'].plot([3, 2, 1], 'C3', label="test4")
# Place a legend to the right of this smaller subplot.
fig.legend(loc='outside upper right')
```

This accepts a slightly different grammar than the normal *loc* keyword, where "outside right upper" is different from "outside upper right".

```
ucl = ['upper', 'center', 'lower']
lcr = ['left', 'center', 'right']
fig, ax = plt.subplots(figsize=(6, 4), layout='constrained', facecolor='0.95')

ax.plot([1, 2], [1, 2], label='TEST')
# Place a legend to the right of this smaller subplot.
for loc in [
        'outside upper left',
        'outside upper center',
        'outside upper right',
        'outside lower left',
        'outside lower center',
        'outside lower right']:
    fig.legend(loc=loc, title=loc)

fig, ax = plt.subplots(figsize=(6, 4), layout='constrained', facecolor='0.95')
ax.plot([1, 2], [1, 2], label='test')

for loc in [
        'outside left upper',
        'outside right upper',
        'outside left center',
        'outside right center',
        'outside left lower',
        'outside right lower']:
    fig.legend(loc=loc, title=loc)
```

Multiple legends on the same Axes

Sometimes it is more clear to split legend entries across multiple legends. Whilst the instinctive approach to doing this might be to call the `legend` function multiple times, you will find that

only one legend ever exists on the Axes. This has been done so that it is possible to call `legend` repeatedly to update the legend to the latest handles on the Axes. To keep old legend instances, we must add them manually to the Axes:

```
fig, ax = plt.subplots()
line1, = ax.plot([1, 2, 3], label="Line 1", linestyle='--')
line2, = ax.plot([3, 2, 1], label="Line 2", linewidth=4)

# Create a legend for the first line.
first_legend = ax.legend(handles=[line1], loc='upper right')

# Add the legend manually to the Axes.
ax.add_artist(first_legend)

# Create another legend for the second line.
ax.legend(handles=[line2], loc='lower right')

plt.show()
```

Legend handlers

In order to create legend entries, handles are given as an argument to an appropriate `HandlerBase` subclass. The choice of handler subclass is determined by the following rules:

1. Update `get_legend_handler_map` with the value in the `handler_map` keyword.
2. Check if the `handle` is in the newly created `handler_map`.
3. Check if the type of `handle` is in the newly created `handler_map`.
4. Check if any of the types in the `handle`'s mro is in the newly created `handler_map`.

For completeness, this logic is mostly implemented in `get_legend_handler`.

All of this flexibility means that we have the necessary hooks to implement custom handlers for our own type of legend key.

The simplest example of using custom handlers is to instantiate one of the existing `legend_handler.HandlerBase` subclasses. For the sake of simplicity, let's choose `legend_handler.HandlerLine2D` which accepts a *numpoints* argument (numpoints is also a keyword on the `legend` function for convenience). We can then pass the mapping of instance to Handler as a keyword to legend.

```
from matplotlib.legend_handler import HandlerLine2D

fig, ax = plt.subplots()
line1, = ax.plot([3, 2, 1], marker='o', label='Line 1')
line2, = ax.plot([1, 2, 3], marker='o', label='Line 2')

ax.legend(handler_map={line1: HandlerLine2D(numpoints=4)}, handlelength=4)
```

As you can see, ”Line 1” now has 4 marker points, where ”Line 2” has 2 (the default). We have also increased the length of the handles with the `handlelength` keyword to fit the larger legend entry. Try the above code, only change the map's key from `line1` to `type(line1)`. Notice how now both `Line2D` instances get 4 markers.

Along with handlers for complex plot types such as errorbars, stem plots and histograms, the default `handler_map` has a special `tuple` handler (`legend_handler.HandlerTuple`) which simply plots the handles on top of one another for each item in the given tuple. The following example demonstrates combining two legend keys on top of one another:

```
from numpy.random import randn

z = randn(10)

fig, ax = plt.subplots()
red_dot, = ax.plot(z, "ro", markersize=15)
# Put a white cross over some of the data.
white_cross, = ax.plot(z[:5], "w+", markeredgewidth=3, markersize=15)

ax.legend([red_dot, (red_dot, white_cross)], ["Attr A", "Attr A+B"])
```

The `legend_handler.HandlerTuple` class can also be used to assign several legend keys to the same entry:

```
from matplotlib.legend_handler import HandlerLine2D, HandlerTuple

fig, ax = plt.subplots()
p1, = ax.plot([1, 2.5, 3], 'r-d')
p2, = ax.plot([3, 2, 1], 'k-o')

l = ax.legend([(p1, p2)], ['Two keys'], numpoints=1,
              handler_map={tuple: HandlerTuple(ndivide=None)})
```

Implementing a custom legend handler

A custom handler can be implemented to turn any handle into a legend key (handles don't necessarily need to be matplotlib artists). The handler must implement a `legend_artist` method which returns a single artist for the legend to use. The required signature for `legend_artist` is documented at `legend_artist`.

```
import matplotlib.patches as mpatches


class AnyObject:
    pass


class AnyObjectHandler:
    def legend_artist(self, legend, orig_handle, fontsize, handlebox):
        x0, y0 = handlebox.xdescent, handlebox.ydescent
        width, height = handlebox.width, handlebox.height
        patch = mpatches.Rectangle([x0, y0], width, height, facecolor='red',
                                   edgecolor='black', hatch='xx', lw=3,
                                   transform=handlebox.get_transform())
        handlebox.add_artist(patch)
        return patch

fig, ax = plt.subplots()

ax.legend([AnyObject()], ['My first handler'],
          handler_map={AnyObject: AnyObjectHandler()})
```

Alternatively, had we wanted to globally accept `AnyObject` instances without needing to manually set the *handler_map* keyword all the time, we could have registered the new handler with:

```
from matplotlib.legend import Legend
Legend.update_default_handler_map({AnyObject: AnyObjectHandler()})
```

Whilst the power here is clear, remember that there are already many handlers implemented

and what you want to achieve may already be easily possible with existing classes. For example, to produce elliptical legend keys, rather than rectangular ones:

```
from matplotlib.legend_handler import HandlerPatch


class HandlerEllipse(HandlerPatch):
    def create_artists(self, legend, orig_handle,
                       xdescent, ydescent, width, height, fontsize, trans):
        center = 0.5 * width - 0.5 * xdescent, 0.5 * height - 0.5 * ydescent
        p = mpatches.Ellipse(xy=center, width=width + xdescent,
                             height=height + ydescent)
        self.update_prop(p, orig_handle, legend)
        p.set_transform(trans)
        return [p]


c = mpatches.Circle((0.5, 0.5), 0.25, facecolor="green",
                    edgecolor="red", linewidth=3)

fig, ax = plt.subplots()

ax.add_patch(c)
ax.legend([c], ["An ellipse, not a rectangle"],
          handler_map={mpatches.Circle: HandlerEllipse()})
```

Specifying colors

Source: *https://matplotlib.org/stable/users/explain/colors/colors.html*

Color formats

Matplotlib recognizes the following formats to specify a color.

Format	Example
RGB or RGBA (red, green, blue, alpha) tuple of float values in a closed interval [0, 1].	• `(0.1, 0.2, 0.5)` • `(0.1, 0.2, 0.5, 0.3)`
Case-insensitive hex RGB or RGBA string.	• `'#0f0f0f'` • `'#0f0f0f80'`
Case-insensitive RGB or RGBA string equivalent hex shorthand of duplicated characters.	• `'#abc'` as `'#aabbcc'` • `'#fb1'` as `'#ffbb11'`
String representation of float value in closed interval `[0, 1]` for grayscale values.	• `'0'` as black • `'1'` as white • `'0.8'` as light gray
Single character shorthand notation for some basic colors. **Note** The colors green, cyan, magenta, and yellow do not coincide with X11/CSS4 colors. Their particular shades were chosen for better visibility of colored lines against typical backgrounds.	• `'b'` as blue • `'g'` as green • `'r'` as red • `'c'` as cyan • `'m'` as magenta • `'y'` as yellow • `'k'` as black • `'w'` as white
Case-insensitive X11/CSS4 color name with no spaces.	• `'aquamarine'` • `'mediumseagreen'`

Format	Example
Case-insensitive color name from xkcd color survey with `'xkcd:'` prefix.	• `'xkcd:sky blue'` • `'xkcd:eggshell'`
Case-insensitive Tableau Colors from 'T10' categorical palette. **Note** This is the default color cycle.	• `'tab:blue'` • `'tab:orange'` • `'tab:green'` • `'tab:red'` • `'tab:purple'` • `'tab:brown'` • `'tab:pink'` • `'tab:gray'` • `'tab:olive'` • `'tab:cyan'`
”CN” color spec where `'C'` precedes a number acting as an index into the default property cycle. **Note** The cycle comes from the global `axes.prop_cycle` , not an Axes-local cycle set by ~.Axes.set_prop_cycle. Matplotlib indexes color at draw time and defaults to black if cycle does not include color.	• `'C0'` • `'C1'`
Tuple of one of the above color formats and an alpha float. 3.8	• `('green', 0.3)` • `('#f00', 0.9)`
The special value ”none” is fully transparent, i.e. equivalent to a RGBA value `(0.0, 0.0, 0.0, 0.0)`	• `'none'`

See also
The following links provide more information on colors in Matplotlib.
- /gallery/color/color_demo Example
- `matplotlib.colors` API
- /gallery/color/named_colors Example

”Red”, ”Green”, and ”Blue” are the intensities of those colors. In combination, they represent the colorspace.

Transparency

The *alpha* value of a color specifies its transparency, where 0 is fully transparent and 1 is fully opaque. When a color is semi-transparent, the background color will show through.

The *alpha* value determines the resulting color by blending the foreground color with the background color according to the formula

$$RGB_{result} = RGB_{background} * (1 - \alpha) + RGB_{foreground} * \alpha$$

The following plot illustrates the effect of transparency.

```
import matplotlib.pyplot as plt
import numpy as np

from matplotlib.patches import Rectangle

fig, ax = plt.subplots(figsize=(6.5, 1.65), layout='constrained')
ax.add_patch(Rectangle((-0.2, -0.35), 11.2, 0.7, color='C1', alpha=0.8))
for i, alpha in enumerate(np.linspace(0, 1, 11)):
    ax.add_patch(Rectangle((i, 0.05), 0.8, 0.6, alpha=alpha, zorder=0))
    ax.text(i+0.4, 0.85, f"{alpha:.1f}", ha='center')
    ax.add_patch(Rectangle((i, -0.05), 0.8, -0.6, alpha=alpha, zorder=2))
ax.set_xlim(-0.2, 13)
ax.set_ylim(-1, 1)
ax.set_title('alpha values')
ax.text(11.3, 0.6, 'zorder=1', va='center', color='C0')
ax.text(11.3, 0, 'zorder=2\nalpha=0.8', va='center', color='C1')
ax.text(11.3, -0.6, 'zorder=3', va='center', color='C0')
ax.axis('off')
```

The orange rectangle is semi-transparent with *alpha* = 0.8. The top row of blue squares is drawn below and the bottom row of blue squares is drawn on top of the orange rectangle.

See also /gallery/misc/zorder_demo to learn more on the drawing order.

"CN" color selection

Matplotlib converts "CN" colors to RGBA when drawing Artists. The color_cycle section contains additional information about controlling colors and style properties.

```
import matplotlib.pyplot as plt
import numpy as np

import matplotlib as mpl

th = np.linspace(0, 2*np.pi, 128)


def demo(sty):
    mpl.style.use(sty)
    fig, ax = plt.subplots(figsize=(3, 3))

    ax.set_title(f'style: {sty!r}', color='C0')

    ax.plot(th, np.cos(th), 'C1', label='C1')
    ax.plot(th, np.sin(th), 'C2', label='C2')
    ax.legend()


demo('default')
demo('seaborn-v0_8')
```

The first color `'C0'` is the title. Each plot uses the second and third colors of each style's `axes.prop_cycle`. They are `'C1'` and `'C2'`, respectively.

Comparison between X11/CSS4 and xkcd colors

The xkcd colors come from a user survey conducted by the webcomic xkcd.

95 out of the 148 X11/CSS4 color names also appear in the xkcd color survey. Almost all of them map to different color values in the X11/CSS4 and in the xkcd palette. Only 'black', 'white' and 'cyan' are identical.

For example, `'blue'` maps to `'#0000FF'` whereas `'xkcd:blue'` maps to `'#0343DF'`. Due to these name collisions, all xkcd colors have the `'xkcd:'` prefix.

The visual below shows name collisions. Color names where color values agree are in bold.

```
import matplotlib.colors as mcolors
import matplotlib.patches as mpatch

overlap = {name for name in mcolors.CSS4_COLORS
           if f'xkcd:{name}' in mcolors.XKCD_COLORS}

fig = plt.figure(figsize=[9, 5])
ax = fig.add_axes((0, 0, 1, 1))

n_groups = 3
n_rows = len(overlap) // n_groups + 1

for j, color_name in enumerate(sorted(overlap)):
    css4 = mcolors.CSS4_COLORS[color_name]
    xkcd = mcolors.XKCD_COLORS[f'xkcd:{color_name}'].upper()

    # Pick text colour based on perceived luminance.
    rgba = mcolors.to_rgba_array([css4, xkcd])
    luma = 0.299 * rgba[:, 0] + 0.587 * rgba[:, 1] + 0.114 * rgba[:, 2]
    css4_text_color = 'k' if luma[0] > 0.5 else 'w'
    xkcd_text_color = 'k' if luma[1] > 0.5 else 'w'

    col_shift = (j // n_rows) * 3
    y_pos = j % n_rows
    text_args = dict(fontsize=10, weight='bold' if css4 == xkcd else None)
    ax.add_patch(mpatch.Rectangle((0 + col_shift, y_pos), 1, 1, color=css4))
    ax.add_patch(mpatch.Rectangle((1 + col_shift, y_pos), 1, 1, color=xkcd))
    ax.text(0.5 + col_shift, y_pos + .7, css4,
            color=css4_text_color, ha='center', **text_args)
    ax.text(1.5 + col_shift, y_pos + .7, xkcd,
            color=xkcd_text_color, ha='center', **text_args)
    ax.text(2 + col_shift, y_pos + .7, f'  {color_name}', **text_args)

for g in range(n_groups):
    ax.hlines(range(n_rows), 3*g, 3*g + 2.8, color='0.7', linewidth=1)
    ax.text(0.5 + 3*g, -0.3, 'X11/CSS4', ha='center')
    ax.text(1.5 + 3*g, -0.3, 'xkcd', ha='center')

ax.set_xlim(0, 3 * n_groups)
ax.set_ylim(n_rows, -1)
ax.axis('off')

plt.show()
```

Choosing colormaps

Source: https://matplotlib.org/stable/users/explain/colors/colormaps.html

Matplotlib has a number of built-in colormaps accessible via `matplotlib.colormaps`. There are also external libraries that have many extra colormaps, which can be viewed in the Third-party colormaps section of the Matplotlib documentation. Here we briefly discuss how to choose between the many options. For help on creating your own colormaps, see colormap-manipulation.

To get a list of all registered colormaps, you can do:

```
from matplotlib import colormaps
list(colormaps)
```

Overview

The idea behind choosing a good colormap is to find a good representation in 3D colorspace for your data set. The best colormap for any given data set depends on many things including:

- Whether representing form or metric data ([Ware])
- Your knowledge of the data set (*e.g.*, is there a critical value from which the other values deviate?)
- If there is an intuitive color scheme for the parameter you are plotting
- If there is a standard in the field the audience may be expecting

For many applications, a perceptually uniform colormap is the best choice; i.e. a colormap in which equal steps in data are perceived as equal steps in the color space. Researchers have found that the human brain perceives changes in the lightness parameter as changes in the data much better than, for example, changes in hue. Therefore, colormaps which have monotonically increasing lightness through the colormap will be better interpreted by the viewer. Wonderful examples of perceptually uniform colormaps can be found in the Third-party colormaps section as well.

Color can be represented in 3D space in various ways. One way to represent color is using CIELAB. In CIELAB, color space is represented by lightness, L^*; red-green, a^*; and yellow-blue, b^*. The lightness parameter L^* can then be used to learn more about how the matplotlib colormaps will be perceived by viewers.

An excellent starting resource for learning about human perception of colormaps is from [IBM].

Classes of colormaps

Colormaps are often split into several categories based on their function (see, *e.g.*, [Moreland]):

1. Sequential: change in lightness and often saturation of color incrementally, often using a single hue; should be used for representing information that has ordering.
2. Diverging: change in lightness and possibly saturation of two different colors that meet in the middle at an unsaturated color; should be used when the information being plotted has a critical middle value, such as topography or when the data deviates around zero.
3. Cyclic: change in lightness of two different colors that meet in the middle and beginning/end at an unsaturated color; should be used for values that wrap around at the endpoints, such as phase angle, wind direction, or time of day.

4. Qualitative: often are miscellaneous colors; should be used to represent information which does not have ordering or relationships.

```
from colorspacious import cspace_converter

import matplotlib.pyplot as plt
import numpy as np

import matplotlib as mpl
```

First, we'll show the range of each colormap. Note that some seem to change more "quickly" than others.

```
cmaps = {}

gradient = np.linspace(0, 1, 256)
gradient = np.vstack((gradient, gradient))


def plot_color_gradients(category, cmap_list):
    # Create figure and adjust figure height to number of colormaps
    nrows = len(cmap_list)
    figh = 0.35 + 0.15 + (nrows + (nrows - 1) * 0.1) * 0.22
    fig, axs = plt.subplots(nrows=nrows + 1, figsize=(6.4, figh))
    fig.subplots_adjust(top=1 - 0.35 / figh, bottom=0.15 / figh,
                        left=0.2, right=0.99)
    axs[0].set_title(f'{category} colormaps', fontsize=14)

    for ax, name in zip(axs, cmap_list):
        ax.imshow(gradient, aspect='auto', cmap=mpl.colormaps[name])
        ax.text(-0.01, 0.5, name, va='center', ha='right', fontsize=10,
                transform=ax.transAxes)

    # Turn off *all* ticks & spines, not just the ones with colormaps.
    for ax in axs:
        ax.set_axis_off()

    # Save colormap list for later.
    cmaps[category] = cmap_list
```

Sequential

For the Sequential plots, the lightness value increases monotonically through the colormaps. This is good. Some of the L^* values in the colormaps span from 0 to 100 (binary and the other grayscale), and others start around $L^* = 20$. Those that have a smaller range of L^* will accordingly have a smaller perceptual range. Note also that the L^* function varies amongst the colormaps: some are approximately linear in L^* and others are more curved.

```
plot_color_gradients('Perceptually Uniform Sequential',
                     ['viridis', 'plasma', 'inferno', 'magma', 'cividis'])
```

```
plot_color_gradients('Sequential',
                     ['Greys', 'Purples', 'Blues', 'Greens', 'Oranges', 'Reds',
                      'YlOrBr', 'YlOrRd', 'OrRd', 'PuRd', 'RdPu', 'BuPu',
                      'GnBu', 'PuBu', 'YlGnBu', 'PuBuGn', 'BuGn', 'YlGn'])
```

Sequential2

Many of the L^* values from the Sequential2 plots are monotonically increasing, but some (autumn, cool, spring, and winter) plateau or even go both up and down in L^* space. Others (afmhot, copper, gist_heat, and hot) have kinks in the L^* functions. Data that is being represented in a region of the colormap that is at a plateau or kink will lead to a perception of banding of the data in those values in the colormap (see [mycarta-banding] for an excellent example of this).

```
plot_color_gradients('Sequential (2)',
                     ['gray', 'bone', 'pink', 'spring', 'summer', 'autumn',
                      'winter', 'cool', 'Wistia', 'hot', 'afmhot', 'gist_heat',
                      'copper'])
```

Discouraged

For backward compatibility we additionally support the following colormap names, which are identical to other builtin colormaps. Their use is discouraged. Use the suggested replacement instead.

Colormap	Use identical replacement instead
gist_gray	gray
gist_yarg	gray_r
binary	gray_r

Diverging

For the Diverging maps, we want to have monotonically increasing L^* values up to a maximum, which should be close to $L^* = 100$, followed by monotonically decreasing L^* values. We are looking for approximately equal minimum L^* values at opposite ends of the colormap. By these measures, BrBG and RdBu are good options. coolwarm is a good option, but it doesn't span a wide range of L^* values (see grayscale section below).

Berlin, Managua, and Vanimo are dark-mode diverging colormaps, with minimum lightness at the center, and maximum at the extremes. These are taken from F. Crameri's [scientific-colour-maps] version 8.0.1.

```
plot_color_gradients('Diverging',
                     ['PiYG', 'PRGn', 'BrBG', 'PuOr', 'RdGy', 'RdBu', 'RdYlBu',
                      'RdYlGn', 'Spectral', 'coolwarm', 'bwr', 'seismic',
                      'berlin', 'managua', 'vanimo'])
```

Cyclic

For Cyclic maps, we want to start and end on the same color, and meet a symmetric center point in the middle. L^* should change monotonically from start to middle, and inversely from middle to end. It should be symmetric on the increasing and decreasing side, and only differ in hue. At the ends and middle, L^* will reverse direction, which should be smoothed in L^* space to reduce artifacts. See [kovesi-colormaps] for more information on the design of cyclic maps.

The often-used HSV colormap is included in this set of colormaps, although it is not symmetric to a center point. Additionally, the L^* values vary widely throughout the colormap, making it a poor choice for representing data for viewers to see perceptually. See an extension on this idea at [mycarta-jet].

```
plot_color_gradients('Cyclic', ['twilight', 'twilight_shifted', 'hsv'])
```

Qualitative

Qualitative colormaps are not aimed at being perceptual maps, but looking at the lightness parameter can verify that for us. The L^* values move all over the place throughout the colormap, and are clearly not monotonically increasing. These would not be good options for use as perceptual colormaps.

```
plot_color_gradients('Qualitative',
                     ['Pastel1', 'Pastel2', 'Paired', 'Accent', 'okabe_ito',
                      'Dark2', 'Set1', 'Set2', 'Set3', 'tab10', 'tab20',
                      'tab20b', 'tab20c'])
```

Miscellaneous

Some of the miscellaneous colormaps have particular uses for which they have been created. For example, gist_earth, ocean, and terrain all seem to be created for plotting topography (green/brown) and water depths (blue) together. We would expect to see a divergence in these colormaps, then, but multiple kinks may not be ideal, such as in gist_earth and terrain. CMRmap was created to convert well to grayscale, though it does appear to have some small kinks in L^*. cubehelix was created to vary smoothly in both lightness and hue, but appears to have a small hump in the green hue area. turbo was created to display depth and disparity data.

The often-used jet colormap is included in this set of colormaps. We can see that the L^* values vary widely throughout the colormap, making it a poor choice for representing data for viewers to see perceptually. See an extension on this idea at [mycarta-jet] and [turbo].

```
plot_color_gradients('Miscellaneous',
                     ['flag', 'prism', 'ocean', 'gist_earth', 'terrain',
                      'gist_stern', 'gnuplot', 'gnuplot2', 'CMRmap',
                      'cubehelix', 'brg', 'gist_rainbow', 'rainbow', 'jet',
                      'turbo', 'nipy_spectral', 'gist_ncar'])

plt.show()
```

Lightness of Matplotlib colormaps

Here we examine the lightness values of the matplotlib colormaps. Note that some documentation on the colormaps is available ([list-colormaps]).

```
mpl.rcParams.update({'font.size': 12})

# Number of colormap per subplot for particular cmap categories
_DSUBS = {'Perceptually Uniform Sequential': 5, 'Sequential': 6,
          'Sequential (2)': 6, 'Diverging': 6, 'Cyclic': 3,
          'Qualitative': 4, 'Miscellaneous': 6}

# Spacing between the colormaps of a subplot
_DC = {'Perceptually Uniform Sequential': 1.4, 'Sequential': 0.7,
       'Sequential (2)': 1.4, 'Diverging': 1.4, 'Cyclic': 1.4,
       'Qualitative': 1.4, 'Miscellaneous': 1.4}

# Indices to step through colormap
x = np.linspace(0.0, 1.0, 100)

# Do plot
for cmap_category, cmap_list in cmaps.items():
```

```
    # Do subplots so that colormaps have enough space.
    # Default is 6 colormaps per subplot.
    dsub = _DSUBS.get(cmap_category, 6)
    nsubplots = int(np.ceil(len(cmap_list) / dsub))

    # squeeze=False to handle similarly the case of a single subplot
    fig, axs = plt.subplots(nrows=nsubplots, squeeze=False,
                            figsize=(7, 2.6*nsubplots))

    for i, ax in enumerate(axs.flat):

        locs = []  # locations for text labels

        for j, cmap in enumerate(cmap_list[i*dsub:(i+1)*dsub]):

            # Get RGB values for colormap and convert the colormap in
            # CAM02-UCS colorspace.  lab[0, :, 0] is the lightness.
            rgb = mpl.colormaps[cmap](x)[np.newaxis, :, :3]
            lab = cspace_converter("sRGB1", "CAM02-UCS")(rgb)

            # Plot colormap L values.  Do separately for each category
            # so each plot can be pretty.  To make scatter markers change
            # color along plot:
            # https://stackoverflow.com/q/8202605/

            if cmap_category == 'Sequential':
                # These colormaps all start at high lightness, but we want them
                # reversed to look nice in the plot, so reverse the order.
                y_ = lab[0, ::-1, 0]
                c_ = x[::-1]
            else:
                y_ = lab[0, :, 0]
                c_ = x

            dc = _DC.get(cmap_category, 1.4)  # cmaps horizontal spacing
            ax.scatter(x + j*dc, y_, c=c_, cmap=cmap, s=300, linewidths=0.0)

            # Store locations for colormap labels
            if cmap_category in ('Perceptually Uniform Sequential',
                                 'Sequential'):
                locs.append(x[-1] + j*dc)
            elif cmap_category in ('Diverging', 'Qualitative', 'Cyclic',
                                   'Miscellaneous', 'Sequential (2)'):
                locs.append(x[int(x.size/2.)] + j*dc)

        # Set up the axis limits:
        #   * the 1st subplot is used as a reference for the x-axis limits
        #   * lightness values goes from 0 to 100 (y-axis limits)
        ax.set_xlim(axs[0, 0].get_xlim())
        ax.set_ylim(0.0, 100.0)

        # Set up labels for colormaps
        ax.xaxis.set_ticks_position('top')
        ticker = mpl.ticker.FixedLocator(locs)
        ax.xaxis.set_major_locator(ticker)
        formatter = mpl.ticker.FixedFormatter(cmap_list[i*dsub:(i+1)*dsub])
        ax.xaxis.set_major_formatter(formatter)
        ax.xaxis.set_tick_params(rotation=50)
        ax.set_ylabel('Lightness $L^*$', fontsize=12)
```

```
    ax.set_xlabel(cmap_category + ' colormaps', fontsize=14)

    fig.tight_layout(h_pad=0.0, pad=1.5)
    plt.show()
```

Grayscale conversion

It is important to pay attention to conversion to grayscale for color plots, since they may be printed on black and white printers. If not carefully considered, your readers may end up with indecipherable plots because the grayscale changes unpredictably through the colormap.

Conversion to grayscale is done in many different ways [bw]. Some of the better ones use a linear combination of the rgb values of a pixel, but weighted according to how we perceive color intensity. A nonlinear method of conversion to grayscale is to use the L^* values of the pixels. In general, similar principles apply for this question as they do for presenting one's information perceptually; that is, if a colormap is chosen that is monotonically increasing in L^* values, it will print in a reasonable manner to grayscale.

With this in mind, we see that the Sequential colormaps have reasonable representations in grayscale. Some of the Sequential2 colormaps have decent enough grayscale representations, though some (autumn, spring, summer, winter) have very little grayscale change. If a colormap like this was used in a plot and then the plot was printed to grayscale, a lot of the information may map to the same gray values. The Diverging colormaps mostly vary from darker gray on the outer edges to white in the middle. Some (PuOr and seismic) have noticeably darker gray on one side than the other and therefore are not very symmetric. coolwarm has little range of gray scale and would print to a more uniform plot, losing a lot of detail. Note that overlaid, labeled contours could help differentiate between one side of the colormap vs. the other since color cannot be used once a plot is printed to grayscale. Many of the Qualitative and Miscellaneous colormaps, such as Accent, hsv, jet and turbo, change from darker to lighter and back to darker grey throughout the colormap. This would make it impossible for a viewer to interpret the information in a plot once it is printed in grayscale.

```
mpl.rcParams.update({'font.size': 14})

# Indices to step through colormap.
x = np.linspace(0.0, 1.0, 100)

gradient = np.linspace(0, 1, 256)
gradient = np.vstack((gradient, gradient))


def plot_color_gradients(cmap_category, cmap_list):
    fig, axs = plt.subplots(nrows=len(cmap_list), ncols=2)
    fig.subplots_adjust(top=0.95, bottom=0.01, left=0.2, right=0.99,
                        wspace=0.05)
    fig.suptitle(cmap_category + ' colormaps', fontsize=14, y=1.0, x=0.6)

    for ax, name in zip(axs, cmap_list):

        # Get RGB values for colormap.
        rgb = mpl.colormaps[name](x)[np.newaxis, :, :3]

        # Get colormap in CAM02-UCS colorspace. We want the lightness.
        lab = cspace_converter("sRGB1", "CAM02-UCS")(rgb)
        L = lab[0, :, 0]
        L = np.float32(np.vstack((L, L, L)))
```

```
        ax[0].imshow(gradient, aspect='auto', cmap=mpl.colormaps[name])
        ax[1].imshow(L, aspect='auto', cmap='binary_r', vmin=0., vmax=100.)
        pos = list(ax[0].get_position().bounds)
        x_text = pos[0] - 0.01
        y_text = pos[1] + pos[3]/2.
        fig.text(x_text, y_text, name, va='center', ha='right', fontsize=10)

    # Turn off *all* ticks & spines, not just the ones with colormaps.
    for ax in axs.flat:
        ax.set_axis_off()

    plt.show()


for cmap_category, cmap_list in cmaps.items():

    plot_color_gradients(cmap_category, cmap_list)
```

Color vision deficiencies

There is a lot of information available about color blindness (*e.g.*, [colorblindness]). Additionally, there are tools available to convert images to how they look for different types of color vision deficiencies.

The most common form of color vision deficiency involves differentiating between red and green. Thus, avoiding colormaps with both red and green will avoid many problems in general.

References

IBM https://doi.org/10.1109/VISUAL.1995.480803

Moreland http://www.kennethmoreland.com/color-maps/ColorMapsExpanded.pdf

Ware https://dl.acm.org/doi/10.1109/38.7760

bw https://tannerhelland.com/3643/grayscale-image-algorithm-vb6/

colorblindness http://www.color-blindness.com/

kovesi-colormaps https://arxiv.org/abs/1509.03700

list-colormaps https://gist.github.com/endolith/2719900#id7

mycarta-banding https://mycarta.wordpress.com/2012/10/14/the-rainbow-is-deadlong-live-the-rainbow-part-4-cie-lab-heated-body/

mycarta-jet https://mycarta.wordpress.com/2012/10/06/the-rainbow-is-deadlong-live-the-rainbow-part-3/

scientific-colour-maps https://doi.org/10.5281/zenodo.1243862

turbo https://ai.googleblog.com/2019/08/turbo-improved-rainbow-colormap-for.html

Text in Matplotlib

Source: https://matplotlib.org/stable/users/explain/text/text_intro.html

Matplotlib has extensive text support, including support for mathematical expressions, truetype support for raster and vector outputs, newline separated text with arbitrary rotations, and Unicode support.

Because it embeds fonts directly in output documents, e.g., for postscript or PDF, what you see on the screen is what you get in the hardcopy. FreeType support produces very nice, antialiased fonts, that look good even at small raster sizes. Matplotlib includes its own `matplotlib.font_manager` (thanks to Paul Barrett), which implements a cross platform, W3C compliant font finding algorithm.

The user has a great deal of control over text properties (font size, font weight, text location and color, etc.) with sensible defaults set in the rc file. And significantly, for those interested in mathematical or scientific figures, Matplotlib implements a large number of TeX math symbols and commands, supporting `mathematical expressions <mathtext>` anywhere in your figure.

Basic text commands

The following commands are used to create text in the implicit and explicit interfaces (see api_interfaces for an explanation of the tradeoffs):

implicit API	explicit API	description
`text` ‘text	‘ Add text at	an arbitrary location of the `Axes`.
`annotate` “annot	ate“ Add an annot	ation, with an optional arrow, at an arbitrary location of the `Axes`.
`xlabel` “set_x	label“ Add a label	to the `Axes`'s x-axis.
`ylabel` “set_y	label“ Add a label	to the `Axes`'s y-axis.
`title` “set_t	itle“ Add a title	to the `Axes`.
`figtext` ‘text	‘ Add text at an	arbitrary location of the `Figure`.
`suptitle` “supti	tle“ Add a title to	the `Figure`.

All of these functions create and return a `Text` instance, which can be configured with a variety of font and other properties. The example below shows all of these commands in action, and more detail is provided in the sections that follow.

```
import matplotlib.pyplot as plt

import matplotlib

fig = plt.figure()
ax = fig.add_subplot()
fig.subplots_adjust(top=0.85)

# Set titles for the figure and the subplot respectively
fig.suptitle('bold figure suptitle', fontsize=14, fontweight='bold')
ax.set_title('axes title')

ax.set_xlabel('xlabel')
```

```
ax.set_ylabel('ylabel')

# Set both x- and y-axis limits to [0, 10] instead of default [0, 1]
ax.axis([0, 10, 0, 10])

ax.text(3, 8, 'boxed italics text in data coords', style='italic',
        bbox={'facecolor': 'red', 'alpha': 0.5, 'pad': 10})

ax.text(2, 6, r'an equation: $E=mc^2$', fontsize=15)

ax.text(3, 2, 'Unicode: Institut für Festkörperphysik')

ax.text(0.95, 0.01, 'colored text in axes coords',
        verticalalignment='bottom', horizontalalignment='right',
        transform=ax.transAxes,
        color='green', fontsize=15)

ax.plot([2], [1], 'o')
ax.annotate('annotate', xy=(2, 1), xytext=(3, 4),
            arrowprops=dict(facecolor='black', shrink=0.05))

plt.show()
```

Labels for x- and y-axis

Specifying the labels for the x- and y-axis is straightforward, via the `set_xlabel` and `set_ylabel` methods.

```
import matplotlib.pyplot as plt
import numpy as np

x1 = np.linspace(0.0, 5.0, 100)
y1 = np.cos(2 * np.pi * x1) * np.exp(-x1)

fig, ax = plt.subplots(figsize=(5, 3))
fig.subplots_adjust(bottom=0.15, left=0.2)
ax.plot(x1, y1)
ax.set_xlabel('Time (s)')
ax.set_ylabel('Damped oscillation (V)')

plt.show()
```

The x- and y-labels are automatically placed so that they clear the x- and y-ticklabels. Compare the plot below with that above, and note the y-label is to the left of the one above.

```
fig, ax = plt.subplots(figsize=(5, 3))
fig.subplots_adjust(bottom=0.15, left=0.2)
ax.plot(x1, y1*10000)
ax.set_xlabel('Time (s)')
ax.set_ylabel('Damped oscillation (V)')

plt.show()
```

If you want to move the labels, you can specify the *labelpad* keyword argument, where the value is points (1/72”, the same unit used to specify font sizes).

```
fig, ax = plt.subplots(figsize=(5, 3))
fig.subplots_adjust(bottom=0.15, left=0.2)
```

```
ax.plot(x1, y1*10000)
ax.set_xlabel('Time (s)')
ax.set_ylabel('Damped oscillation (V)', labelpad=18)

plt.show()
```

Alternatively, the labels accept all the `Text` keyword arguments, including *position*, via which we can manually specify the label positions. Here we put the xlabel to the far left of the axis. Note, that the y-coordinate of this position has no effect - to adjust the y-position we need to use the *labelpad* keyword argument.

```
fig, ax = plt.subplots(figsize=(5, 3))
fig.subplots_adjust(bottom=0.15, left=0.2)
ax.plot(x1, y1)
ax.set_xlabel('Time (s)', position=(0., 1e6), horizontalalignment='left')
ax.set_ylabel('Damped oscillation (V)')

plt.show()
```

All the labelling in this tutorial can be changed by manipulating the `matplotlib.font_manager.FontProperties` method, or by named keyword arguments to `set_xlabel`.

```
from matplotlib.font_manager import FontProperties

font = FontProperties(family='Times New Roman', style='italic')

fig, ax = plt.subplots(figsize=(5, 3))
fig.subplots_adjust(bottom=0.15, left=0.2)
ax.plot(x1, y1)
ax.set_xlabel('Time (s)', fontsize='large', fontweight='bold')
ax.set_ylabel('Damped oscillation (V)', fontproperties=font)

plt.show()
```

Finally, we can use native TeX rendering in all text objects and have multiple lines:

```
fig, ax = plt.subplots(figsize=(5, 3))
fig.subplots_adjust(bottom=0.2, left=0.2)
ax.plot(x1, np.cumsum(y1**2))
ax.set_xlabel('Time (s) \n This was a long experiment')
ax.set_ylabel(r'$\int\ Y^2\ dt\ \ (V^2 s)$')
plt.show()
```

Titles

Subplot titles are set in much the same way as labels, but there is the *loc* keyword argument that can change the position and justification (the default value is ”center”).

```
fig, axs = plt.subplots(3, 1, figsize=(5, 6), tight_layout=True)
locs = ['center', 'left', 'right']
for ax, loc in zip(axs, locs):
    ax.plot(x1, y1)
    ax.set_title('Title with loc at ' + loc, loc=loc)
plt.show()
```

Vertical spacing for titles is controlled via `axes.titlepad`. Setting to a different value moves the title.

```
fig, ax = plt.subplots(figsize=(5, 3))
fig.subplots_adjust(top=0.8)
ax.plot(x1, y1)
ax.set_title('Vertically offset title', pad=30)
plt.show()
```

Ticks and ticklabels

Placing ticks and ticklabels is a very tricky aspect of making a figure. Matplotlib does its best to accomplish the task automatically, but it also offers a very flexible framework for determining the choices for tick locations, and how they are labelled.

Terminology

Axes have a `matplotlib.axis.Axis` object for the `ax.xaxis` and `ax.yaxis` that contain the information about how the labels in the axis are laid out.

The axis API is explained in detail in the documentation to `axis`.

An Axis object has major and minor ticks. The Axis has `Axis.set_major_locator` and `Axis.set_minor_locator` methods that use the data being plotted to determine the location of major and minor ticks. There are also `Axis.set_major_formatter` and `Axis.set_minor_formatter` methods that format the tick labels.

Simple ticks

It is often convenient to simply define the tick values, and sometimes the tick labels, overriding the default locators and formatters. However, this is discouraged because it breaks interactive navigation of the plot. It also can reset the axis limits: note that the second plot has the ticks we asked for, including ones that are well outside the automatic view limits.

```
fig, axs = plt.subplots(2, 1, figsize=(5, 3), tight_layout=True)
axs[0].plot(x1, y1)
axs[1].plot(x1, y1)
axs[1].xaxis.set_ticks(np.arange(0., 8.1, 2.))
plt.show()
```

We can of course fix this after the fact, but it does highlight a weakness of hard-coding the ticks. This example also changes the format of the ticks:

```
fig, axs = plt.subplots(2, 1, figsize=(5, 3), tight_layout=True)
axs[0].plot(x1, y1)
axs[1].plot(x1, y1)
ticks = np.arange(0., 8.1, 2.)
# list comprehension to get all tick labels...
tickla = [f'{tick:1.2f}' for tick in ticks]
axs[1].xaxis.set_ticks(ticks)
axs[1].xaxis.set_ticklabels(tickla)
axs[1].set_xlim(axs[0].get_xlim())
plt.show()
```

Tick locators and formatters

Instead of making a list of all the ticklabels, we could have used `matplotlib.ticker.StrMethodFormatter` (new-style `str.format()` format string) or `matplotlib.ticker.FormatStrFormatter` (old-style '%' format string) and passed it to the `ax.xaxis`.

A `matplotlib.ticker.StrMethodFormatter` can also be created by passing a `str` without having to explicitly create the formatter.

```
fig, axs = plt.subplots(2, 1, figsize=(5, 3), tight_layout=True)
axs[0].plot(x1, y1)
axs[1].plot(x1, y1)
ticks = np.arange(0., 8.1, 2.)
axs[1].xaxis.set_ticks(ticks)
axs[1].xaxis.set_major_formatter('{x:1.1f}')
axs[1].set_xlim(axs[0].get_xlim())
plt.show()
```

And of course we could have used a non-default locator to set the tick locations. Note we still pass in the tick values, but the x-limit fix used above is *not* needed.

```
fig, axs = plt.subplots(2, 1, figsize=(5, 3), tight_layout=True)
axs[0].plot(x1, y1)
axs[1].plot(x1, y1)
locator = matplotlib.ticker.FixedLocator(ticks)
axs[1].xaxis.set_major_locator(locator)
axs[1].xaxis.set_major_formatter('±{x}°')
plt.show()
```

The default formatter is the `matplotlib.ticker.MaxNLocator` called as `ticker.MaxNLocator(self, nbins='auto', steps=[1, 2, 2.5, 5, 10])`. The `steps` argument contains a list of multiples that can be used for tick values. In this case, 2, 4, 6 would be acceptable ticks, as would 20, 40, 60 or 0.2, 0.4, 0.6. However, 3, 6, 9 would not be acceptable because 3 doesn't appear in the list of steps.

Setting `nbins=auto` uses an algorithm to determine how many ticks will be acceptable based on the axis length. The fontsize of the ticklabel is taken into account, but the length of the tick string is not (because it's not yet known.) In the bottom row, the ticklabels are quite large, so we set `nbins=4` to make the labels fit in the right-hand plot.

```
fig, axs = plt.subplots(2, 2, figsize=(8, 5), tight_layout=True)
for n, ax in enumerate(axs.flat):
    ax.plot(x1*10., y1)

formatter = matplotlib.ticker.FormatStrFormatter('%1.1f')
locator = matplotlib.ticker.MaxNLocator(nbins='auto', steps=[1, 4, 10])
axs[0, 1].xaxis.set_major_locator(locator)
axs[0, 1].xaxis.set_major_formatter(formatter)

formatter = matplotlib.ticker.FormatStrFormatter('%1.5f')
locator = matplotlib.ticker.AutoLocator()
axs[1, 0].xaxis.set_major_formatter(formatter)
axs[1, 0].xaxis.set_major_locator(locator)

formatter = matplotlib.ticker.FormatStrFormatter('%1.5f')
locator = matplotlib.ticker.MaxNLocator(nbins=4)
axs[1, 1].xaxis.set_major_formatter(formatter)
axs[1, 1].xaxis.set_major_locator(locator)

plt.show()


Finally, we can specify functions for the formatter using
```

`matplotlib.ticker.FuncFormatter`. Further, like `matplotlib.ticker.StrMethodFormatter`, passing a function will automatically create a `matplotlib.ticker.FuncFormatter`.

```
def formatoddticks(x, pos):
    """Format odd tick positions."""
    if x % 2:
        return f'{x:1.2f}'
    else:
        return ''


fig, ax = plt.subplots(figsize=(5, 3), tight_layout=True)
ax.plot(x1, y1)
locator = matplotlib.ticker.MaxNLocator(nbins=6)
ax.xaxis.set_major_formatter(formatoddticks)
ax.xaxis.set_major_locator(locator)

plt.show()
```

Dateticks

Matplotlib can accept `datetime.datetime` and `numpy.datetime64` objects as plotting arguments. Dates and times require special formatting, which can often benefit from manual intervention. In order to help, dates have special locators and formatters, defined in the `matplotlib.dates` module.

The following simple example illustrates this concept. Note how we rotate the tick labels so that they don't overlap.

```
import datetime

fig, ax = plt.subplots(figsize=(5, 3), tight_layout=True)
base = datetime.datetime(2017, 1, 1, 0, 0, 1)
time = [base + datetime.timedelta(days=x) for x in range(len(x1))]

ax.plot(time, y1)
ax.tick_params(axis='x', rotation=70)
plt.show()
```

We can pass a format to `matplotlib.dates.DateFormatter`. If two tick labels are very close together, we can use the `dates.DayLocator` class, which allows us to specify a list of days of the month to use. Similar formatters are listed in the `matplotlib.dates` module.

```
import matplotlib.dates as mdates

locator = mdates.DayLocator(bymonthday=[1, 15])
formatter = mdates.DateFormatter('%b %d')

fig, ax = plt.subplots(figsize=(5, 3), tight_layout=True)
ax.xaxis.set_major_locator(locator)
ax.xaxis.set_major_formatter(formatter)
ax.plot(time, y1)
ax.tick_params(axis='x', rotation=70)
plt.show()
```

Legends and annotations

- legend_guide
- annotations

Annotations

Source: https://matplotlib.org/stable/users/explain/text/annotations.html

Annotations are graphical elements, often pieces of text, that explain, add context to, or otherwise highlight some portion of the visualized data. `annotate` supports a number of coordinate systems for flexibly positioning data and annotations relative to each other and a variety of options of for styling the text. Axes.annotate also provides an optional arrow from the text to the data and this arrow can be styled in various ways. `text` can also be used for simple text annotation, but does not provide as much flexibility in positioning and styling as `annotate`.

```
import matplotlib.pyplot as plt
import numpy as np

fig, ax = plt.subplots(figsize=(3, 3))

t = np.arange(0.0, 5.0, 0.01)
s = np.cos(2*np.pi*t)
line, = ax.plot(t, s, lw=2)

ax.annotate('local max', xy=(2, 1), xytext=(3, 1.5),
            arrowprops=dict(facecolor='black', shrink=0.05))
ax.set_ylim(-2, 2)
```

In this example, both the *xy* (arrow tip) and *xytext* locations (text location) are in data coordinates. There are a variety of other coordinate systems one can choose -- you can specify the coordinate system of *xy* and *xytext* with one of the following strings for *xycoords* and *textcoords* (default is 'data')

argument	coordinate system
'figure points'	points from the lower left corner of the figure
'figure pixels'	pixels from the lower left corner of the figure
'figure fraction'	(0, 0) is lower left of figure and (1, 1) is upper right
'axes points'	points from lower left corner of the Axes
'axes pixels'	pixels from lower left corner of the Axes
'axes fraction'	(0, 0) is lower left of Axes and (1, 1) is upper right
'data'	use the axes data coordinate system

The following strings are also valid arguments for *textcoords*

argument	coordinate system
'offset points'	offset (in points) from the xy value
'offset pixels'	offset (in pixels) from the xy value

For physical coordinate systems (points or pixels) the origin is the bottom-left of the figure or Axes. Points are typographic points meaning that they are a physical unit measuring 1/72 of an inch. Points and pixels are discussed in further detail in transforms-fig-scale-dpi.

Annotating data

This example places the text coordinates in fractional axes coordinates:

```
fig, ax = plt.subplots(figsize=(3, 3))

t = np.arange(0.0, 5.0, 0.01)
s = np.cos(2*np.pi*t)
line, = ax.plot(t, s, lw=2)

ax.annotate('local max', xy=(2, 1), xycoords='data',
            xytext=(0.01, .99), textcoords='axes fraction',
            va='top', ha='left',
            arrowprops=dict(facecolor='black', shrink=0.05))
ax.set_ylim(-2, 2)
```

Annotating an Artist

Annotations can be positioned relative to an `Artist` instance by passing that Artist in as *xycoords*. Then *xy* is interpreted as a fraction of the Artist's bounding box.

```
import matplotlib.patches as mpatches

fig, ax = plt.subplots(figsize=(3, 3))
arr = mpatches.FancyArrowPatch((1.25, 1.5), (1.75, 1.5),
                               arrowstyle='->,head_width=.15', mutation_scale=20)
ax.add_patch(arr)
ax.annotate("label", (.5, .5), xycoords=arr, ha='center', va='bottom')
ax.set(xlim=(1, 2), ylim=(1, 2))
```

Here the annotation is placed at position (.5,.5) relative to the arrow's lower left corner and is vertically and horizontally at that position. Vertically, the bottom aligns to that reference point so that the label is above the line. For an example of chaining annotation Artists, see the Artist section of annotating_coordinate_systems.

Annotating with arrows

You can enable drawing of an arrow from the text to the annotated point by giving a dictionary of arrow properties in the optional keyword argument *arrowprops*.

arrowprops key	description
width	the width of the arrow in points
frac	the fraction of the arrow length occupied by the head
headwidth	the width of the base of the arrow head in points
shrink	move the tip and base some percent away from the annotated point and text
**kwargs	any key for `matplotlib.patches.Polygon`, e.g., `facecolor`

In the example below, the *xy* point is in the data coordinate system since *xycoords* defaults to 'data'. For a polar Axes, this is in (theta, radius) space. The text in this example is placed in the fractional figure coordinate system. `matplotlib.text.Text` keyword arguments like *horizontalalignment*, *verticalalignment* and *fontsize* are passed from `annotate` to the `Text` instance.

```
fig = plt.figure()
ax = fig.add_subplot(projection='polar')
r = np.arange(0, 1, 0.001)
theta = 2 * 2*np.pi * r
line, = ax.plot(theta, r, color='#ee8d18', lw=3)

ind = 800
thisr, thistheta = r[ind], theta[ind]
ax.plot([thistheta], [thisr], 'o')
ax.annotate('a polar annotation',
            xy=(thistheta, thisr),  # theta, radius
            xytext=(0.05, 0.05),    # fraction, fraction
            textcoords='figure fraction',
            arrowprops=dict(facecolor='black', shrink=0.05),
            horizontalalignment='left',
            verticalalignment='bottom')
```

For more on plotting with arrows, see annotation_with_custom_arrow

Placing text annotations relative to data

Annotations can be positioned at a relative offset to the *xy* input to annotation by setting the *textcoords* keyword argument to `'offset points'` or `'offset pixels'`.

```
fig, ax = plt.subplots(figsize=(3, 3))
x = [1, 3, 5, 7, 9]
y = [2, 4, 6, 8, 10]
annotations = ["A", "B", "C", "D", "E"]
ax.scatter(x, y, s=20)

for xi, yi, text in zip(x, y, annotations):
    ax.annotate(text,
                xy=(xi, yi), xycoords='data',
                xytext=(1.5, 1.5), textcoords='offset points')
```

The annotations are offset 1.5 points (1.5*1/72 inches) from the *xy* values.

Advanced annotation

We recommend reading annotations-tutorial, `text` and `annotate` before reading this section.

Annotating with boxed text

`text` takes a *bbox* keyword argument, which draws a box around the text:

```
fig, ax = plt.subplots(figsize=(5, 5))
t = ax.text(0.5, 0.5, "Direction",
            ha="center", va="center", rotation=45, size=15,
            bbox=dict(boxstyle="rarrow,pad=0.3",
                      fc="lightblue", ec="steelblue", lw=2))
```

The arguments are the name of the box style with its attributes as keyword arguments. Currently, following box styles are implemented:

Class	Name	Attrs
Circle	`circle`	pad=0.3
DArrow	`darrow`	pad=0.3,head_width=1.5,head_angle=90
Ellipse	`ellipse`	pad=0.3
LArrow	`larrow`	pad=0.3,head_width=1.5,head_angle=90
RArrow	`rarrow`	pad=0.3,head_width=1.5,head_angle=90
Round	`round`	pad=0.3,rounding_size=None
Round4	`round4`	pad=0.3,rounding_size=None
Roundtooth	`roundtooth`	pad=0.3,tooth_size=None
Sawtooth	`sawtooth`	pad=0.3,tooth_size=None
Square	`square`	pad=0.3

The patch object (box) associated with the text can be accessed using:

```
bb = t.get_bbox_patch()
```

The return value is a `FancyBboxPatch`; patch properties (facecolor, edgewidth, etc.) can be accessed and modified as usual. `FancyBboxPatch.set_boxstyle` sets the box shape:

```
bb.set_boxstyle("rarrow", pad=0.6)
```

The attribute arguments can also be specified within the style name with separating comma:

```
bb.set_boxstyle("rarrow, pad=0.6")
```

Defining custom box styles

Custom box styles can be implemented as a function that takes arguments specifying both a rectangular box and the amount of ”mutation”, and returns the ”mutated” path. The specific signature is the one of `custom_box_style` below.

Here, we return a new path which adds an ”arrow” shape on the left of the box.

The custom box style can then be used by passing `bbox=dict(boxstyle=custom_box_style, ...)` to `Axes.text`.

```
from matplotlib.path import Path


def custom_box_style(x0, y0, width, height, mutation_size):
    """
    Given the location and size of the box, return the path of the box around it.

    Rotation is automatically taken care of.

    Parameters
    ----------
    x0, y0, width, height : float
       Box location and size.
    mutation_size : float
        Mutation reference scale, typically the text font size.
    """
    # padding
    mypad = 0.3
    pad = mutation_size * mypad
    # width and height with padding added.
    width = width + 2 * pad
    height = height + 2 * pad
```

```
    # boundary of the padded box
    x0, y0 = x0 - pad, y0 - pad
    x1, y1 = x0 + width, y0 + height
    # return the new path
    return Path([(x0, y0), (x1, y0), (x1, y1), (x0, y1),
                 (x0-pad, (y0+y1)/2), (x0, y0), (x0, y0)],
                closed=True)

fig, ax = plt.subplots(figsize=(3, 3))
ax.text(0.5, 0.5, "Test", size=30, va="center", ha="center", rotation=30,
        bbox=dict(boxstyle=custom_box_style, alpha=0.2))
```

Likewise, custom box styles can be implemented as classes that implement `__call__`.

The classes can then be registered into the `BoxStyle._style_list` dict, which allows specifying the box style as a string, `bbox=dict(boxstyle="registered_name,param=value,...", ...)`. Note that this registration relies on internal APIs and is therefore not officially supported.

```
from matplotlib.patches import BoxStyle


class MyStyle:
    """A simple box."""

    def __init__(self, pad=0.3):
        """
        The arguments must be floats and have default values.

        Parameters
        ----------
        pad : float
            amount of padding
        """
        self.pad = pad
        super().__init__()

    def __call__(self, x0, y0, width, height, mutation_size):
        """
        Given the location and size of the box, return the path of the box around it.

        Rotation is automatically taken care of.

        Parameters
        ----------
        x0, y0, width, height : float
            Box location and size.
        mutation_size : float
            Reference scale for the mutation, typically the text font size.
        """
        # padding
        pad = mutation_size * self.pad
        # width and height with padding added
        width = width + 2 * pad
        height = height + 2 * pad
        # boundary of the padded box
        x0, y0 = x0 - pad, y0 - pad
        x1, y1 = x0 + width, y0 + height
        # return the new path
        return Path([(x0, y0), (x1, y0), (x1, y1), (x0, y1),
                     (x0-pad, (y0+y1)/2), (x0, y0), (x0, y0)],
```

```
                    closed=True)


BoxStyle._style_list["angled"] = MyStyle  # Register the custom style.

fig, ax = plt.subplots(figsize=(3, 3))
ax.text(0.5, 0.5, "Test", size=30, va="center", ha="center", rotation=30,
        bbox=dict(boxstyle="angled,pad=0.5", alpha=0.2))

del BoxStyle._style_list["angled"]  # Unregister it.
```

Similarly, you can define a custom `ConnectionStyle` and a custom `ArrowStyle`. View the source code at `patches` to learn how each class is defined.

Customizing annotation arrows

An arrow connecting *xy* to *xytext* can be optionally drawn by specifying the *arrowprops* argument. To draw only an arrow, use empty string as the first argument:

```
fig, ax = plt.subplots(figsize=(3, 3))
ax.annotate("",
            xy=(0.2, 0.2), xycoords='data',
            xytext=(0.8, 0.8), textcoords='data',
            arrowprops=dict(arrowstyle="->", connectionstyle="arc3"))
```

The arrow is drawn as follows:

1. A path connecting the two points is created, as specified by the *connectionstyle* parameter.
2. The path is clipped to avoid patches *patchA* and *patchB*, if these are set.
3. The path is further shrunk by *shrinkA* and *shrinkB* (in pixels).
4. The path is transmuted to an arrow patch, as specified by the *arrowstyle* parameter.

```
import matplotlib.patches as mpatches

x1, y1 = 0.3, 0.3
x2, y2 = 0.7, 0.7
arrowprops = {
    "1. connect with connectionstyle":
        dict(arrowstyle="-", patchB=False, shrinkB=0),
    "2. clip against patchB": dict(arrowstyle="-", patchB=True, shrinkB=0),
    "3. shrink by shrinkB": dict(arrowstyle="-", patchB=True, shrinkB=5),
    "4. mutate with arrowstyle": dict(arrowstyle="fancy", patchB=True, shrinkB=5),
}

fig, axs = plt.subplots(2, 2, figsize=(6, 6), layout='compressed')
for ax, (name, props) in zip(axs.flat, arrowprops.items()):
    ax.plot([x1, x2], [y1, y2], ".")

    el = mpatches.Ellipse((x1, y1), 0.3, 0.4, angle=30, alpha=0.2)
    ax.add_artist(el)

    props["patchB"] = el if props["patchB"] else None

    ax.annotate(
        "",
        xy=(x1, y1), xycoords='data',
        xytext=(x2, y2), textcoords='data',
        arrowprops={"color": "0.5", "connectionstyle": "arc3,rad=0.3", **props})
```

```
    ax.text(.05, .95, name, transform=ax.transAxes, ha="left", va="top")

    ax.set(xlim=(0, 1), ylim=(0, 1), xticks=[], yticks=[], aspect=1)

fig.get_layout_engine().set(wspace=0, hspace=0, w_pad=0, h_pad=0)
```

The creation of the connecting path between two points is controlled by `connectionstyle` key and the following styles are available:

Name	Attrs
`angle`	angleA=90,angleB=0,rad=0.0
`angle3`	angleA=90,angleB=0
`arc`	angleA=0,angleB=0,armA=None,armB=None,rad=0.0
`arc3`	rad=0.0
`bar`	armA=0.0,armB=0.0,fraction=0.3,angle=None

Note that "3" in `angle3` and `arc3` is meant to indicate that the resulting path is a quadratic spline segment (three control points). As will be discussed below, some arrow style options can only be used when the connecting path is a quadratic spline.

The behavior of each connection style is (limitedly) demonstrated in the example below. (Warning: The behavior of the `bar` style is currently not well-defined and may be changed in the future).

```
def demo_con_style(ax, connectionstyle):
    x1, y1 = 0.3, 0.2
    x2, y2 = 0.8, 0.6

    ax.plot([x1, x2], [y1, y2], ".")
    ax.annotate("",
                xy=(x1, y1), xycoords='data',
                xytext=(x2, y2), textcoords='data',
                arrowprops=dict(arrowstyle="->", color="0.5",
                                shrinkA=5, shrinkB=5,
                                patchA=None, patchB=None,
                                connectionstyle=connectionstyle,
                                ),
                )

    ax.text(.05, .95, connectionstyle.replace(",", ",\n"),
            transform=ax.transAxes, ha="left", va="top")

    ax.set(xlim=(0, 1), ylim=(0, 1.25), xticks=[], yticks=[], aspect=1.25)

fig, axs = plt.subplots(3, 5, figsize=(7, 6.3), layout="compressed")
demo_con_style(axs[0, 0], "angle3,angleA=90,angleB=0")
demo_con_style(axs[1, 0], "angle3,angleA=0,angleB=90")
demo_con_style(axs[0, 1], "arc3,rad=0.")
demo_con_style(axs[1, 1], "arc3,rad=0.3")
demo_con_style(axs[2, 1], "arc3,rad=-0.3")
demo_con_style(axs[0, 2], "angle,angleA=-90,angleB=180,rad=0")
demo_con_style(axs[1, 2], "angle,angleA=-90,angleB=180,rad=5")
demo_con_style(axs[2, 2], "angle,angleA=-90,angleB=10,rad=5")
demo_con_style(axs[0, 3], "arc,angleA=-90,angleB=0,armA=30,armB=30,rad=0")
demo_con_style(axs[1, 3], "arc,angleA=-90,angleB=0,armA=30,armB=30,rad=5")
demo_con_style(axs[2, 3], "arc,angleA=-90,angleB=0,armA=0,armB=40,rad=0")
demo_con_style(axs[0, 4], "bar,fraction=0.3")
demo_con_style(axs[1, 4], "bar,fraction=-0.3")
```

```
demo_con_style(axs[2, 4], "bar,angle=180,fraction=-0.2")

axs[2, 0].remove()
fig.get_layout_engine().set(wspace=0, hspace=0, w_pad=0, h_pad=0)
```

The connecting path (after clipping and shrinking) is then mutated to an arrow patch, according to the given `arrowstyle`:

Name	Attrs
`-`	None
`->`	head_length=0.4,head_width=0.2
`-[`	widthB=1.0,lengthB=0.2,angleB=None
`|-|`	widthA=1.0,widthB=1.0
`-|>`	head_length=0.4,head_width=0.2
`<-`	head_length=0.4,head_width=0.2
`<->`	head_length=0.4,head_width=0.2
`<|-`	head_length=0.4,head_width=0.2
`<|-|>`	head_length=0.4,head_width=0.2
`fancy`	head_length=0.4,head_width=0.4,tail_width=0.4
`simple`	head_length=0.5,head_width=0.5,tail_width=0.2
`wedge`	tail_width=0.3,shrink_factor=0.5

Some arrowstyles only work with connection styles that generate a quadratic-spline segment. They are `fancy`, `simple`, and `wedge`. For these arrow styles, you must use the "angle3" or "arc3" connection style.

If the annotation string is given, the patch is set to the bbox patch of the text by default.

```
fig, ax = plt.subplots(figsize=(3, 3))

ax.annotate("Test",
            xy=(0.2, 0.2), xycoords='data',
            xytext=(0.8, 0.8), textcoords='data',
            size=20, va="center", ha="center",
            arrowprops=dict(arrowstyle="simple",
                            connectionstyle="arc3,rad=-0.2"))
```

As with `text`, a box around the text can be drawn using the *bbox* argument.

```
fig, ax = plt.subplots(figsize=(3, 3))

ann = ax.annotate("Test",
                  xy=(0.2, 0.2), xycoords='data',
                  xytext=(0.8, 0.8), textcoords='data',
                  size=20, va="center", ha="center",
                  bbox=dict(boxstyle="round4", fc="w"),
                  arrowprops=dict(arrowstyle="-|>",
                                  connectionstyle="arc3,rad=-0.2",
                                  fc="w"))
```

By default, the starting point is set to the center of the text extent. This can be adjusted with `relpos` key value. The values are normalized to the extent of the text. For example, (0, 0) means lower-left corner and (1, 1) means top-right.

```
fig, ax = plt.subplots(figsize=(3, 3))

ann = ax.annotate("Test",
```

```
                  xy=(0.2, 0.2), xycoords='data',
                  xytext=(0.8, 0.8), textcoords='data',
                  size=20, va="center", ha="center",
                  bbox=dict(boxstyle="round4", fc="w"),
                  arrowprops=dict(arrowstyle="-|>",
                                  connectionstyle="arc3,rad=0.2",
                                  relpos=(0., 0.),
                                  fc="w"))

ann = ax.annotate("Test",
                  xy=(0.2, 0.2), xycoords='data',
                  xytext=(0.8, 0.8), textcoords='data',
                  size=20, va="center", ha="center",
                  bbox=dict(boxstyle="round4", fc="w"),
                  arrowprops=dict(arrowstyle="-|>",
                                  connectionstyle="arc3,rad=-0.2",
                                  relpos=(1., 0.),
                                  fc="w"))
```

Placing Artist at anchored Axes locations

There are classes of artists that can be placed at an anchored location in the Axes. A common example is the legend. This type of artist can be created by using the `OffsetBox` class. A few predefined classes are available in `matplotlib.offsetbox` and in `mpl_toolkits.axes_grid1.anchored_artists`.

```
from matplotlib.offsetbox import AnchoredText

fig, ax = plt.subplots(figsize=(3, 3))
at = AnchoredText("Figure 1a",
                  prop=dict(size=15), frameon=True, loc='upper left')
at.patch.set_boxstyle("round,pad=0.,rounding_size=0.2")
ax.add_artist(at)
```

The *loc* keyword has same meaning as in the legend command.

A simple application is when the size of the artist (or collection of artists) is known in pixel size during the time of creation. For example, If you want to draw a circle with fixed size of 20 pixel x 20 pixel (radius = 10 pixel), you can utilize `AnchoredDrawingArea`. The instance is created with a size of the drawing area (in pixels), and arbitrary artists can be added to the drawing area. Note that the extents of the artists that are added to the drawing area are not related to the placement of the drawing area itself. Only the initial size matters.

The artists that are added to the drawing area should not have a transform set (it will be overridden) and the dimensions of those artists are interpreted as a pixel coordinate, i.e., the radius of the circles in above example are 10 pixels and 5 pixels, respectively.

```
from matplotlib.patches import Circle
from mpl_toolkits.axes_grid1.anchored_artists import AnchoredDrawingArea

fig, ax = plt.subplots(figsize=(3, 3))
ada = AnchoredDrawingArea(40, 20, 0, 0,
                          loc='upper right', pad=0., frameon=False)
p1 = Circle((10, 10), 10)
ada.drawing_area.add_artist(p1)
p2 = Circle((30, 10), 5, fc="r")
ada.drawing_area.add_artist(p2)
ax.add_artist(ada)
```

Sometimes, you want your artists to scale with the data coordinate (or coordinates other than canvas pixels). You can use `AnchoredAuxTransformBox` class. This is similar to `AnchoredDrawingArea` except that the extent of the artist is determined during the drawing time respecting the specified transform.

The ellipse in the example below will have width and height corresponding to 0.1 and 0.4 in data coordinates and will be automatically scaled when the view limits of the Axes change.

```
from matplotlib.patches import Ellipse
from mpl_toolkits.axes_grid1.anchored_artists import AnchoredAuxTransformBox

fig, ax = plt.subplots(figsize=(3, 3))
box = AnchoredAuxTransformBox(ax.transData, loc='upper left')
el = Ellipse((0, 0), width=0.1, height=0.4, angle=30)  # in data coordinates!
box.drawing_area.add_artist(el)
ax.add_artist(box)
```

Another method of anchoring an artist relative to a parent Axes or anchor point is via the *bbox_to_anchor* argument of `AnchoredOffsetbox`. This artist can then be automatically positioned relative to another artist using `HPacker` and `VPacker`:

```
from matplotlib.offsetbox import (AnchoredOffsetbox, DrawingArea, HPacker,
                                  TextArea)

fig, ax = plt.subplots(figsize=(3, 3))

box1 = TextArea(" Test: ", textprops=dict(color="k"))
box2 = DrawingArea(60, 20, 0, 0)

el1 = Ellipse((10, 10), width=16, height=5, angle=30, fc="r")
el2 = Ellipse((30, 10), width=16, height=5, angle=170, fc="g")
el3 = Ellipse((50, 10), width=16, height=5, angle=230, fc="b")
box2.add_artist(el1)
box2.add_artist(el2)
box2.add_artist(el3)

box = HPacker(children=[box1, box2],
              align="center",
              pad=0, sep=5)

anchored_box = AnchoredOffsetbox(loc='lower left',
                                 child=box, pad=0.,
                                 frameon=True,
                                 bbox_to_anchor=(0., 1.02),
                                 bbox_transform=ax.transAxes,
                                 borderpad=0.,)

ax.add_artist(anchored_box)
fig.subplots_adjust(top=0.8)
```

Note that, unlike in `Legend`, the `bbox_transform` is set to `IdentityTransform` by default

Using an Artist as an annotation

`AnnotationBbox` uses artists in `OffsetBox` container artists as the annotations and supports positioning these annotations using the same coordinate systems as the other annotation methods. For more examples, see /gallery/text_labels_and_annotations/demo_annotation_box

```
from matplotlib.offsetbox import AnnotationBbox, DrawingArea, OffsetImage
from matplotlib.patches import Annulus

fig, ax = plt.subplots()

text = ax.text(.2, .8, "Green!", color='green')

da = DrawingArea(20, 20)
annulus = Annulus((10, 10), 10, 5, color='tab:green')
da.add_artist(annulus)

# position annulus relative to text
ab1 = AnnotationBbox(da, xy=(.5, 0),
                     xybox=(.5, .25),
                     xycoords=text,
                     boxcoords=(text, "data"),
                     arrowprops=dict(arrowstyle="->"),
                     bboxprops=dict(alpha=0.5))
ax.add_artist(ab1)

N = 25
arr = np.repeat(np.linspace(0, 1, N), N).reshape(N, N)
im = OffsetImage(arr, cmap='Greens')
im.image.axes = ax

# position gradient relative to text and annulus
ab2 = AnnotationBbox(im, xy=(.5, 0),
                     xybox=(.75, 0),
                     xycoords=text,
                     boxcoords=('data', annulus),
                     arrowprops=dict(arrowstyle="->"),
                     bboxprops=dict(alpha=0.5))
ax.add_artist(ab2)
```

Coordinate systems for annotations

Matplotlib Annotations support several types of coordinate systems. The examples in annotations-tutorial used the `data` coordinate system; Some others more advanced options are:

`Transform` instance ^^^^^^^^^^^^^^^^^^^^^^^^

Transforms map coordinates into different coordinate systems, usually the display coordinate system. See transforms_tutorial for a detailed explanation. Here Transform objects are used to identify the coordinate system of the corresponding points. For example, the `Axes.transAxes` transform positions the annotation relative to the Axes coordinates; therefore using it is identical to setting the coordinate system to ”axes fraction”:

```
fig, (ax1, ax2) = plt.subplots(nrows=1, ncols=2, figsize=(6, 3))
ax1.annotate("Test", xy=(0.2, 0.2), xycoords=ax1.transAxes)
ax2.annotate("Test", xy=(0.2, 0.2), xycoords="axes fraction")
```

Another commonly used `Transform` instance is `Axes.transData`. This transform is the coordinate system of the data plotted in the Axes. In this example, it is used to draw an arrow between related data points in two Axes. We have passed an empty text because in this case, the annotation connects data points.

```
x = np.linspace(-1, 1)

fig, (ax1, ax2) = plt.subplots(nrows=1, ncols=2, figsize=(6, 3))
```

```
ax1.plot(x, -x**3)
ax2.plot(x, -3*x**2)
ax2.annotate("",
             xy=(0, 0), xycoords=ax1.transData,
             xytext=(0, 0), textcoords=ax2.transData,
             arrowprops=dict(arrowstyle="<->"))
```

`Artist` instance ^^^^^^^^^^^^^^^^^^^

The *xy* value (or *xytext*) is interpreted as a fractional coordinate of the bounding box (bbox) of the artist:

```
fig, ax = plt.subplots(nrows=1, ncols=1, figsize=(3, 3))
an1 = ax.annotate("Test 1",
                  xy=(0.5, 0.5), xycoords="data",
                  va="center", ha="center",
                  bbox=dict(boxstyle="round", fc="w"))

an2 = ax.annotate("Test 2",
                  xy=(1, 0.5), xycoords=an1,  # (1, 0.5) of an1's bbox
                  xytext=(30, 0), textcoords="offset points",
                  va="center", ha="left",
                  bbox=dict(boxstyle="round", fc="w"),
                  arrowprops=dict(arrowstyle="->"))
```

Note that you must ensure that the extent of the coordinate artist (*an1* in this example) is determined before *an2* gets drawn. Usually, this means that *an2* needs to be drawn after *an1*. The base class for all bounding boxes is `BboxBase`

Callable that returns `Transform` of `BboxBase` ^^^^^^^^^^^^^^^^^^^

Blended coordinate specification

A blended pair of coordinate specifications -- the first for the x-coordinate, and the second is for the y-coordinate. For example, x=0.5 is in data coordinates, and y=1 is in normalized axes coordinates:

```
fig, ax = plt.subplots(figsize=(3, 3))
ax.annotate("Test", xy=(0.5, 1), xycoords=("data", "axes fraction"))
ax.axvline(x=.5, color='lightgray')
ax.set(xlim=(0, 2), ylim=(1, 2))
```

Any of the supported coordinate systems can be used in a blended specification. For example, the text ”Anchored to 1 & 2” is positioned relative to the two `Text` Artists:

```
fig, ax = plt.subplots(figsize=(3, 3))

t1 = ax.text(0.05, .05, "Text 1", va='bottom', ha='left')
t2 = ax.text(0.90, .90, "Text 2", ha='right')
t3 = ax.annotate("Anchored to 1 & 2", xy=(0, 0), xycoords=(t1, t2),
                 va='bottom', color='tab:orange',)
```

`text.OffsetFrom` ^^^^^^^^^^^^^^^^^^^^

Sometimes, you want your annotation with some ”offset points”, not from the annotated point but from some other point or artist. `text.OffsetFrom` is a helper for such cases.

```
from matplotlib.text import OffsetFrom

fig, ax = plt.subplots(figsize=(3, 3))
an1 = ax.annotate("Test 1", xy=(0.5, 0.5), xycoords="data",
                  va="center", ha="center",
                  bbox=dict(boxstyle="round", fc="w"))

offset_from = OffsetFrom(an1, (0.5, 0))
an2 = ax.annotate("Test 2", xy=(0.1, 0.1), xycoords="data",
                  xytext=(0, -10), textcoords=offset_from,
                  # xytext is offset points from "xy=(0.5, 0), xycoords=an1"
                  va="top", ha="center",
                  bbox=dict(boxstyle="round", fc="w"),
                  arrowprops=dict(arrowstyle="->"))
```

Non-text annotations

Using ConnectionPatch

`ConnectionPatch` is like an annotation without text. While `annotate` is sufficient in most situations, `ConnectionPatch` is useful when you want to connect points in different Axes. For example, here we connect the point *xy* in the data coordinates of `ax1` to point *xy* in the data coordinates of `ax2`:

```
from matplotlib.patches import ConnectionPatch

fig, (ax1, ax2) = plt.subplots(nrows=1, ncols=2, figsize=(6, 3))
xy = (0.3, 0.2)
con = ConnectionPatch(xyA=xy, coordsA=ax1.transData,
                      xyB=xy, coordsB=ax2.transData)

fig.add_artist(con)
```

Here, we added the `ConnectionPatch` to the *figure* (with `add_artist`) rather than to either Axes.

This ensures that the ConnectionPatch artist is drawn on top of both Axes, and is also necessary when using constrained_layout for positioning the Axes.

Zoom effect between Axes

`mpl_toolkits.axes_grid1.inset_locator` defines some patch classes useful for interconnecting two Axes.

The code for this figure is at /gallery/subplots_axes_and_figures/axes_zoom_effect and familiarity with transforms_tutorial is recommended.

Customizing with style sheets and rcParams

Source: https://matplotlib.org/stable/users/explain/customizing.html

Many aspects of Matplotlib's behavior and default styles can be customized through the use of rc (runtime configuration) settings. The current values are stored in `rcParams`.

There are three ways to customize Matplotlib, all of which effectively change `rcParams`:

1. Setting rcParams at runtime.
2. Using style sheets.
3. Changing your matplotlibrc file.

Setting rcParams at runtime takes precedence over style sheets, style sheets take precedence over `matplotlibrc` files.

Runtime rc settings

You can dynamically change the default rc (runtime configuration) settings in a python script or interactively from the python shell. All rc settings are stored in a dictionary-like variable called `matplotlib.rcParams`, which is global to the matplotlib package. See `matplotlib.rcParams` for a full list of configurable rcParams. rcParams can be modified directly, for example:

```
from cycler import cycler

import matplotlib.pyplot as plt
import numpy as np

import matplotlib as mpl

mpl.rcParams['lines.linewidth'] = 2
mpl.rcParams['lines.linestyle'] = '--'
data = np.random.randn(50)
plt.plot(data)
```

Note, that in order to change the usual `plot` color you have to change the *prop_cycle* property of *axes*:

```
mpl.rcParams['axes.prop_cycle'] = cycler(color=['r', 'g', 'b', 'y'])
plt.plot(data)  # first color is red
```

Matplotlib also provides a couple of convenience functions for modifying rc settings. `matplotlib.rc` can be used to modify multiple settings in a single group at once, using keyword arguments:

```
mpl.rc('lines', linewidth=4, linestyle='-.')
plt.plot(data)
```

Temporary rc settings

The `matplotlib.rcParams` object can also be changed temporarily using the `matplotlib.rc_context` context manager:

```
with mpl.rc_context({'lines.linewidth': 2, 'lines.linestyle': ':'}):
    plt.plot(data)
```

`matplotlib.rc_context` can also be used as a decorator to modify the defaults within a function:

```
@mpl.rc_context({'lines.linewidth': 3, 'lines.linestyle': '-'})
def plotting_function():
    plt.plot(data)

plotting_function()
```

`matplotlib.rcdefaults` will restore the standard Matplotlib default settings.

There is some degree of validation when setting the values of rcParams, see `matplotlib.rcsetup` for details.

Using style sheets

Another way to change the visual appearance of plots is to set the rcParams in a so-called style sheet and import that style sheet with `matplotlib.style.use`. In this way you can switch easily between different styles by simply changing the imported style sheet. A style sheets looks the same as a matplotlibrc file, but in a style sheet you can only set rcParams that are related to the actual style of a plot. Other rcParams, like *backend*, will be ignored. `matplotlibrc` files support all rcParams. The rationale behind this is to make style sheets portable between different machines without having to worry about dependencies which might or might not be installed on another machine. For a full list of rcParams see `matplotlib.rcParams`. For a list of rcParams that are ignored in style sheets see `matplotlib.style.use`.

There are a number of pre-defined styles `provided by Matplotlib </gallery/style_sheets/style_sheets_reference>`. For example, there's a pre-defined style called "ggplot", which emulates the aesthetics of ggplot (a popular plotting package for R). To use this style, add:

```
plt.style.use('ggplot')
```

To list all available styles, use:

```
print(plt.style.available)
```

Defining your own style

You can create custom styles and use them by calling `style.use` with the path or URL to the style sheet.

For example, you might want to create `./images/presentation.mplstyle` with the following:

```
axes.titlesize : 24
axes.labelsize : 20
lines.linewidth : 3
lines.markersize : 10
xtick.labelsize : 16
ytick.labelsize : 16
```

Then, when you want to adapt a plot designed for a paper to one that looks good in a presentation, you can just add:

```
>>> import matplotlib.pyplot as plt
>>> plt.style.use('./images/presentation.mplstyle')
```

Distributing styles

You can include style sheets into standard importable Python packages (which can be e.g. distributed on PyPI). If your package is importable as `import mypackage`, with a `mypackage/__init__.py` module, and you add a `mypackage/presentation.mplstyle` style sheet, then it can be used as `plt.style.use("mypackage.presentation")`. Subpackages (e.g. `dotted.package.name`) are also supported.

Alternatively, you can make your style known to Matplotlib by placing your `<style-name>.mplstyle` file into `mpl_configdir/stylelib`. You can then load your custom style sheet with a call to `style.use(<style-name>)`. By default `mpl_configdir` should be `~/.config/matplotlib`, but you can check where yours is with `matplotlib.get_configdir()`; you may need to create this directory. You also can change the directory where Matplotlib looks for the stylelib/ folder by setting the `MPLCONFIGDIR` environment variable, see locating-matplotlib-config-dir.

Note that a custom style sheet in `mpl_configdir/stylelib` will override a style sheet defined by Matplotlib if the styles have the same name.

Once your `<style-name>.mplstyle` file is in the appropriate `mpl_configdir` you can specify your style with:

```
>>> import matplotlib.pyplot as plt
>>> plt.style.use(<style-name>)
```

Composing styles

Style sheets are designed to be composed together. So you can have a style sheet that customizes colors and a separate style sheet that alters element sizes for presentations. These styles can easily be combined by passing a list of styles:

```
>>> import matplotlib.pyplot as plt
>>> plt.style.use(['dark_background', 'presentation'])
```

Note that styles further to the right will overwrite values that are already defined by styles on the left.

Temporary styling

If you only want to use a style for a specific block of code but don't want to change the global styling, the style package provides a context manager for limiting your changes to a specific scope. To isolate your styling changes, you can write something like the following:

```
with plt.style.context('dark_background'):
    plt.plot(np.sin(np.linspace(0, 2 * np.pi)), 'r-o')
plt.show()
```

The `matplotlibrc` file

Matplotlib uses `matplotlibrc` configuration files to customize all kinds of properties, which we call 'rc settings' or 'rc parameters'. You can control the defaults of almost every property in Matplotlib: figure size and DPI, line width, color and style, Axes, axis and grid properties, text and font properties and so on. The `matplotlibrc` is read at startup to configure Matplotlib. Matplotlib looks for `matplotlibrc` in four locations, in the following order:

1. `matplotlibrc` in the current working directory, usually used for specific customizations that you do not want to apply elsewhere.
2. `$MATPLOTLIBRC` if it is a file, else `$MATPLOTLIBRC/matplotlibrc`.

3. It next looks in a user-specific place, depending on your platform:

   - On Linux and FreeBSD, it looks in `.config/matplotlib/matplotlibrc` (or `$XDG_CONFIG_HOME/matplotlib/matplotlibrc`) if you've customized your environment.
   - On other platforms, it looks in `.matplotlib/matplotlibrc`.

   See locating-matplotlib-config-dir.

4. `{INSTALL}/matplotlib/mpl-data/matplotlibrc`, where `{INSTALL}` is something like `/usr/lib/python3.10/site-packages` on Linux, and maybe `C:\\Python310\\Lib\\site-packages` on Windows. Every time you install matplotlib, this file will be overwritten, so if you want your customizations to be saved, please move this file to your user-specific matplotlib directory.

Once a `matplotlibrc` file has been found, it will *not* search any of the other paths. When a style sheet is given with `style.use('<path>/<style-name>.mplstyle')`, settings specified in the style sheet take precedence over settings in the `matplotlibrc` file.

To display where the currently active `matplotlibrc` file was loaded from, one can do the following:

```
>>> import matplotlib
>>> matplotlib.matplotlib_fname()
'/home/foo/.config/matplotlib/matplotlibrc'
```

See below for a sample matplotlibrc file and see `matplotlib.rcParams` for a full list of configurable rcParams.

The default `matplotlibrc` file

Performance

Source: https://matplotlib.org/stable/users/explain/artists/performance.html

Whether exploring data in interactive mode or programmatically saving lots of plots, rendering performance can be a challenging bottleneck in your pipeline. Matplotlib provides multiple ways to greatly reduce rendering time at the cost of a slight change (to a settable tolerance) in your plot's appearance. The methods available to reduce rendering time depend on the type of plot that is being created.

Line segment simplification

For plots that have line segments (e.g. typical line plots, outlines of polygons, etc.), rendering performance can be controlled by `path.simplify` and `path.simplify_threshold`, which can be defined e.g. in the `matplotlibrc` file (see customizing for more information about the `matplotlibrc` file). `path.simplify` is a Boolean indicating whether or not line segments are simplified at all. `path.simplify_threshold` controls how much line segments are simplified; higher thresholds result in quicker rendering.

The following script will first display the data without any simplification, and then display the same data with simplification. Try interacting with both of them:

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib as mpl

# Setup, and create the data to plot
y = np.random.rand(100000)
y[50000:] *= 2
y[np.geomspace(10, 50000, 400).astype(int)] = -1
mpl.rcParams['path.simplify'] = True

mpl.rcParams['path.simplify_threshold'] = 0.0
plt.plot(y)
plt.show()

mpl.rcParams['path.simplify_threshold'] = 1.0
plt.plot(y)
plt.show()
```

Matplotlib currently defaults to a conservative simplification threshold of `1/9`. To change default settings to use a different value, change the `matplotlibrc` file. Alternatively, users can create a new style for interactive plotting (with maximal simplification) and another style for publication quality plotting (with minimal simplification) and activate them as necessary. See customizing for instructions on how to perform these actions.

The simplification works by iteratively merging line segments into a single vector until the next line segment's perpendicular distance to the vector (measured in display-coordinate space) is greater than the `path.simplify_threshold` parameter.

> Note
> Changes related to how line segments are simplified were made in version 2.1. Rendering

time will still be improved by these parameters prior to 2.1, but rendering time for some kinds of data will be vastly improved in versions 2.1 and greater.

Marker subsampling

Markers can also be simplified, albeit less robustly than line segments. Marker subsampling is only available to `Line2D` objects (through the `markevery` property). Wherever `Line2D` construction parameters are passed through, such as `pyplot.plot` and `Axes.plot`, the `markevery` parameter can be used:

```
plt.plot(x, y, markevery=10)
```

The `markevery` argument allows for naive subsampling, or an attempt at evenly spaced (along the *x* axis) sampling. See the /gallery/lines_bars_and_markers/markevery_demo for more information.

Splitting lines into smaller chunks

If you are using the Agg backend (see what-is-a-backend), then you can make use of `agg.path.chunksize` This allows users to specify a chunk size, and any lines with greater than that many vertices will be split into multiple lines, each of which has no more than `agg.path.chunksize` many vertices. (Unless `agg.path.chunksize` is zero, in which case there is no chunking.) For some kind of data, chunking the line up into reasonable sizes can greatly decrease rendering time.

The following script will first display the data without any chunk size restriction, and then display the same data with a chunk size of 10,000. The difference can best be seen when the figures are large, try maximizing the GUI and then interacting with them:

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib as mpl
mpl.rcParams['path.simplify_threshold'] = 1.0

# Setup, and create the data to plot
y = np.random.rand(100000)
y[50000:] *= 2
y[np.geomspace(10, 50000, 400).astype(int)] = -1
mpl.rcParams['path.simplify'] = True

mpl.rcParams['agg.path.chunksize'] = 0
plt.plot(y)
plt.show()

mpl.rcParams['agg.path.chunksize'] = 10000
plt.plot(y)
plt.show()
```

Legends

The default legend behavior for axes attempts to find the location that covers the fewest data points (`loc='best'`). This can be a very expensive computation if there are lots of data points. In this case, you may want to provide a specific location.

Using the *fast* style

The *fast* style can be used to automatically set simplification and chunking parameters to reasonable settings to speed up plotting large amounts of data. The following code runs it:

```
import matplotlib.style as mplstyle
mplstyle.use('fast')
```

It is very lightweight, so it works well with other styles. Be sure the fast style is applied last so that other styles do not overwrite the settings:

```
mplstyle.use(['dark_background', 'ggplot', 'fast'])
```

API quick reference

Generated from the docstrings of the installed Matplotlib 3.11.2 package, which are the text the official API reference pages are built from. Full reference: https://matplotlib.org/stable/api/index.html

pyplot entry points

matplotlib.pyplot.subplots

```
matplotlib.pyplot.subplots(nrows: 'int' = 1, ncols: 'int' = 1, *, sharex: "bool | Literal['none', 'all',
↪  'row', 'col']" = False, sharey: "bool | Literal['none', 'all', 'row', 'col']" = False, squeeze:
↪  'bool' = True, width_ratios: 'Sequence[float] | None' = None, height_ratios: 'Sequence[float] | None'
↪  = None, subplot_kw: 'dict[str, Any] | None' = None, gridspec_kw: 'dict[str, Any] | None' = None,
↪  **fig_kw: 'Any') -> 'tuple ...)
```

Create a figure and a set of subplots.

This utility wrapper makes it convenient to create common layouts of subplots, including the enclosing figure object, in a single call.

Parameters

`nrows, ncols` : ***int, default: 1*** Number of rows/columns of the subplot grid.

`sharex, sharey` : ***bool or {'none', 'all', 'row', 'col'}, default: False*** Controls sharing of properties among x (*sharex*) or y (*sharey*) axes:

- True or 'all': x- or y-axis will be shared among all subplots.
- False or 'none': each subplot x- or y-axis will be independent.
- 'row': each subplot row will share an x- or y-axis.
- 'col': each subplot column will share an x- or y-axis.

When subplots have a shared x-axis along a column, only the x tick labels of the bottom subplot are created. Similarly, when subplots have a shared y-axis along a row, only the y tick labels of the first column subplot are created. To later turn other subplots' ticklabels on, use `tick_params`.

When subplots have a shared axis that has units, calling `Axis.set_units` will update each axis with the new units.

Note that it is not possible to unshare axes.

`squeeze` : ***bool, default: True***

- If True, extra dimensions are squeezed out from the returned array of `Axes`:
  - if only one subplot is constructed (nrows=ncols=1), the resulting single Axes object is returned as a scalar.
  - for Nx1 or 1xM subplots, the returned object is a 1D numpy object array of Axes objects.
  - for NxM, subplots with N>1 and M>1 are returned as a 2D array.
- If False, no squeezing at all is done: the returned Axes object is always a 2D array containing Axes instances, even if it ends up being 1x1.

width_ratios : *array-like of length ncols, optional* Defines the relative widths of the columns. Each column gets a relative width of `width_ratios[i] / sum(width_ratios)`. If not given, all columns will have the same width. Equivalent to `gridspec_kw={'width_ratios': [...]}`.

height_ratios : *array-like of length nrows, optional* Defines the relative heights of the rows. Each row gets a relative height of `height_ratios[i] / sum(height_ratios)`. If not given, all rows will have the same height. Convenience for `gridspec_kw={'height_ratios': [...]}`.

subplot_kw : *dict, optional* Dict with keywords passed to the `add_subplot` call used to create each subplot.

gridspec_kw : *dict, optional* Dict with keywords passed to the `GridSpec` constructor used to create the grid the subplots are placed on.

****fig_kw** All additional keyword arguments are passed to the `pyplot.figure` call.

Returns

fig : *"Figure"*

ax : *"Axes" or array of Axes* *ax* can be either a single `Axes` object, or an array of Axes objects if more than one subplot was created. The dimensions of the resulting array can be controlled with the squeeze keyword, see above.

Typical idioms for handling the return value are:

```
# using the variable ax for a single Axes
fig, ax = plt.subplots()

# using the variable axs for multiple Axes
fig, axs = plt.subplots(2, 2)

# using tuple unpacking for multiple Axes
fig, (ax1, ax2) = plt.subplots(1, 2)
fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2)
```

The names `ax` and pluralized `axs` are preferred over `axes` because for the latter it's not clear if it refers to a single `Axes` instance or a collection of these.

See also: `.pyplot.figure`, `.pyplot.subplot`, `.pyplot.axes`, `.Figure.subplots`, `.Figure.add_subplot`

Examples

:

First create some toy data: x = np.linspace(0, 2*np.pi, 400) y = np.sin(x**2) # Create just a figure and only one subplot fig, ax = plt.subplots() ax.plot(x, y) ax.set_title('Simple plot') # Create two subplots and unpack the output array immediately f, (ax1, ax2) = plt.subplots(1, 2, sharey=True) ax1.plot(x, y) ax1.set_title('Sharing Y axis') ax2.scatter(x, y)

...

matplotlib.pyplot.subplot_mosaic

```
matplotlib.pyplot.subplot_mosaic(mosaic: 'str | list[HashableList[_T]] | list[HashableList[Hashable]]',
↪  *, sharex: 'bool' = False, sharey: 'bool' = False, width_ratios: 'ArrayLike | None' = None,
↪  height_ratios: 'ArrayLike | None' = None, empty_sentinel: 'Any' = '.', subplot_kw: 'dict[str, Any] |
↪  None' = None, gridspec_kw: 'dict[str, Any] | None' = None, per_subplot_kw: 'dict[str | tuple[str,
↪  ...], dict[str, Any]] | dict[_T | tup ...)
```

Build a layout of Axes based on ASCII art or nested lists.

This is a helper function to build complex GridSpec layouts visually.

See mosaic for an example and full API documentation

Parameters

mosaic : *list of list of {hashable or nested} or str* A visual layout of how you want your Axes to be arranged labeled as strings. For example :

```
x = [['A panel', 'A panel', 'edge'],
     ['C panel', '.',       'edge']]
```

produces 4 Axes:

- 'A panel' which is 1 row high and spans the first two columns
- 'edge' which is 2 rows high and is on the right edge
- 'C panel' which in 1 row and 1 column wide in the bottom left
- a blank space 1 row and 1 column wide in the bottom center

Any of the entries in the layout can be a list of lists of the same form to create nested layouts.

If input is a str, then it must be of the form :

```
'''
AAE
C.E
'''
```

where each character is a column and each line is a row. This only allows only single character Axes labels and does not allow nesting but is very terse.

sharex, sharey : *bool, default: False* If True, the x-axis (*sharex*) or y-axis (*sharey*) will be shared among all subplots. In that case, tick label visibility and axis units behave as for `subplots`. If False, each subplot's x- or y-axis will be independent.

width_ratios : *array-like of length ncols, optional* Defines the relative widths of the columns. Each column gets a relative width of `width_ratios[i] / sum(width_ratios)`. If not given, all columns will have the same width. Convenience for `gridspec_kw={'width_ratios': [...]}`.

height_ratios : *array-like of length nrows, optional* Defines the relative heights of the rows. Each row gets a relative height of `height_ratios[i] / sum(height_ratios)`. If not given, all rows will have the same height. Convenience for `gridspec_kw={'height_ratios': [...]}`.

empty_sentinel : *object, optional* Entry in the layout to mean ”leave this space empty”. Defaults to `'.'`. Note, if *layout* is a string, it is processed via `inspect.cleandoc` to remove leading white space, which may interfere with using white-space as the empty sentinel.

subplot_kw : *dict, optional* Dictionary with keywords passed to the `Figure.add_subplot` call used to create each subplot. These values may be overridden by values in *per_subplot_kw*.

per_subplot_kw : *dict, optional* A dictionary mapping the Axes identifiers or tuples of identifiers to a dictionary of keyword arguments to be passed to the `Figure.add_subplot` call used to create each subplot. The values in these dictionaries have precedence over the values in *subplot_kw*.

If *mosaic* is a string, and thus all keys are single characters, it is possible to use a single string instead of a tuple as keys; i.e. `"AB"` is equivalent to `("A", "B")`.

New in version 3.7.

gridspec_kw : *dict, optional* Dictionary with keywords passed to the `GridSpec` constructor used to create the grid the subplots are placed on.

****fig_kw** All additional keyword arguments are passed to the `pyplot.figure` call.

Returns

fig : *“Figure“* The new figure

dict[label, Axes] A dictionary mapping the labels to the Axes objects. The order of the Axes is left-to-right and top-to-bottom of their position in the total layout.

matplotlib.pyplot.figure

```
matplotlib.pyplot.figure(num: 'int | str | Figure | SubFigure | None' = None, figsize: "ArrayLike |
↪  tuple[float, float, Literal['in', 'cm', 'px']] | None" = None, dpi: 'float | None' = None, *,
↪  facecolor: 'ColorType | None' = None, edgecolor: 'ColorType | None' = None, frameon: 'bool' = True,
↪  FigureClass: 'type[Figure]' = <class 'matplotlib.figure.Figure'>, clear: 'bool' = False, **kwargs) ->
↪  'Figure'
```

Create a new figure, or activate an existing figure.

Parameters

num : *int or str or “Figure“ or “SubFigure“, optional* A unique identifier for the figure.

If a figure with that identifier already exists, this figure is made active and returned. An integer refers to the `Figure.number` attribute, a string refers to the figure label.

If there is no figure with the identifier or *num* is not given, a new figure is created, made active and returned. If *num* is an int, it will be used for the `Figure.number` attribute, otherwise, an auto-generated integer value is used (starting at 1 and incremented for each new figure). If *num* is a string, the figure label and the window title is set to this value. If num is a `SubFigure`, its parent `Figure` is activated.

If *num* is a Figure instance that is already tracked in pyplot, it is activated. If *num* is a Figure instance that is not tracked in pyplot, it is added to the tracked figures and activated.

figsize : *(float, float) or (float, float, str), default: “figure.figsize“* The figure dimensions. This can be

- a tuple `(width, height, unit)`, where *unit* is one of ”in”, ”cm”, ”px”.
- a tuple `(x, y)`, which is interpreted as `(x, y, "in")`.

One of *width* or *height* may be `None`; the respective value is taken from `figure.figsize`.

dpi : *float, default: “figure.dpi“* The resolution of the figure in dots-per-inch.

facecolor : *“color“, default: “figure.facecolor“* The background color.

edgecolor : *“color“, default: “figure.edgecolor“* The border color.

frameon : *bool, default: True* If False, suppress drawing the figure frame.

FigureClass : *subclass of “Figure“* If set, an instance of this subclass will be created, rather than a plain `Figure`.

clear : *bool, default: False* If True and the figure already exists, then it is cleared.

layout : *{'constrained', 'compressed', 'tight', 'none', “LayoutEngine“, None}, default: None* The layout mechanism for positioning of plot elements to avoid overlapping Axes decorations (labels, ticks, etc). Note that layout managers can measurably slow down figure display.

- 'constrained': The constrained layout solver adjusts Axes sizes to avoid overlapping Axes decorations. Can handle complex plot layouts and colorbars, and is thus recommended.

  See constrainedlayout_guide for examples.

- 'compressed': uses the same algorithm as 'constrained', but removes extra space between fixed-aspect-ratio Axes. Best for simple grids of Axes.
- 'tight': Use the tight layout mechanism. This is a relatively simple algorithm that adjusts the subplot parameters so that decorations do not overlap. See `Figure.set_tight_layout` for further details.
- 'none': Do not use a layout engine.
- A `LayoutEngine` instance. Builtin layout classes are `ConstrainedLayoutEngine` and `TightLayoutEngine`, more easily accessible by 'constrained' and 'tight'. Passing an instance allows third parties to provide their own layout engine.

If not given, fall back to using the parameters *tight_layout* and *constrained_layout*, including their config defaults `figure.autolayout` and `figure.constrained_layout.use`.

`**kwargs` Additional keyword arguments are passed to the `Figure` constructor.

Returns

"Figure"

Notes

A newly created figure is passed to the `new_manager` method or the `new_figure_manager` function provided by the current backend, which install a canvas and a manager on the figure.

Once this is done, `figure.hooks` are called, one at a time, on the figure; these hooks allow arbitrary customization of the figure (e.g., attaching callbacks) or of associated elements (e.g., modifying the toolbar). See /gallery/user_interfaces/mplcvd for an example of toolbar customization.

If you are creating many figures, make sure you explicitly call `pyplot.close` on the figures you are not using, because this will enable pyplot to properly clean up the memory.

(Notes truncated; see the full reference page.)

matplotlib.pyplot.savefig

```
matplotlib.pyplot.savefig(fname: 'str | os.PathLike | IO', **kwargs) -> 'None'
```

Save the current figure as an image or vector graphic to a file.

Call signature:

```
savefig(fname, *, transparent=None, dpi='figure', format=None,
        metadata=None, bbox_inches=None, pad_inches=0.1,
        facecolor='auto', edgecolor='auto', backend=None,
        **kwargs
       )
```

The available output formats depend on the backend being used.

Parameters

`fname` **: *str or path-like or binary file-like*** A path, or a Python file-like object, or possibly some backend-dependent object such as `matplotlib.backends.backend_pdf.PdfPages`.

If *format* is set, it determines the output format, and the file is saved as *fname*. Note that *fname* is used verbatim, and there is no attempt to make the extension, if any, of *fname* match *format*, and no extension is appended.

If *format* is not set, then the format is inferred from the extension of *fname*, if there is one. If *format* is not set and *fname* has no extension, then the file is saved with `savefig.format` and the appropriate extension is appended to *fname*.

Notes

> Note
> This is the pyplot wrapper for `Figure.savefig`.

matplotlib.pyplot.close

```
matplotlib.pyplot.close(fig: "None | int | str | Figure | Literal['all']" = None) -> 'None'
```

Close a figure window, and unregister it from pyplot.

Parameters

`fig` : ***None or int or str or "Figure"*** The figure to close. There are a number of ways to specify this:

- *None*: the current figure
- `Figure`: the given `Figure` instance
- `int`: a figure number
- `str`: a figure name
- 'all': all figures

Notes

pyplot maintains a reference to figures created with `figure()`. When work on the figure is completed, it should be closed, i.e. deregistered from pyplot, to free its memory (see also `figure.max_open_warning`). Closing a figure window created by `show()` automatically deregisters the figure. For all other use cases, most prominently `savefig()` without `show()`, the figure must be deregistered explicitly using `close()`.

matplotlib.pyplot.style.use

```
matplotlib.pyplot.style.use(style)
```

Use Matplotlib style settings from a style specification.

The style name of 'default' is reserved for reverting back to the default style settings.

> Note
> This updates the `rcParams` with the settings from the style. `rcParams` not defined in the style are kept.

Parameters

`style` : ***str, dict, Path or list*** A style specification. Valid options are:

str
- One of the style names in `style.available` (a builtin style or a style installed in the user library path).
- A dotted name of the form "package.style_name"; in that case, "package" should be an importable Python package name, e.g. at `/path/to/package/__init__.py`; the loaded style file is `/path/to/package/style_name.mplstyle`. (Style files in subpackages are likewise supported.)
- The path or URL to a style file, which gets loaded by `rc_params_from_file`.

dict A mapping of key/value pairs for `matplotlib.rcParams`.

Path The path to a style file, which gets loaded by `rc_params_from_file`.

list A list of style specifiers (str, Path or dict), which are applied from first to last in the list.

Notes

The following `rcParams` are not related to style and will be ignored if found in a style specification:

(Notes truncated; see the full reference page.)

Plotting on an Axes

matplotlib.axes.Axes.plot

```
matplotlib.axes.Axes.plot(self, *args, scalex=True, scaley=True, data=None, **kwargs)
```

Plot y versus x as lines and/or markers.

Call signatures:

```
plot([x], y, [fmt], *, data=None, **kwargs)
plot([x], y, [fmt], [x2], y2, [fmt2], ..., **kwargs)
```

The coordinates of the points or line nodes are given by *x*, *y*.

The optional parameter *fmt* is a convenient way for defining basic formatting like color, marker and linestyle. It's a shortcut string notation described in the *Notes* section below.

```
>>> plot(x, y)        # plot x and y using default line style and color
>>> plot(x, y, 'bo')  # plot x and y using blue circle markers
>>> plot(y)           # plot y using x as index array 0..N-1
>>> plot(y, 'r+')     # ditto, but with red plusses
```

You can use `Line2D` properties as keyword arguments for more control on the appearance. Line properties and *fmt* can be mixed. The following two calls yield identical results:

```
>>> plot(x, y, 'go--', linewidth=2, markersize=12)
>>> plot(x, y, color='green', marker='o', linestyle='dashed',
...      linewidth=2, markersize=12)
```

When conflicting with *fmt*, keyword arguments take precedence.

Plotting labelled data

There's a convenient way for plotting objects with labelled data (i.e. data that can be accessed by index `obj['y']`). Instead of giving the data in *x* and *y*, you can provide the object in the *data* parameter and just give the labels for *x* and *y*:

```
>>> plot('xlabel', 'ylabel', data=obj)
```

All indexable objects are supported. This could e.g. be a `dict`, a `pandas.DataFrame` or a structured numpy array.

Plotting multiple sets of data

There are various ways to plot multiple sets of data.

- The most straight forward way is just to call `plot` multiple times. Example:

  ```
  >>> plot(x1, y1, 'bo')
  >>> plot(x2, y2, 'go')
  ```

- If *x* and/or *y* are 2D arrays, a separate data set will be drawn for every column. If both *x* and *y* are 2D, they must have the same shape. If only one of them is 2D with shape (N, m) the other must have length N and will be used for every data set m.

Example:

```
>>> x = [1, 2, 3]
>>> y = np.array([[1, 2], [3, 4], [5, 6]])
>>> plot(x, y)
```

is equivalent to:

```
>>> for col in range(y.shape[1]):
...     plot(x, y[:, col])
```

- The third way is to specify multiple sets of *[x], y, [fmt]* groups:

```
>>> plot(x1, y1, 'g^', x2, y2, 'g-')
```

In this case, any additional keyword argument applies to all datasets. Also, this syntax cannot be combined with the *data* parameter.

By default, each line is assigned a different style specified by a 'style cycle'. The *fmt* and line property parameters are only necessary if you want explicit deviations from these defaults. Alternatively, you can also change the style cycle using `axes.prop_cycle`.

Parameters

`x, y` : *array-like or float* The horizontal / vertical coordinates of the data points. *x* values are optional and default to `range(len(y))`.

Commonly, these parameters are 1D arrays.

They can also be scalars, or two-dimensional (in that case, the columns represent separate data sets).

These arguments cannot be passed as keywords.

`fmt` : *str, optional* A format string, e.g. 'ro' for red circles. See the *Notes* section for a full description of the format strings.

Format strings are just an abbreviation for quickly setting basic line properties. All of these and more can also be controlled by keyword arguments.

This argument cannot be passed as keyword.

`data` : *indexable object, optional* An object with labelled data. If given, provide the label names to plot in *x* and *y*.

> Note
> Technically there's a slight ambiguity in calls where the second label is a valid *fmt*. `plot('n', 'o', data=obj)` could be `plt(x, y)` or `plt(y, fmt)`. In such cases, the former interpretation is chosen, but a warning is issued. You may suppress the warning by adding an empty format string `plot('n', 'o', '', data=obj)`.

Returns

list of "Line2D" A list of lines representing the plotted data.

See also: `scatter`

Notes

Format Strings

A format string consists of a part for color, marker and line:

```
fmt = '[marker][line][color]'
```

Each of them is optional. If not provided, the value from the style cycle is used. Exception: If `line` is given, but no `marker`, the data will be a line without markers.

Other combinations such as `[color][marker][line]` are also supported, but note that their parsing may be ambiguous.

(Notes truncated; see the full reference page.)

matplotlib.axes.Axes.scatter

```
matplotlib.axes.Axes.scatter(self, x, y, s=None, c=None, *, marker=None, cmap=None, norm=None, vmin=None,
↪  vmax=None, alpha=None, linewidths=None, edgecolors=None, colorizer=None, plotnonfinite=False,
↪  data=None, **kwargs)
```

A scatter plot of *y* vs. *x* with varying marker size and/or color.

Parameters

`x, y` : *float or array-like, shape (n,)* The data positions.

`s` : *float or array-like, shape (n,), optional* The marker size in points*2 (typographic points are 1/72 in.). Default is “rcParams['lines.markersize']* 2“.

The linewidth and edgecolor can visually interact with the marker size, and can lead to artifacts if the marker size is smaller than the linewidth.

If the linewidth is greater than 0 and the edgecolor is anything but '*none*', then the effective size of the marker will be increased by half the linewidth because the stroke will be centered on the edge of the shape.

To eliminate the marker edge either set *linewidth=0* or *edgecolor*='*none*'.

`c` : *array-like or list of “color“ or “color“, optional* The marker colors. Possible values:

- A scalar or sequence of n numbers to be mapped to colors using *cmap* and *norm*.
- A 2D array in which the rows are RGB or RGBA.
- A sequence of colors of length n.
- A single color format string.

Note that *c* should not be a single numeric RGB or RGBA sequence because that is indistinguishable from an array of values to be colormapped. If you want to specify the same RGB or RGBA value for all points, use a 2D array with a single row. Otherwise, value-matching will have precedence in case of a size matching with *x* and *y*.

If you wish to specify a single color for all points prefer the *color* keyword argument.

Defaults to `None`. In that case the marker color is determined by the value of *color*, *facecolor* or *facecolors*. In case those are not specified or `None`, the marker color is determined by the next color of the `Axes`' current ”shape and fill” color cycle. This cycle defaults to `axes.prop_cycle`.

`marker` : *“MarkerStyle“, default: “scatter.marker“* The marker style. *marker* can be either an instance of the class or the text shorthand for a particular marker. See `matplotlib.markers` for more information about marker styles.

`cmap` : *str or “Colormap“, default: “image.cmap“* The Colormap instance or registered colormap name used to map scalar data to colors.

This parameter is ignored if *c* is RGB(A).

`norm` : *str or “Normalize“, optional* The normalization method used to scale scalar data to the [0, 1] range before mapping to colors using *cmap*. By default, a linear scaling is used, mapping the lowest value to 0 and the highest to 1.

If given, this can be one of the following:

- An instance of `Normalize` or one of its subclasses (see colormapnorms).
- A scale name, i.e. one of "linear", "log", "symlog", "logit", etc. For a list of available scales, call `matplotlib.scale.get_scale_names()`. In that case, a suitable `Normalize` subclass is dynamically generated and instantiated.

This parameter is ignored if *c* is RGB(A).

`vmin, vmax` : *float, optional* When using scalar data and no explicit *norm*, *vmin* and *vmax* define the data range that the colormap covers. By default, the colormap covers the complete value range of the supplied data. It is an error to use *vmin*/*vmax* when a *norm* instance is given (but using a `str` *norm* name together with *vmin*/*vmax* is acceptable).

This parameter is ignored if *c* is RGB(A).

`alpha` : *float, default: None* The alpha blending value, between 0 (transparent) and 1 (opaque).

`linewidths` : *float or array-like, default: "lines.linewidth"* The linewidth of the marker edges. Note: The default *edgecolors* is 'face'. You may want to change this as well.

`edgecolors` : *{'face', 'none', None} or "color" or list of "color", default: "scatter.edgecolors"* The edge color of the marker. Possible values:

- 'face': The edge color will always be the same as the face color.
- 'none': No patch boundary will be drawn.
- A color or sequence of colors.

For non-filled markers, *edgecolors* is ignored. Instead, the color is determined like with 'face', i.e. from *c*, *colors*, or *facecolors*.

`colorizer` : *"Colorizer" or None, default: None* The Colorizer object used to map color to data. If None, a Colorizer object is created from a *norm* and *cmap*.

This parameter is ignored if *c* is RGB(A).

`plotnonfinite` : *bool, default: False* Whether to plot points with nonfinite *c* (i.e. `inf`, `-inf` or `nan`). If `True` the points are drawn with the *bad* colormap color (see `Colormap.set_bad`).

Returns

"PathCollection"

See also: `plot`

Notes

- The `plot` function will be faster for scatterplots where markers don't vary in size or color.
- Any or all of *x*, *y*, *s*, and *c* may be masked arrays, in which case all masks will be combined and only unmasked points will be plotted.
- Fundamentally, scatter works with 1D arrays; *x*, *y*, *s*, and *c* may be input as N-D arrays, but within scatter they will be flattened. The exception is *c*, which will be flattened only if its size matches the size of *x* and *y*.

matplotlib.axes.Axes.bar

```
matplotlib.axes.Axes.bar(self, x, height, width=0.8, bottom=None, *, align='center', data=None, **kwargs)
```

Make a bar plot.

The bars are positioned at *x* with the given *align*ment. Their dimensions are given by *height* and *width*. The vertical baseline is *bottom* (default 0).

Many parameters can take either a single value applying to all bars or a sequence of values, one for each bar.

Parameters

x : *float or array-like* The x coordinates of the bars. See also *align* for the alignment of the bars to the coordinates.

Bars are often used for categorical data, i.e. string labels below the bars. You can provide a list of strings directly to *x*. `bar(['A', 'B', 'C'], [1, 2, 3])` is often a shorter and more convenient notation compared to `bar(range(3), [1, 2, 3], tick_label=['A', 'B', 'C'])`. They are equivalent as long as the names are unique. The explicit *tick_label* notation draws the names in the sequence given. However, when having duplicate values in categorical *x* data, these values map to the same numerical x coordinate, and hence the corresponding bars are drawn on top of each other.

height : *float or array-like* The height(s) of the bars.

Note that if *bottom* has units (e.g. datetime), *height* should be in units that are a difference from the value of *bottom* (e.g. timedelta).

width : *float or array-like, default: 0.8* The width(s) of the bars.

Note that if *x* has units (e.g. datetime), then *width* should be in units that are a difference (e.g. timedelta) around the *x* values.

bottom : *float or array-like, default: 0* The y coordinate(s) of the bottom side(s) of the bars.

Note that if *bottom* has units, then the y-axis will get a Locator and Formatter appropriate for the units (e.g. dates, or categorical).

align : *{'center', 'edge'}, default: 'center'* Alignment of the bars to the *x* coordinates:

- 'center': Center the base on the *x* positions.
- 'edge': Align the left edges of the bars with the *x* positions.

To align the bars on the right edge pass a negative *width* and `align='edge'`.

Returns

"BarContainer" Container with all the bars and optionally errorbars.

See also: `barh`, `grouped_bar`, `bar_label`

Notes

Stacked bars can be achieved by passing individual *bottom* values per bar. See /gallery/-lines_bars_and_markers/bar_stacked.

matplotlib.axes.Axes.hist

```
matplotlib.axes.Axes.hist(self, x, bins=None, *, range=None, density=False, weights=None,
↪  cumulative=False, bottom=None, histtype='bar', align='mid', orientation='vertical', rwidth=None,
↪  log=False, color=None, label=None, stacked=False, data=None, **kwargs)
```

Compute and plot a histogram.

This method uses `numpy.histogram` to bin the data in *x* and count the number of values in each bin, then draws the distribution either as a `BarContainer` or `Polygon`. The *bins*, *range*, *density*, and *weights* parameters are forwarded to `numpy.histogram`.

If the data has already been binned and counted, use `bar` or `stairs` to plot the distribution:

```
counts, bins = np.histogram(x)
plt.stairs(counts, bins)
```

Alternatively, plot pre-computed bins and counts using `hist()` by treating each bin as a single point with a weight equal to its count:

```
plt.hist(bins[:-1], bins, weights=counts)
```

The data input *x* can be a singular array, a list of datasets of potentially different lengths ([*x0*, *x1*, ...]), or a 2D ndarray in which each column is a dataset. Note that the ndarray form is transposed relative to the list form. If the input is an array, then the return value is a tuple (*n*, *bins*, *patches*); if the input is a sequence of arrays, then the return value is a tuple ([*n0*, *n1*, ...], *bins*, [*patches0*, *patches1*, ...]).

Masked arrays are not supported.

Parameters

`x` : *(n,) array or sequence of (n,) arrays* Input values, this takes either a single array or a sequence of arrays which are not required to be of the same length.

`bins` : *int or sequence or str, default: "hist.bins"* If *bins* is an integer, it defines the number of equal-width bins in the range.

If *bins* is a sequence, it defines the bin edges, including the left edge of the first bin and the right edge of the last bin; in this case, bins may be unequally spaced. All but the last (righthand-most) bin is half-open. In other words, if *bins* is:

```
[1, 2, 3, 4]
```

then the first bin is `[1, 2)` (including 1, but excluding 2) and the second `[2, 3)`. The last bin, however, is `[3, 4]`, which *includes* 4.

If *bins* is a string, it is one of the binning strategies supported by `numpy.histogram_bin_edges`: 'auto', 'fd', 'doane', 'scott', 'stone', 'rice', 'sturges', or 'sqrt'.

`range` : *tuple or None, default: None* The lower and upper range of the bins. Lower and upper outliers are ignored. If not provided, *range* is `(x.min(), x.max())`. Range has no effect if *bins* is a sequence.

If *bins* is a sequence or *range* is specified, autoscaling is based on the specified bin range instead of the range of x.

`density` : *bool, default: False* If `True`, draw and return a probability density: each bin will display the bin's raw count divided by the total number of counts *and the bin width* (`density = counts / (sum(counts) * np.diff(bins))`), so that the area under the histogram integrates to 1 (`np.sum(density * np.diff(bins)) == 1`).

If *stacked* is also `True`, the sum of the histograms is normalized to 1.

`weights` : *(n,) array-like or None, default: None* An array of weights, of the same shape as *x*. Each value in *x* only contributes its associated weight towards the bin count (instead of 1). If *density* is `True`, the weights are normalized, so that the integral of the density over the range remains 1.

`cumulative` : *bool or -1, default: False* If `True`, then a histogram is computed where each bin gives the counts in that bin plus all bins for smaller values. The last bin gives the total number of datapoints.

If *density* is also `True` then the histogram is normalized such that the last bin equals 1.

If *cumulative* is a number less than 0 (e.g., -1), the direction of accumulation is reversed. In this case, if *density* is also `True`, then the histogram is normalized such that the first bin equals 1.

`bottom` : *array-like or float, default: 0* Location of the bottom of each bin, i.e. bins are drawn from `bottom` to `bottom + hist(x, bins)` If a scalar, the bottom of each bin is shifted by the

same amount. If an array, each bin is shifted independently and the length of bottom must match the number of bins. If None, defaults to 0.

`histtype` : *{'bar', 'barstacked', 'step', 'stepfilled'}, default: 'bar'* The type of histogram to draw.

- 'bar' is a traditional bar-type histogram. If multiple data are given the bars are arranged side by side.
- 'barstacked' is a bar-type histogram where multiple data are stacked on top of each other.
- 'step' generates a lineplot that is by default unfilled.
- 'stepfilled' generates a lineplot that is by default filled.

`align` : *{'left', 'mid', 'right'}, default: 'mid'* The horizontal alignment of the histogram bars.

- 'left': bars are centered on the left bin edges.
- 'mid': bars are centered between the bin edges.
- 'right': bars are centered on the right bin edges.

`orientation` : *{'vertical', 'horizontal'}, default: 'vertical'* If 'horizontal', `barh` will be used for bar-type histograms and the *bottom* kwarg will be the left edges.

`rwidth` : *float or None, default: None* The relative width of the bars as a fraction of the bin width. If `None`, automatically compute the width.

Ignored if *histtype* is 'step' or 'stepfilled'.

`log` : *bool, default: False* If `True`, the histogram axis will be set to a log scale.

`color` : *“color“ or list of “color“ or None, default: None* Color or sequence of colors, one per dataset. Default (`None`) uses the standard line color sequence.

New in version 3.10. It is now possible to use a single color with multiple datasets.

`label` : *str or list of str, optional* String, or sequence of strings to match multiple datasets. Bar charts yield multiple patches per dataset, but only the first gets the label, so that `legend` will work as expected.

`stacked` : *bool, default: False* If `True`, multiple data are stacked on top of each other If `False` multiple data are arranged side by side if histtype is 'bar' or on top of each other if histtype is 'step'

Returns

`n` : *array or list of arrays* The values of the histogram bins. See *density* and *weights* for a description of the possible semantics. If input *x* is an array, then this is an array of length *nbins*. If input is a sequence of arrays `[data1, data2, ...]`, then this is a list of arrays with the values of the histograms for each of the arrays in the same order. The dtype of the array *n* (or of its element arrays) will always be float even if no weighting or normalization is used.

`bins` : *array* The edges of the bins. Length nbins + 1 (nbins left edges and right edge of last bin). Always a single array even when multiple data sets are passed in.

`patches` : *“BarContainer“ or list of a single “Polygon“ or list of such objects* Container of individual artists used to create the histogram or list of such containers if there are multiple input datasets.

See also: `hist2d`, `hexbin`, `stairs`, `bar`

Notes

For large numbers of bins (>1000), plotting can be significantly accelerated by using `stairs` to plot a pre-computed histogram (`plt.stairs(*np.histogram(data))`), or by setting *histtype* to 'step' or 'stepfilled' rather than 'bar' or 'barstacked'.

matplotlib.axes.Axes.boxplot

```
matplotlib.axes.Axes.boxplot(self, x, *, notch=None, sym=None, vert=None, orientation='vertical',
↪  whis=None, positions=None, widths=None, patch_artist=None, bootstrap=None, usermedians=None,
↪  conf_intervals=None, meanline=None, showmeans=None, showcaps=None, showbox=None, showfliers=None,
↪  boxprops=None, tick_labels=None, flierprops=None, medianprops=None, meanprops=None, capprops=None,
↪  whiskerprops=None, manage_ticks=True, au ...)
```

Draw a box and whisker plot.

The box extends from the first quartile (Q1) to the third quartile (Q3) of the data, with a line at the median. The whiskers extend from the box to the farthest data point lying within 1.5x the inter-quartile range (IQR) from the box. Flier points are those past the end of the whiskers. See https://en.wikipedia.org/wiki/Box_plot for reference.

```
Q1-1.5IQR   Q1   median  Q3   Q3+1.5IQR
             |-----:-----|
o      |--------|     :     |--------|    o  o
             |-----:-----|
flier             <----------->            fliers
                  IQR
```

Parameters

x : *1D array or sequence of 1D arrays or 2D array* The input data. Possible values:

- 1D array: A single box is drawn.
- sequence of 1D arrays: A box is drawn for each array in the sequence.
- 2D array: A box is drawn for each column in the array.

notch : *bool, default: "boxplot.notch"* Whether to draw a notched boxplot (`True`), or a rectangular boxplot (`False`). The notches represent the confidence interval (CI) around the median. The documentation for *bootstrap* describes how the locations of the notches are computed by default, but their locations may also be overridden by setting the *conf_intervals* parameter.

> Note
> In cases where the values of the CI are less than the lower quartile or greater than the upper quartile, the notches will extend beyond the box, giving it a distinctive "flipped" appearance. This is expected behavior and consistent with other statistical visualization packages.

sym : *str, optional* The default symbol for flier points. An empty string ('') hides the fliers. If `None`, then the fliers default to 'b+'. More control is provided by the *flierprops* parameter.

`vert` : *bool, optional*

> *Deprecated since version 3.11.* Use *orientation* instead. If this is given during the deprecation period, it overrides the *orientation* parameter.
> If True, plots the boxes vertically. If False, plots the boxes horizontally.

orientation : *{'vertical', 'horizontal'}, default: 'vertical'* If 'horizontal', plots the boxes horizontally. Otherwise, plots the boxes vertically.

New in version 3.10.

whis : *float or (float, float), default: 1.5* The position of the whiskers.

If a float, the lower whisker is at the lowest datum above `Q1 - whis*(Q3-Q1)`, and the upper whisker at the highest datum below `Q3 + whis*(Q3-Q1)`, where Q1 and Q3 are the first and

third quartiles. The default value of `whis = 1.5` corresponds to Tukey's original definition of boxplots.

If a pair of floats, they indicate the percentiles at which to draw the whiskers (e.g., (5, 95)). In particular, setting this to (0, 100) results in whiskers covering the whole range of the data.

In the edge case where `Q1 == Q3`, *whis* is automatically set to (0, 100) (cover the whole range of the data) if *autorange* is True.

Beyond the whiskers, data are considered outliers and are plotted as individual points.

`bootstrap` : *int, optional* Specifies whether to bootstrap the confidence intervals around the median for notched boxplots. If *bootstrap* is None, no bootstrapping is performed, and notches are calculated using a Gaussian-based asymptotic approximation (see McGill, R., Tukey, J.W., and Larsen, W.A., 1978, and Kendall and Stuart, 1967). Otherwise, bootstrap specifies the number of times to bootstrap the median to determine its 95% confidence intervals. Values between 1000 and 10000 are recommended.

`usermedians` : *1D array-like, optional* A 1D array-like of length `len(x)`. Each entry that is not `None` forces the value of the median for the corresponding dataset. For entries that are `None`, the medians are computed by Matplotlib as normal.

`conf_intervals` : *array-like, optional* A 2D array-like of shape `(len(x), 2)`. Each entry that is not None forces the location of the corresponding notch (which is only drawn if *notch* is `True`). For entries that are `None`, the notches are computed by the method specified by the other parameters (e.g., *bootstrap*).

`positions` : *array-like, optional* The positions of the boxes. The ticks and limits are automatically set to match the positions. Defaults to `range(1, N+1)` where N is the number of boxes to be drawn.

`widths` : *float or array-like* The widths of the boxes. The default is 0.5, or `0.15*(distance between extreme positions)`, if that is smaller.

`patch_artist` : *bool, default: "boxplot.patchartist"* If `False` produces boxes with the Line2D artist. Otherwise, boxes are drawn with Patch artists.

`tick_labels` : *list of str, optional* The tick labels of each boxplot. Ticks are always placed at the box *positions*. If *tick_labels* is given, the ticks are labelled accordingly. Otherwise, they keep their numeric values.

Changed in version 3.9. Renamed from *labels*, which is also removed in 3.11.

`manage_ticks` : *bool, default: True* If True, the tick locations and labels will be adjusted to match the boxplot positions.

`autorange` : *bool, default: False* When `True` and the data are distributed such that the 25th and 75th percentiles are equal, *whis* is set to (0, 100) such that the whisker ends are at the minimum and maximum of the data.

`meanline` : *bool, default: "boxplot.meanline"* If `True` (and *showmeans* is `True`), will try to render the mean as a line spanning the full width of the box according to *meanprops* (see below). Not recommended if *shownotches* is also True. Otherwise, means will be shown as points.

`zorder` : *float, default: "Line2D.zorder = 2"* The zorder of the boxplot.

Returns

dict A dictionary mapping each component of the boxplot to a list of the `Line2D` instances created. That dictionary has the following keys (assuming vertical boxplots):

- `boxes`: the main body of the boxplot showing the quartiles and the median's confidence intervals if enabled.
- `medians`: horizontal lines at the median of each box.
- `whiskers`: the vertical lines extending to the most extreme, non-outlier data points.
- `caps`: the horizontal lines at the ends of the whiskers.
- `fliers`: points representing data that extend beyond the whiskers (fliers).
- `means`: points or lines representing the means.

See also: `.Axes.bxp`, `violinplot`

matplotlib.axes.Axes.imshow

```
matplotlib.axes.Axes.imshow(self, X, cmap=None, norm=None, *, aspect=None, interpolation=None,
↪  alpha=None, vmin=None, vmax=None, colorizer=None, origin=None, extent=None, interpolation_stage=None,
↪  filternorm=True, filterrad=4.0, resample=None, url=None, data=None, **kwargs)
```

Display data as an image, i.e., on a 2D regular raster.

The input may either be actual RGB(A) data, or 2D scalar data, which will be rendered as a pseudocolor image. For displaying a grayscale image, set up the colormapping using the parameters `cmap='gray', vmin=0, vmax=255`.

The number of pixels used to render an image is set by the Axes size and the figure *dpi*. This can lead to aliasing artifacts when the image is resampled, because the displayed image size will usually not match the size of *X* (see /gallery/images_contours_and_fields/image_antialiasing). The resampling can be controlled via the *interpolation* parameter and/or `image.interpolation`.

Parameters

`X` : *array-like or PIL image* The image data. Supported array shapes are:

- (M, N): an image with scalar data. The values are mapped to colors using normalization and a colormap. See parameters *norm*, *cmap*, *vmin*, *vmax*.
- (M, N, 3): an image with RGB values (0-1 float or 0-255 int).
- (M, N, 4): an image with RGBA values (0-1 float or 0-255 int), i.e. including transparency.

The first two dimensions (M, N) define the rows and columns of the image.

Out-of-range RGB(A) values are clipped.

`cmap` : *str or “Colormap“, default: “image.cmap“* The Colormap instance or registered colormap name used to map scalar data to colors.

This parameter is ignored if *X* is RGB(A).

`norm` : *str or “Normalize“, optional* The normalization method used to scale scalar data to the [0, 1] range before mapping to colors using *cmap*. By default, a linear scaling is used, mapping the lowest value to 0 and the highest to 1.

If given, this can be one of the following:

- An instance of `Normalize` or one of its subclasses (see colormapnorms).
- A scale name, i.e. one of ”linear”, ”log”, ”symlog”, ”logit”, etc. For a list of available scales, call `matplotlib.scale.get_scale_names()`. In that case, a suitable `Normalize` subclass is dynamically generated and instantiated.

This parameter is ignored if *X* is RGB(A).

`vmin, vmax` : *float, optional* When using scalar data and no explicit *norm*, *vmin* and *vmax* define the data range that the colormap covers. By default, the colormap covers the complete

value range of the supplied data. It is an error to use *vmin*/*vmax* when a *norm* instance is given (but using a `str` *norm* name together with *vmin*/*vmax* is acceptable).

This parameter is ignored if *X* is RGB(A).

`colorizer` : ***"Colorizer" or None, default: None*** The Colorizer object used to map color to data. If None, a Colorizer object is created from a *norm* and *cmap*.

This parameter is ignored if *X* is RGB(A).

`aspect` : ***{'equal', 'auto'} or float or None, default: None*** The aspect ratio of the Axes. This parameter is particularly relevant for images since it determines whether data pixels are square.

This parameter is a shortcut for explicitly calling `Axes.set_aspect`. See there for further details.

- 'equal': Ensures an aspect ratio of 1. Pixels will be square (unless pixel sizes are explicitly made non-square in data coordinates using *extent*).
- 'auto': The Axes is kept fixed and the aspect is adjusted so that the data fit in the Axes. In general, this will result in non-square pixels.

Normally, None (the default) means to use `image.aspect`. However, if the image uses a transform that does not contain the axes data transform, then None means to not modify the axes aspect at all (in that case, directly call `Axes.set_aspect` if desired).

`interpolation` : ***str, default: "image.interpolation"*** The interpolation method used.

Supported values are 'none', 'auto', 'nearest', 'bilinear', 'bicubic', 'spline16', 'spline36', 'hanning', 'hamming', 'hermite', 'kaiser', 'quadric', 'catrom', 'gaussian', 'bessel', 'mitchell', 'sinc', 'lanczos', 'blackman'.

The data *X* is resampled to the pixel size of the image on the figure canvas, using the interpolation method to either up- or downsample the data.

If *interpolation* is 'none', then for the ps, pdf, and svg backends no down- or upsampling occurs, and the image data is passed to the backend as a native image. Note that different ps, pdf, and svg viewers may display these raw pixels differently. On other backends, 'none' is the same as 'nearest'.

If *interpolation* is the default 'auto', then 'nearest' interpolation is used if the image is upsampled by more than a factor of three (i.e. the number of display pixels is at least three times the size of the data array). If the upsampling rate is smaller than 3, or the image is downsampled, then 'hanning' interpolation is used to act as an anti-aliasing filter, unless the image happens to be upsampled by exactly a factor of two or one.

See /gallery/images_contours_and_fields/interpolation_methods for an overview of the supported interpolation methods, and /gallery/images_contours_and_fields/image_antialiasing for a discussion of image antialiasing.

Some interpolation methods require an additional radius parameter, which can be set by *filterrad*. Additionally, the antigrain image resize filter is controlled by the parameter *filternorm*.

`interpolation_stage` : ***{'auto', 'data', 'rgba'}, default: 'auto'*** Supported values:

- 'data': Interpolation is carried out on the data provided by the user This is useful if interpolating between pixels during upsampling.
- 'rgba': The interpolation is carried out in RGBA-space after the color-mapping has been applied. This is useful if downsampling and combining pixels visually.
- 'auto': Select a suitable interpolation stage automatically. This uses 'rgba' when downsampling, or upsampling at a rate less than 3, and 'data' when upsampling at a higher rate.

See /gallery/images_contours_and_fields/image_antialiasing for a discussion of image antialiasing.

alpha : *float or array-like, optional* The alpha blending value, between 0 (transparent) and 1 (opaque). If *alpha* is an array, the alpha blending values are applied pixel by pixel, and *alpha* must have the same shape as *X*.

origin : *{'upper', 'lower'}, default: “image.origin“* Place the [0, 0] index of the array in the upper left or lower left corner of the Axes. The convention (the default) 'upper' is typically used for matrices and images.

Note that the vertical axis points upward for 'lower' but downward for 'upper'.

See the imshow_extent tutorial for examples and a more detailed description.

extent : *floats (left, right, bottom, top), optional* The bounding box in data coordinates that the image will fill. These values may be unitful and match the units of the Axes. The image is stretched individually along x and y to fill the box.

The default extent is determined by the following conditions. Pixels have unit size in data coordinates. Their centers are on integer coordinates, and their center coordinates range from 0 to columns-1 horizontally and from 0 to rows-1 vertically.

Note that the direction of the vertical axis and thus the default values for top and bottom depend on *origin*:

- For `origin == 'upper'` the default is `(-0.5, numcols-0.5, numrows-0.5, -0.5)`.
- For `origin == 'lower'` the default is `(-0.5, numcols-0.5, -0.5, numrows-0.5)`.

See the imshow_extent tutorial for examples and a more detailed description.

filternorm : *bool, default: True* A parameter for the antigrain image resize filter (see the antigrain documentation). If *filternorm* is set, the filter normalizes integer values and corrects the rounding errors. It doesn't do anything with the source floating point values, it corrects only integers according to the rule of 1.0 which means that any sum of pixel weights must be equal to 1.0. So, the filter function must produce a graph of the proper shape.

filterrad : *float > 0, default: 4.0* The filter radius for filters that have a radius parameter, i.e. when interpolation is one of: 'sinc', 'lanczos' or 'blackman'.

resample : *bool, default: “image.resample“* When *True*, use a full resampling method. When *False*, only resample when the output image is larger than the input image.

url : *str, optional* Set the url of the created `AxesImage`. See `Artist.set_url`.

Returns

“AxesImage“

See also: `matshow`

Notes

Unless *extent* is used, pixel centers will be located at integer coordinates. In other words: the origin will coincide with the center of pixel (0, 0).

There are two common representations for RGB images with an alpha channel:

- Straight (unassociated) alpha: R, G, and B channels represent the color of the pixel, disregarding its opacity.
- Premultiplied (associated) alpha: R, G, and B channels represent the color of the pixel, adjusted for its opacity by multiplication.

`imshow` expects RGB images adopting the straight (unassociated) alpha representation.

matplotlib.axes.Axes.fill_between

```
matplotlib.axes.Axes.fill_between(self, x, y1, y2=0, where=None, interpolate=False, step=None, *,
↪  data=None, **kwargs)
```

Fill the area between two horizontal curves.

The curves are defined by the points (*x*, *y1*) and (*x*, *y2*). This creates one or multiple polygons describing the filled area.

You may exclude some horizontal sections from filling using *where*.

By default, the edges connect the given points directly. Use *step* if the filling should be a step function, i.e. constant in between *x*.

Parameters

`x` : *array-like* The x coordinates of the nodes defining the curves.

`y1` : *array-like or float* The y coordinates of the nodes defining the first curve.

`y2` : *array-like or float, default: 0* The y coordinates of the nodes defining the second curve.

`where` : *array-like of bool, optional* Define *where* to exclude some horizontal regions from being filled. The filled regions are defined by the coordinates `x[where]`. More precisely, fill between `x[i]` and `x[i+1]` if `where[i] and where[i+1]`. Note that this definition implies that an isolated *True* value between two *False* values in *where* will not result in filling. Both sides of the *True* position remain unfilled due to the adjacent *False* values.

`interpolate` : *bool, default: False* This option is only relevant if *where* is used and the two curves are crossing each other.

Semantically, *where* is often used for *y1* > *y2* or similar. By default, the nodes of the polygon defining the filled region will only be placed at the positions in the *x* array. Such a polygon cannot describe the above semantics close to the intersection. The x-sections containing the intersection are simply clipped.

Setting *interpolate* to *True* will calculate the actual intersection point and extend the filled region up to this point.

`step` : *{'pre', 'post', 'mid'}, optional* Define *step* if the filling should be a step function, i.e. constant in between *x*. The value determines where the step will occur:

- 'pre': The y value is continued constantly to the left from every *x* position, i.e. the interval `(x[i-1], x[i]]` has the value `y[i]`.
- 'post': The y value is continued constantly to the right from every *x* position, i.e. the interval `[x[i], x[i+1])` has the value `y[i]`.
- 'mid': Steps occur half-way between the *x* positions.

Returns

"FillBetweenPolyCollection" A `FillBetweenPolyCollection` containing the plotted polygons.

See also: `fill_between`, `fill_betweenx`

matplotlib.axes.Axes.errorbar

```
matplotlib.axes.Axes.errorbar(self, x, y, yerr=None, xerr=None, fmt='', *, ecolor=None, elinewidth=None,
↪  capsize=None, barsabove=False, lolims=False, uplims=False, xlolims=False, xuplims=False,
↪  errorevery=1, capthick=None, elinestyle=None, data=None, **kwargs)
```

Plot y versus x as lines and/or markers with attached errorbars.

x, *y* define the data locations, *xerr*, *yerr* define the errorbar sizes. By default, this draws the data markers/lines as well as the errorbars. Use fmt='none' to draw errorbars without any data markers.

New in version 3.7. Caps and error lines are drawn in polar coordinates on polar plots.

Parameters

`x, y` : ***float or array-like*** The data positions.

`xerr, yerr` : ***float or array-like, shape(N,) or shape(2, N), optional*** The errorbar sizes:

- scalar: Symmetric +/- values for all data points.
- shape(N,): Symmetric +/-values for each data point.
- shape(2, N): Separate - and + values for each bar. First row contains the lower errors, the second row contains the upper errors.
- *None*: No errorbar.

All values must be >= 0.

See /gallery/statistics/errorbar_features for an example on the usage of `xerr` and `yerr`.

`fmt` : ***str, default: ''*** The format for the data points / data lines. See `plot` for details.

Use 'none' (case-insensitive) to plot errorbars without any data markers.

`ecolor` : ***"color", default: None*** The color of the errorbar lines. If None, use the color of the line connecting the markers.

`elinewidth` : ***float, default: None*** The linewidth of the errorbar lines. If None, the linewidth of the current style is used.

`elinestyle` : ***str or tuple, default: 'solid'*** The linestyle of the errorbar lines. Valid values for linestyles include {'-', '--', '-.', ':', '', (offset, on-off-seq)}. See `Line2D.set_linestyle` for a complete description.

`capsize` : ***float, default: "errorbar.capsize"*** The length of the error bar caps in points.

`capthick` : ***float, default: None*** An alias to the keyword argument *markeredgewidth* (a.k.a. *mew*). This setting is a more sensible name for the property that controls the thickness of the error bar cap in points. For backwards compatibility, if *mew* or *markeredgewidth* are given, then they will over-ride *capthick*. This may change in future releases.

`barsabove` : ***bool, default: False*** If True, will plot the errorbars above the plot symbols. Default is below.

`lolims, uplims, xlolims, xuplims` : ***bool or array-like, default: False*** These arguments can be used to indicate that a value gives only upper/lower limits. In that case a caret symbol is used to indicate this. *lims*-arguments may be scalars, or array-likes of the same length as *xerr* and *yerr*. To use limits with inverted axes, `set_xlim` or `set_ylim` must be called before `errorbar`. Note the tricky parameter names: setting e.g. *lolims* to True means that the y-value is a *lower* limit of the True value, so, only an *upward*-pointing arrow will be drawn!

`errorevery` : ***int or (int, int), default: 1*** draws error bars on a subset of the data. *errorevery* =N draws error bars on the points (x[::N], y[::N]). *errorevery* =(start, N) draws error bars on the points (x[start::N], y[start::N]). e.g. errorevery=(6, 3) adds error bars to the data at (x[6], x[9], x[12], x[15], ...). Used to avoid overlapping error bars when two series share x-axis values.

Returns

"ErrorbarContainer" The container contains:

- data_line : A `Line2D` instance of x, y plot markers and/or line.
- caplines : A tuple of `Line2D` instances of the error bar caps.

- barlinecols : A tuple of `LineCollection` with the horizontal and vertical error ranges.

Labels, legends and annotation

matplotlib.axes.Axes.set

```
matplotlib.axes.Axes.set(self, *, adjustable=<UNSET>, agg_filter=<UNSET>, alpha=<UNSET>, anchor=<UNSET>,
↪  animated=<UNSET>, aspect=<UNSET>, autoscale_on=<UNSET>, autoscalex_on=<UNSET>, autoscaley_on=<UNSET>,
↪  axes_locator=<UNSET>, axisbelow=<UNSET>, box_aspect=<UNSET>, clip_box=<UNSET>, clip_on=<UNSET>,
↪  clip_path=<UNSET>, facecolor=<UNSET>, forward_navigation_events=<UNSET>, frame_on=<UNSET>,
↪  gid=<UNSET>, in_layout=<UNSET> ...)
```

Set multiple properties at once.

`a.set(a=A, b=B, c=C)`

is equivalent to :

```
a.set_a(A)
a.set_b(B)
a.set_c(C)
```

In addition to the full property names, aliases are also supported, e.g. `set(lw=2)` is equivalent to `set(linewidth=2)`, but it is an error to pass both simultaneously.

The order of the individual setter calls matches the order of parameters in `set()`. However, most properties do not depend on each other so that order is rarely relevant.

Supported properties are

Properties: adjustable: {'box', 'datalim'} agg_filter: a filter function, which takes a (m, n, 3) float array and a dpi value, and returns a (m, n, 3) array and two offsets from the bottom left corner of the image alpha: float or None anchor: (float, float) or {'C', 'SW', 'S', 'SE', 'E', 'NE', ...} animated: bool aspect: {'auto', 'equal'} or float autoscale_on: bool autoscalex_on: unknown autoscaley_on: unknown axes_locator: Callable[[Axes, Renderer], Bbox] axisbelow: bool or 'line' box_aspect: float or None clip_box: `BboxBase` or None clip_on: bool clip_path: Patch or (Path, Transform) or None facecolor or fc: `color` figure: `Figure` or `SubFigure` forward_navigation_events: bool or "auto" frame_on: bool gid: str in_layout: bool label: object mouseover: bool navigate: bool navigate_mode: unknown path_effects: list of `AbstractPathEffect` picker: None or bool or float or callable position: [left, bottom, width, height] or `Bbox` prop_cycle: `Cycler` rasterization_zorder: float or None rasterized: bool sketch_params: (scale: float, length: float, randomness: float) snap: bool or None subplotspec: unknown title: str transform: `Transform` url: str visible: bool xbound: (lower: float, upper: float) xinverted: unknown xlabel: str xlim: (left: float, right: float) xmargin: float greater than -0.5 xscale: unknown xticklabels: unknown xticks: unknown ybound: (lower: float, upper: float) yinverted: unknown ylabel: str ylim: (bottom: float, top: float) ymargin: float greater than -0.5 yscale: unknown yticklabels: unknown yticks: unknown zorder: float

matplotlib.axes.Axes.legend

```
matplotlib.axes.Axes.legend(self, *args, **kwargs)
```

Place a legend on the Axes.

Call signatures:

```
legend()
legend(handles, labels)
```

```
legend(handles=handles)
legend(labels)
```

The call signatures correspond to the following different ways to use this method:

1. Automatic detection of elements to be shown in the legend

The elements to be added to the legend are automatically determined, when you do not pass in any extra arguments.

In this case, the labels are taken from the artist. You can specify them either at artist creation or by calling the `set_label` method on the artist:

```
ax.plot([1, 2, 3], label='Inline label')
ax.legend()
```

or:

```
line, = ax.plot([1, 2, 3])
line.set_label('Label via method')
ax.legend()
```

> Note
> Specific artists can be excluded from the automatic legend element selection by using a label starting with an underscore, "_". A string starting with an underscore is the default label for all artists, so calling `Axes.legend` without any arguments and without setting the labels manually will result in a `UserWarning` and an empty legend being drawn.

2. Explicitly listing the artists and labels in the legend

For full control of which artists have a legend entry, it is possible to pass an iterable of legend artists followed by an iterable of legend labels respectively:

```
ax.legend([line1, line2, line3], ['label1', 'label2', 'label3'])
```

3. Explicitly listing the artists in the legend

This is similar to 2, but the labels are taken from the artists' label properties. Example:

```
line1, = ax.plot([1, 2, 3], label='label1')
line2, = ax.plot([1, 2, 3], label='label2')
ax.legend(handles=[line1, line2])
```

4. Labeling existing plot elements

> Discouraged
> This call signature is discouraged, because the relation between plot elements and labels is only implicit by their order and can easily be mixed up.

To make a legend for all artists on an Axes, call this function with an iterable of strings, one for each legend item. For example:

```
ax.plot([1, 2, 3])
ax.plot([5, 6, 7])
ax.legend(['First line', 'Second line'])
```

Parameters

`handles` : *list of ("Artist" or tuple of "Artist"), optional* A list of Artists (lines, patches) to be added to the legend. Use this together with *labels*, if you need full control on what is shown in the legend and the automatic mechanism described above is not sufficient.

The length of handles and labels should be the same in this case. If they are not, they are truncated to the smaller length.

If an entry contains a tuple, then the legend handler for all Artists in the tuple will be placed alongside a single label.

labels : *list of str, optional* A list of labels to show next to the artists. Use this together with *handles*, if you need full control on what is shown in the legend and the automatic mechanism described above is not sufficient.

Returns

“Legend“

See also: `.Figure.legend`

Notes

Some artists are not supported by this function. See legend_guide for details.

Examples

matplotlib.axes.Axes.annotate

```
matplotlib.axes.Axes.annotate(self, text, xy, xytext=None, xycoords='data', textcoords=None,
↪  arrowprops=None, annotation_clip=None, **kwargs)
```

Annotate the point *xy* with text *text*.

In the simplest form, the text is placed at *xy*.

Optionally, the text can be displayed in another position *xytext*. An arrow pointing from the text to the annotated point *xy* can then be added by defining *arrowprops*.

Parameters

text : *str* The text of the annotation.

xy : *(float, float)* The point *(x, y)* to annotate. The coordinate system is determined by *xycoords*.

xytext : *(float, float), default: xy* The position *(x, y)* to place the text at. The coordinate system is determined by *textcoords*.

xycoords : *single or two-tuple of str or “Artist“ or “Transform“ or callable, default: 'data'* The coordinate system that *xy* is given in. The following types of values are supported:

- One of the following strings:

Value	Description
'figure points'	Points from the lower left of the figure
'figure pixels'	Pixels from the lower left of the figure
'figure fraction'	Fraction of figure from lower left
'subfigure points'	Points from the lower left of the subfigure
'subfigure pixels'	Pixels from the lower left of the subfigure
'subfigure fraction'	Fraction of subfigure from lower left
'axes points'	Points from lower left corner of the Axes
'axes pixels'	Pixels from lower left corner of the Axes
'axes fraction'	Fraction of Axes from lower left
'data'	Use the coordinate system of the object being annotated (default)
'polar'	*(theta, r)* if not native 'data' coordinates

Note that 'subfigure pixels' and 'figure pixels' are the same for the parent figure, so users who want code that is usable in a subfigure can use 'subfigure pixels'.

- An `Artist`: *xy* is interpreted as a fraction of the artist's `Bbox`. E.g. *(0, 0)* would be the lower left corner of the bounding box and *(0.5, 1)* would be the center top of the bounding box.
- A `Transform` to transform *xy* to screen coordinates.
- A function with one of the following signatures:

  ```
  def transform(renderer) -> Bbox
  def transform(renderer) -> Transform
  ```

  where *renderer* is a `RendererBase` subclass.

  The result of the function is interpreted like the `Artist` and `Transform` cases above.
- A tuple *(xcoords, ycoords)* specifying separate coordinate systems for *x* and *y*. *xcoords* and *ycoords* must each be of one of the above described types.

See plotting-guide-annotation for more details.

`textcoords` : *single or two-tuple of str or "Artist" or "Transform" or callable, default: value of xycoo*

The coordinate system that *xytext* is given in.

All *xycoords* values are valid as well as the following strings:

Value	Description
'offset points'	Offset, in points, from the *xy* value
'offset pixels'	Offset, in pixels, from the *xy* value
'offset fontsize'	Offset, relative to fontsize, from the *xy* value

`arrowprops` : *dict, optional* The properties used to draw a `FancyArrowPatch` arrow between the positions *xy* and *xytext*. Defaults to None, i.e. no arrow is drawn.

For historical reasons there are two different ways to specify arrows, "simple" and "fancy":

Simple arrow:

If *arrowprops* does not contain the key 'arrowstyle' the allowed keys are:

Key	Description
width	The width of the arrow in points
headwidth	The width of the base of the arrow head in points
headlength	The length of the arrow head in points
shrink	Fraction of total length to shrink from both ends
?	Any `FancyArrowPatch` property

The arrow is attached to the edge of the text box, the exact position (corners or centers) depending on where it's pointing to.

Fancy arrow:

This is used if 'arrowstyle' is provided in the *arrowprops*.

Valid keys are the following `FancyArrowPatch` parameters:

Key	Description
arrowstyle	The arrow style
connectionstyle	The connection style

Key	Description
relpos	See below; default is (0.5, 0.5)
patchA	Default is bounding box of the text
patchB	Default is None
shrinkA	In points. Default is 2 points
shrinkB	In points. Default is 2 points
mutation_scale	Default is text size (in points)
mutation_aspect	Default is 1
?	Any `FancyArrowPatch` property

The exact starting point position of the arrow is defined by *relpos*. It's a tuple of relative coordinates of the text box, where (0, 0) is the lower left corner and (1, 1) is the upper right corner. Values <0 and >1 are supported and specify points outside the text box. By default (0.5, 0.5), so the starting point is centered in the text box.

`annotation_clip` : *bool or None, default: None* Whether to clip (i.e. not draw) the annotation when the annotation point *xy* is outside the Axes area.

- If *True*, the annotation will be clipped when *xy* is outside the Axes.
- If *False*, the annotation will always be drawn.
- If *None*, the annotation will be clipped when *xy* is outside the Axes and *xycoords* is 'data'.

****`kwargs`** Additional kwargs are passed to `Text`.

Returns

"Annotation"

See also: `annotations`

matplotlib.axes.Axes.text

```
matplotlib.axes.Axes.text(self, x, y, s, fontdict=None, **kwargs)
```

Add text to the Axes.

Add the text *s* to the Axes at location *x*, *y* in data coordinates, with a default `horizontalalignment` on the `left` and `verticalalignment` at the `baseline`. See /gallery/-text_labels_and_annotations/text_alignment.

Parameters

`x, y` : *float* The position to place the text. By default, this is in data coordinates. The coordinate system can be changed using the *transform* parameter.

`s` : *str* The text.

`fontdict` : *dict, default: None*

> Discouraged
> The use of *fontdict* is discouraged. Parameters should be passed as individual keyword arguments or using dictionary-unpacking `text(..., **fontdict)`.

A dictionary to override the default text properties. If fontdict is None, the defaults are determined by `rcParams`.

Returns

"Text" The created `Text` instance.

Examples

Individual keyword arguments can be used to override any given parameter:

```
>>> text(x, y, s, fontsize=12)
```

The default transform specifies that text is in data coords, alternatively, you can specify text in axis coords ((0, 0) is lower-left and (1, 1) is upper-right). The example below places text in the center of the Axes:

```
>>> text(0.5, 0.5, 'matplotlib', horizontalalignment='center',
...      verticalalignment='center', transform=ax.transAxes)
```

You can put a rectangular box around the text instance (e.g., to set a background color) by using the keyword *bbox*. *bbox* is a dictionary of `Rectangle` properties. For example:: ...

matplotlib.axes.Axes.grid

```
matplotlib.axes.Axes.grid(self, visible=None, which='major', axis='both', **kwargs)
```

Configure the grid lines.

Parameters

`visible` : *bool or None, optional* Whether to show the grid lines. If any *kwargs* are supplied, it is assumed you want the grid on and *visible* will be set to True.

If *visible* is *None* and there are no *kwargs*, this toggles the visibility of the lines.

`which` : *{'major', 'minor', 'both'}, optional* The grid lines to apply the changes on.

`axis` : *{'both', 'x', 'y'}, optional* The axis to apply the changes on.

`**kwargs` : *"Line2D" properties* Define the line properties of the grid, e.g.:

```
grid(color='r', linestyle='-', linewidth=2)
```

Valid keyword arguments are:

Properties: agg_filter: a filter function, which takes a (m, n, 3) float array and a dpi value, and returns a (m, n, 3) array and two offsets from the bottom left corner of the image alpha: float or None animated: bool antialiased or aa: bool clip_box: `BboxBase` or None clip_on: bool clip_path: Patch or (Path, Transform) or None color or c: `color` dash_capstyle: `CapStyle` or {'butt', 'projecting', 'round'} dash_joinstyle: `JoinStyle` or {'miter', 'round', 'bevel'} dashes: sequence of floats (on/off ink in points) or (None, None) data: (2, N) array or two 1D arrays drawstyle or ds: {'default', 'steps', 'steps-pre', 'steps-mid', 'steps-post'}, default: 'default' figure: `Figure` or `SubFigure` fillstyle: {'full', 'left', 'right', 'bottom', 'top', 'none'} gapcolor: `color` or None gid: str in_layout: bool label: object linestyle or ls: {'-', '--', '-.', ':', '', ...} or (offset, on-off-seq) linewidth or lw: float marker: marker style string, `Path` or `MarkerStyle` markeredgecolor or mec: `color` markeredgewidth or mew: float markerfacecolor or mfc: `color` markerfacecoloralt or mfcalt: `color` markersize or ms: float markevery: None or int or (int, int) or slice or list[int] or float or (float, float) or list[bool] mouseover: bool path_effects: list of `AbstractPathEffect` picker: float or callable[[Artist, Event], tuple[bool, dict]] pickradius: float rasterized: bool sketch_params: (scale: float, length: float, randomness: float) snap: bool or None solid_capstyle: `CapStyle` or {'butt', 'projecting', 'round'} solid_joinstyle: `JoinStyle` or {'miter', 'round', 'bevel'} transform: unknown url: str visible: bool xdata: 1D array ydata: 1D array zorder: float

Notes

The axis is drawn as a unit, so the effective zorder for drawing the grid is determined by the zorder of each axis, not by the zorder of the `Line2D` objects comprising the grid. Therefore, to set grid zorder, use `set_axisbelow` or, for more control, call the `set_zorder` method of each axis.

matplotlib.axes.Axes.twinx

```
matplotlib.axes.Axes.twinx(self, axes_class=None, *, delta_zorder=0.0, **kwargs)
```

Create a twin Axes sharing the xaxis.

Create a new Axes with an invisible x-axis and an independent y-axis positioned opposite to the original one (i.e. at right). The x-axis autoscale setting will be inherited from the original Axes. To ensure that the tick marks of both y-axes align, see `LinearLocator`.

Parameters

`axes_class` : ***subclass type of "Axes", optional*** The `axes.Axes` subclass that is instantiated. This parameter is incompatible with *projection* and *polar*. See axisartist_users-guide-index for examples.

By default, `Axes` is used.

New in version 3.11.

`delta_zorder` : ***float, default: 0*** A zorder offset for the twin Axes, relative to the original Axes. The twin's zorder is set to `self.get_zorder() + delta_zorder`. By default (*delta_zorder* is 0), the twin has the same zorder as the original Axes.

`kwargs` : ***dict*** The keyword arguments passed to `Figure.add_subplot` or `Figure.add_axes`.

New in version 3.11.

Returns

Axes The newly created Axes instance

Notes

For those who are 'picking' artists while using twinx, pick events are only called for the artists in the top-most Axes.

Figures, layout and colors

matplotlib.figure.Figure.colorbar

```
matplotlib.figure.Figure.colorbar(self, mappable, cax=None, ax=None, use_gridspec=True, **kwargs)
```

Add a colorbar to a plot.

Parameters

`mappable` The `matplotlib.colorizer.ColorizingArtist` (i.e., `AxesImage`, `ContourSet`, etc.) described by this colorbar. This argument is mandatory for the `Figure.colorbar` method but optional for the `pyplot.colorbar` function, which sets the default to the current image.

Note that one can create a `colorizer.ColorizingArtist` "on-the-fly" to generate colorbars not attached to a previously drawn artist, e.g. :

```
cr = colorizer.Colorizer(norm=norm, cmap=cmap)
fig.colorbar(colorizer.ColorizingArtist(cr), ax=ax)
```

cax : *“Axes“, optional* Axes into which the colorbar will be drawn. If `None`, then a new Axes is created and the space for it will be stolen from the Axes(s) specified in *ax*.

ax : *“Axes“ or iterable or “numpy.ndarray“ of Axes, optional* The one or more parent Axes from which space for a new colorbar Axes will be stolen. This parameter is only used if *cax* is not set.

Defaults to the Axes that contains the mappable used to create the colorbar.

use_gridspec : *bool, optional* If *cax* is `None`, a new *cax* is created as an instance of Axes. If *ax* is positioned with a subplotspec and *use_gridspec* is `True`, then *cax* is also positioned with a subplotspec.

Returns

colorbar : *“Colorbar“*

Notes

If *mappable* is a `ContourSet`, its *extend* kwarg is included automatically.

The *shrink* kwarg provides a simple way to scale the colorbar with respect to the Axes. Note that if *cax* is specified, it determines the size of the colorbar, and *shrink* and *aspect* are ignored.

For more precise control, you can manually specify the positions of the axes objects in which the mappable and the colorbar are drawn. In this case, do not use any of the Axes properties kwargs.

(Notes truncated; see the full reference page.)

matplotlib.figure.Figure.savefig

```
matplotlib.figure.Figure.savefig(self, fname, *, transparent=None, **kwargs)
```

Save the current figure as an image or vector graphic to a file.

Call signature:

```
savefig(fname, *, transparent=None, dpi='figure', format=None,
        metadata=None, bbox_inches=None, pad_inches=0.1,
        facecolor='auto', edgecolor='auto', backend=None,
        **kwargs
       )
```

The available output formats depend on the backend being used.

Parameters

fname : *str or path-like or binary file-like* A path, or a Python file-like object, or possibly some backend-dependent object such as `matplotlib.backends.backend_pdf.PdfPages`.

If *format* is set, it determines the output format, and the file is saved as *fname*. Note that *fname* is used verbatim, and there is no attempt to make the extension, if any, of *fname* match *format*, and no extension is appended.

If *format* is not set, then the format is inferred from the extension of *fname*, if there is one. If *format* is not set and *fname* has no extension, then the file is saved with `savefig.format` and the appropriate extension is appended to *fname*.

matplotlib.colors.Normalize

```
matplotlib.colors.Normalize(vmin=None, vmax=None, clip=False)
```

A class which, when called, maps values within the interval `[vmin, vmax]` linearly to the interval `[0.0, 1.0]`. The mapping of values outside `[vmin, vmax]` depends on *clip*.

See also: `colormapnorms`

Examples

:

```
x = [-2, -1, 0, 1, 2]
norm = mpl.colors.Normalize(vmin=-1, vmax=1, clip=False) norm(x) # [-0.5, 0., 0.5, 1., 1.5]
norm = mpl.colors.Normalize(vmin=-1, vmax=1, clip=True) norm(x) # [0., 0., 0.5, 1., 1.]
```

matplotlib.pyplot.get_cmap

```
matplotlib.pyplot.get_cmap(name: 'Colormap | str | None' = None, lut: 'int | None' = None) -> 'Colormap'
```

Get a colormap instance, defaulting to rc values if *name* is None.

Parameters

`name` : ***"Colormap" or str or None, default: None*** If a `Colormap` instance, it will be returned. Otherwise, the name of a colormap known to Matplotlib, which will be resampled by *lut*. The default, None, means `image.cmap`.

`lut` : ***int or None, default: None*** If *name* is not already a Colormap instance and *lut* is not None, the colormap will be resampled to have *lut* entries in the lookup table.

Returns

Colormap